Database Management System

BCS-501 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** write each 7-mark answer in this order: definition, diagram, numbered points, small table, advantages and disadvantages, conclusion. **Bold words are the keywords examiners look for.** Always draw the diagram.

UNIT 1: ARCHITECTURE & ER MODELLING

Section A: 2-Mark Questions

Q1. What is a Data Dictionary? *[2 Marks]*

Ans. A **data dictionary** is a central store of **metadata** ("data about data"). It records table names, attributes, data types, constraints, indexes, users and access rights. An **active** dictionary is updated automatically by the DBMS. Example: `INFORMATION_SCHEMA` in MySQL.

Q2. Differentiate between Schema and Instance. *[2 Marks]*

Ans.

Schema	Instance
Logical design (structure) of the database	**Actual data** at a particular moment
Defined once with DDL; rarely changes	Changes with every insert, update or delete
Example: STUDENT(Roll, Name, Age)	Example: (101, 'Harsh', 21)

Q3. State the responsibilities of a DBA. *[2 Marks]*

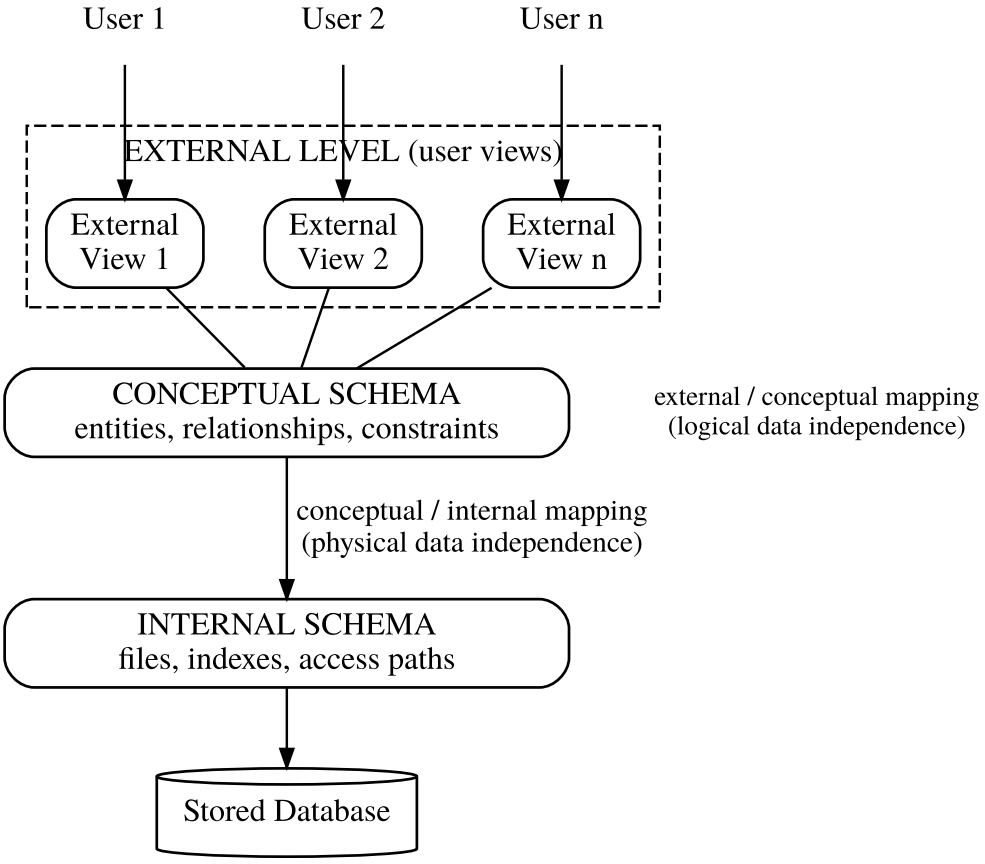
Ans. The **Database Administrator** is responsible for:

1. **Schema definition** (with DDL).
2. **Storage structure** and access methods (indexes).
3. **Granting authorization** (GRANT / REVOKE).
4. **Backup and recovery**.
5. **Integrity constraints**.
6. **Performance tuning**.

Section B: 7-Mark Questions

Q4. Explain the three-schema architecture and data independence. *[7 Marks]*

Ans. The **three-schema (ANSI/SPARC) architecture** divides a database into **three levels of abstraction**: **External, Conceptual and Internal**. Its main aim is **data independence**, which means the user's view is separated from physical storage.



Three-schema (ANSI/SPARC) architecture

Levels

1. **Internal level: how** data is stored, including files, indexes and access paths. Example: STUDENT stored in a B+ tree on Roll_No.
2. **Conceptual level: what** data is stored for the whole database: entities, relationships and constraints. It is designed by the **DBA**.
3. **External level:** the **user views**. Each user group sees only the part it needs. Example: Accounts sees (Roll, Fee_Status).

Data independence

- **Physical data independence:** the internal schema can change **without changing the conceptual schema**. Example: adding an index or changing disks.
- **Logical data independence:** the conceptual schema can change **without changing the external views**. Example: adding a column. This is **harder** to achieve.

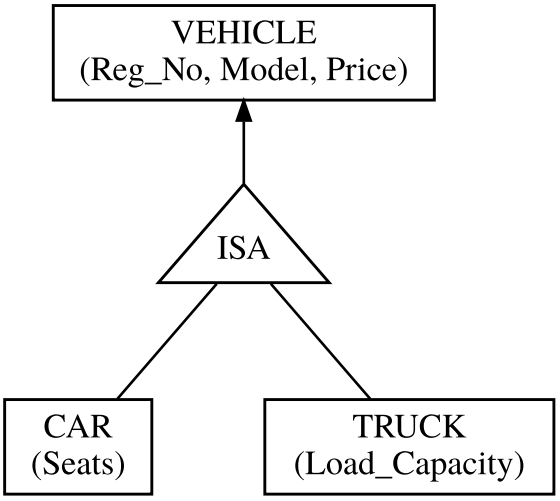
Physical DI	Logical DI
Change at the internal level	Change at the conceptual level
Conceptual/internal mapping updated	External/conceptual mapping updated
Easy	**Difficult**

Advantages: data independence, security through views, easier maintenance. **Disadvantages:** mapping overhead, complex design.

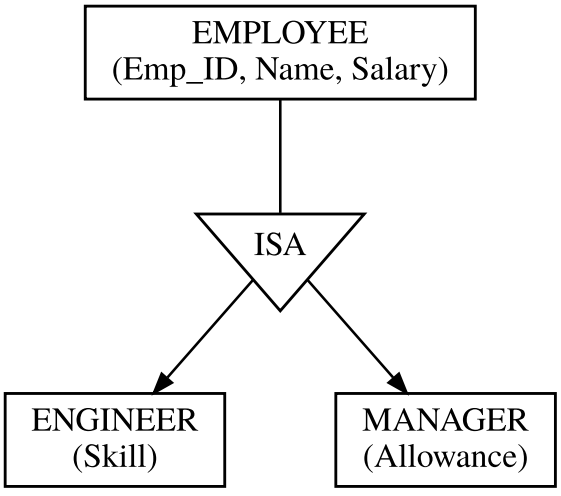
Conclusion: every RDBMS (Oracle, MySQL) follows this model: **views** are the external level and **tablespaces/indexes** are the internal level.

Q5. Differentiate Generalization, Specialization and Aggregation with examples. *[7 Marks]*

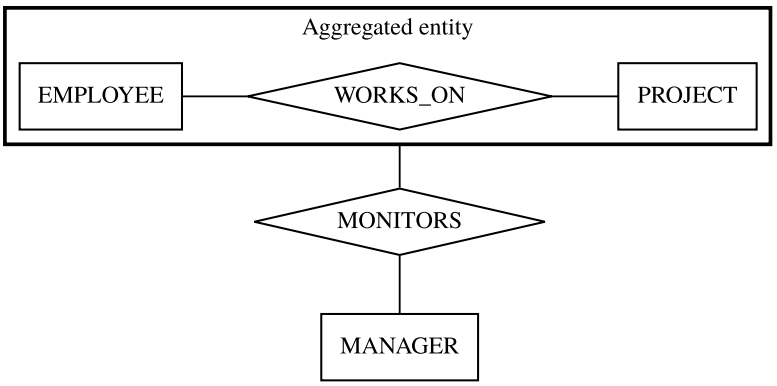
Ans. These are **Enhanced ER (EER)** concepts. Generalization and specialization use the **IS-A** relationship (inheritance). **Aggregation** treats a relationship as an entity.



Generalization (bottom-up): common attributes move up to VEHICLE



Specialization (top-down): EMPLOYEE split into sub-classes



Aggregation: the relationship (EMPLOYEE works on PROJECT) is treated as one entity

1. **Generalization (bottom-up):** lower-level entities with common attributes are combined into a **super-class**. Example: CAR + TRUCK → VEHICLE.
2. **Specialization (top-down):** an entity is split into **sub-classes** that inherit its attributes. Example: EMPLOYEE → ENGINEER, MANAGER.
3. **Aggregation:** a **relationship set is treated as a higher-level entity**, so it can take part in another relationship. Example: (EMPLOYEE works on PROJECT) is **monitored by** MANAGER.
4. **Constraints: disjoint / overlapping**, and **total / partial** participation.

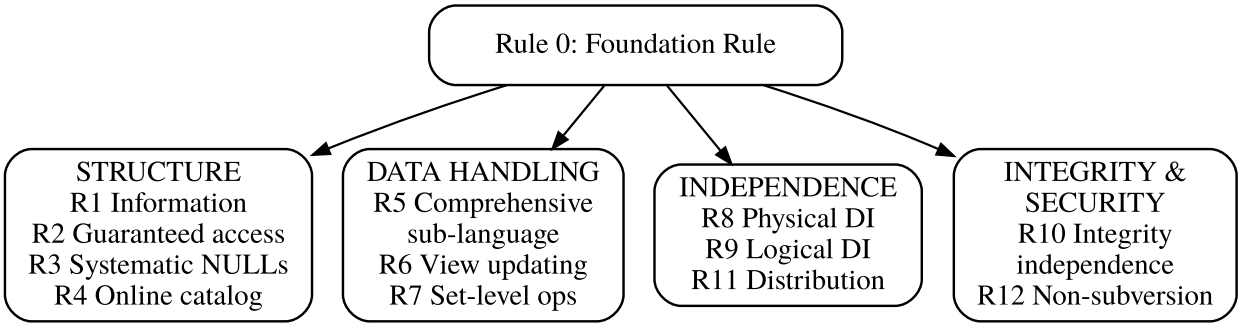
Basis	Generalization	Specialization	Aggregation
Approach	Bottom-up	Top-down	Abstraction of a relationship
Result	Fewer entities (super-class)	More entities (sub-classes)	Relationship becomes an entity
Relation type	IS-A	IS-A	HAS-A

Conclusion: used to model **ERP, banking and university** databases with entity hierarchies.

Q6. Explain Codd's 12 rules.

[7 Marks]

Ans. Dr. E.F. Codd (1985) gave **13 rules (Rule 0 to Rule 12)** that a DBMS must follow to be **fully relational**.



Codd's rules grouped by purpose

Rule	Name	Meaning
0	Foundation	Manage data **only through relational** capabilities
1	Information	All data, including metadata, is stored **in tables**
2	Guaranteed access	Any value = **table + primary key + column**
3	NULL treatment	NULL handled uniformly as missing data
4	Online catalog	Metadata stored as tables and queried with SQL
5	Data sub-language	One complete language (**SQL**) for DDL, DML, security
6	View updating	Theoretically updatable views must be updatable
7	Set-level operations	Insert / update / delete on **sets of rows**
8	Physical DI	Storage changes do not affect applications
9	Logical DI	Table changes do not affect applications
10	Integrity independence	Constraints stored in the catalog, not in programs
11	Distribution independence	Users unaware of data distribution
12	Non-subversion	Low-level access cannot **bypass** the rules

> **Exam Tip:** no commercial RDBMS satisfies all 12 rules fully. Most satisfy 8 to 10.

Conclusion: Codd's rules are the **benchmark** for judging how relational a DBMS is.

UNIT 2: RELATIONAL ALGEBRA & CALCULUS

Section A: 2-Mark Questions

Q1. Define Cartesian product and Theta join. *[2 Marks]*

Ans.

- **Cartesian product (R × S):** combines **every tuple of R with every tuple of S**. If R has m tuples and S has n, the result has **m × n tuples**.
- **Theta join:** $R \bowtie_\theta S = \sigma_\theta (R \times S)$, a join on any condition θ (<, >, =, ≠). With only '=', it is an **equi-join**.

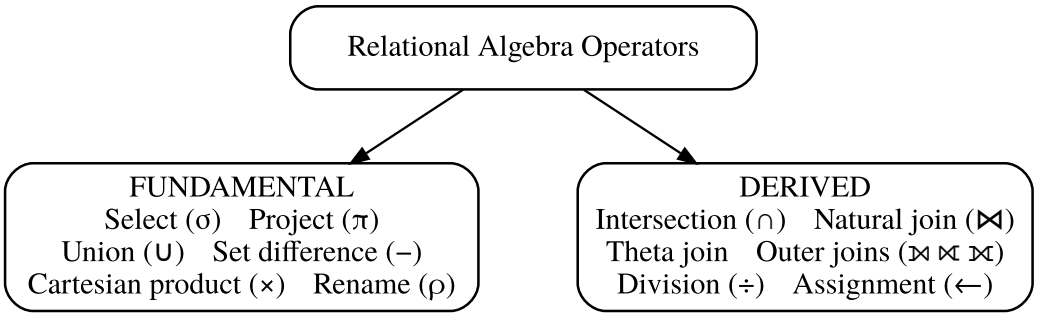
Q2. How do foreign keys cause cascading deletes? *[2 Marks]*

Ans. If a foreign key is declared with **ON DELETE CASCADE**, deleting a parent row **automatically deletes all the child rows** that refer to it. This keeps referential integrity, with no orphan rows. `FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID) ON DELETE CASCADE`

Section B: 7-Mark Questions

Q3. Explain relational algebra operators: natural join, outer joins and division. *[7 Marks]*

Ans. Relational algebra is a **procedural** query language. Its operators take relations as input and give a relation as output.



Classification of relational algebra operators

Fundamental operators:

- Select σ (rows)
- Project π (columns)
- Union ∪
- Set difference −
- Cartesian product ×
- Rename ρ

Example tables: EMP(EID, Name, DID) = {(1, Amit, 10), (2, Riya, 20), (3, Karan, 40)}, DEPT(DID, DName) = {(10, CSE), (20, IT), (30, ECE)}

1. **Natural join (EMP ⋈ DEPT):** equality on **common attributes** (DID), with duplicate columns removed. Result: (1, Amit, 10, CSE), (2, Riya, 20, IT).
2. **Outer joins:** keep **unmatched tuples** and fill the gaps with **NULL**.
   - **Left (⟕):** also gives (3, Karan, 40, NULL).
   - **Right (⟖):** also gives (NULL, NULL, 30, ECE).
   - **Full (⟗):** both of the above.
3. **Division (R ÷ S):** used for **"for all"** queries. ENROLLED = {(A,C1), (A,C2), (B,C1), (C,C1), (C,C2), (C,C3)}, COURSE = {C1, C2}.

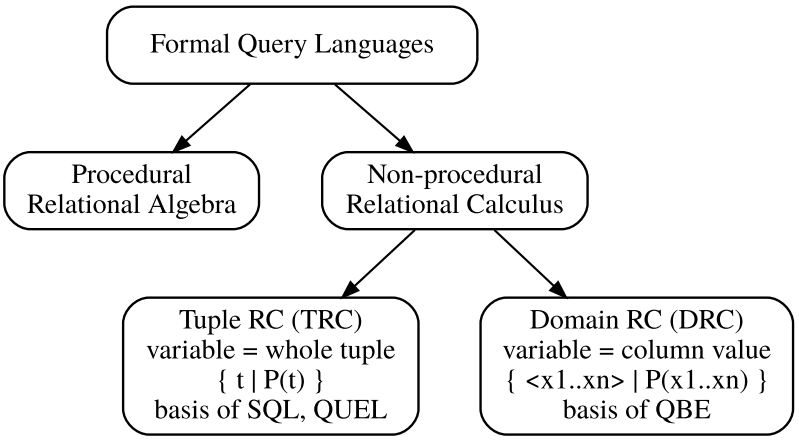
$$R \div S = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R) = \{A, B, C\} - \{B\} = \{A, C\}$$

Join	Unmatched tuples	Duplicate columns
Theta / Equi	Dropped	Kept
Natural	Dropped	Removed
Outer (L / R / Full)	**Kept with NULL**	Removed

Conclusion: relational algebra is used **inside the query optimizer** to build execution plans for SQL.

Q4. Differentiate between TRC and DRC with syntax. *[7 Marks]*

Ans. Relational calculus is **non-procedural**: the user says *what* is wanted, not *how*. In **TRC** variables are **tuples**. In **DRC** variables are **column values (domains)**.



Formal query languages

Example relation: STUDENT(Roll, Name, Age, Dept)

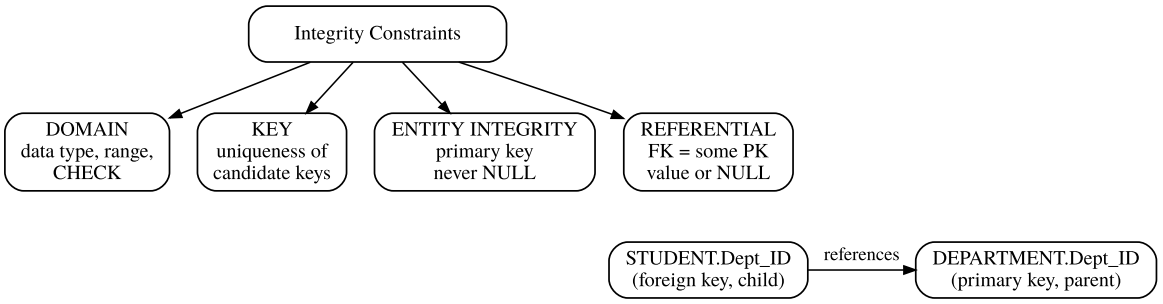
- **TRC:** names of students older than 20: $\{ t.\text{Name} \mid t \in \text{STUDENT} \wedge t.\text{Age} > 20 \}$
- **DRC:** the same query: $\{ \langle n \rangle \mid \exists r, a, d \ (\langle r, n, a, d \rangle \in \text{STUDENT} \wedge a > 20) \}$
- **Safety:** $\{t \mid \neg (t \in R)\}$ is **unsafe**, because it gives an infinite result. Safe TRC = safe DRC = relational algebra (**relational completeness**).

Basis	TRC	DRC
Variable	Whole **tuple**	Single **attribute value**
Syntax	{ t | P(t) }	$\{ \langle x_1 \ldots x_n \rangle \mid P(x_1 \ldots x_n) \}$
Attribute access	t.Name	positional ⟨r, n, a, d⟩
Language based on it	**SQL**, QUEL	**QBE**

Conclusion: TRC is the basis of SQL's SELECT-FROM-WHERE, and DRC is the basis of QBE (MS Access).

Q5. Explain domain, entity and referential integrity with examples. *[7 Marks]*

Ans. Integrity constraints are rules that keep data **valid and consistent**. They are enforced automatically by the DBMS.



Types of integrity constraints, and the foreign-key link

1. **Domain constraint:** every value must come from its **domain** (type, range). Example: `Age INT CHECK (Age BETWEEN 17 AND 30)`, so Age = 'abc' or 45 is **rejected**.
2. **Key constraint:** candidate key values must be **unique**.
3. **Entity integrity: a primary key can never be NULL**. Example: (NULL, 'Aman') is rejected.
4. **Referential integrity:** a **foreign key must match a primary key value, or be NULL**. Example: STUDENT.Dept_ID = 50 is rejected if 50 is not in DEPARTMENT.
5. **On violation:** RESTRICT, **CASCADE**, SET NULL or SET DEFAULT.

Constraint	Applies to	Rule	Violated by
Domain	Every attribute	Valid type / range	Insert, Update
Entity	Primary key	PK ≠ NULL	Insert, Update
Referential	Foreign key	FK = PK value or NULL	Insert (child), Delete (parent)

Conclusion: constraints prevent **orphan records and invalid data** in banking and ERP systems.

UNIT 3: DATABASE DESIGN & NORMALIZATION

Section A: 2-Mark Questions

Q1. Define insertion and update anomalies. *[2 Marks]*

Ans.

- **Insertion anomaly:** a fact cannot be inserted without unrelated data. Example: a new department cannot be added until a student joins it.
- **Update anomaly:** the same fact is stored in many rows, so a change must be repeated everywhere, otherwise the data becomes **inconsistent**.
- *(Deletion anomaly:* deleting one fact loses another.)

Q2. Trivial vs. non-trivial FD; Armstrong's axioms. *[2 Marks]*

Ans.

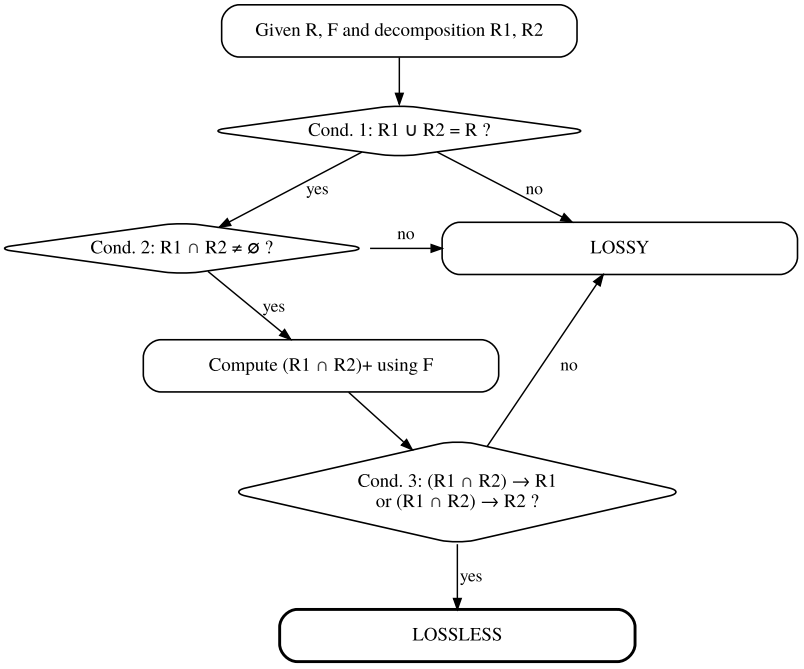
- **Trivial FD:** $X \rightarrow Y$ with $Y \subseteq X$ (for example {Roll, Name} → Name).
- **Non-trivial FD:** $Y \nsubseteq X$ (for example Roll → Name).

- **Armstrong's axioms:**
  - **Reflexivity:** if $Y \subseteq X$ then $X \to Y$.
  - **Augmentation:** if $X \to Y$ then $XZ \to YZ$.
  - **Transitivity:** if $X \to Y$ and $Y \to Z$ then $X \to Z$.

Section B: 7-Mark Questions

Q3. Prove whether a decomposition is lossless, and explain dependency preservation. *[7 Marks]*

Ans. A decomposition of R into R1 and R2 is **lossless** if $R_1 \bowtie R_2 = R$, that is, the join produces no **spurious tuples**.



Lossless-join test for a binary decomposition

Test: (1) $R_1 \cup R_2 = R$, (2) $R_1 \cap R_2 \neq \emptyset$, (3) $(R_1 \cap R_2) \to R_1$ or $(R_1 \cap R_2) \to R_2$.

Example: R(A, B, C, D, E), F = {A → BC, CD → E, B → D, E → A}, R1(A, B, C), R2(A, D, E)

1. $R_1 \cup R_2 = ABCDE = R$ ✓
2. $R_1 \cap R_2 = \{A\} \neq \emptyset$ ✓
3. A^+: A → BC gives ABC; B → D gives ABCD; CD → E gives **ABCDE**. So A → R1 ✓

Result: LOSSLESS.

Dependency preservation: the decomposition is preserving if $(F_1 \cup F_2)^+ = F^+$.

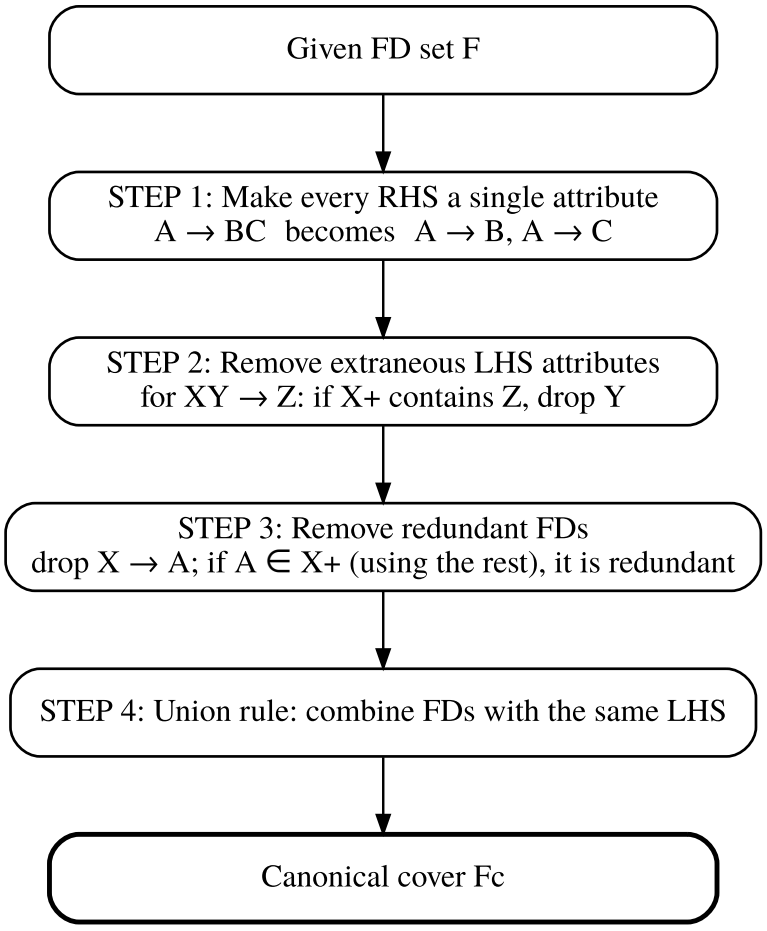
- F1 = {A → BC, BC → A} and F2 = {A → DE, E → A}.
- B → D: $B^+ = \{B\}$, which does not contain D, so it is **not preserved**.
- **Result: lossless but NOT dependency preserving.**

Lossy example: R(A, B, C) decomposed into R1(A, B) and R2(B, C), with only A → B. Then $B^+ = B$ is not a key of either part, so the join gives spurious tuples.

Conclusion: lossless join is **mandatory**. Dependency preservation is desirable (3NF always gives both; BCNF may lose dependencies).

Q4. Find the canonical cover of F = {A → B, C → B, D → ABC, AC → D}. *[7 Marks]*

Ans. A **canonical cover Fc** is the **minimal equivalent set** of FDs: single-attribute RHS, no extraneous attributes and no redundant FDs.



Algorithm for finding the canonical cover

Step 1: Split the RHS.

$$F = \{A \to B,\ C \to B,\ D \to A,\ D \to B,\ D \to C,\ AC \to D\}$$

Step 2: Extraneous LHS attributes in AC → D.

- $A^+ = AB$ does not contain D, so C is needed.
- $C^+ = CB$ does not contain D, so A is needed. **AC → D stays.**

Step 3: Remove redundant FDs.

FD	Closure without this FD	Redundant?
A → B	$A^+ = A$	No
C → B	$C^+ = C$	No
D → A	$D^+ = DBC$	No
D → B	$D^+ = DACB$ (via D → A, A → B)	**Yes, remove**
D → C	$D^+ = DAB$	No
AC → D	$AC^+ = ACB$	No

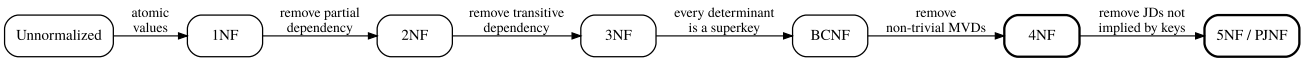
Step 4: Union. D → A and D → C combine to D → AC.

$$F_c = \{A \to B,\ C \to B,\ D \to AC,\ AC \to D\}$$

Conclusion: the canonical cover is the input to **3NF synthesis** and makes FD checking cheap.

Q5. Explain MVD and 4NF, and join dependency and 5NF. *[7 Marks]*

Ans. A **multivalued dependency** $X \twoheadrightarrow Y$ means that one X value has a **set of Y values independent of the other attributes**. **4NF** removes MVDs. A **join dependency** means the relation can be rebuilt **only by joining 3 or more projections**. **5NF** removes such JDs.



Normal forms: each step removes one kind of dependency

4NF example: STUDENT(SID, Course, Hobby). Courses and hobbies are independent, so SID →→ Course and SID →→ Hobby.

- S1 with 2 courses and 2 hobbies needs **4 rows**. This is redundant.
- **4NF rule:** for every non-trivial X →→ Y, **X must be a superkey**. SID is not, so the relation is not in 4NF.
- **Decompose** into (SID, Course) and (SID, Hobby).

5NF example: SPJ(Supplier, Part, Project) = {S1P1J2, S1P2J1, S2P1J1, S1P1J1}.

- SP ⋈ PJ gives a spurious tuple S2P1J2. Joining also with JS removes it.
- So $*\,(SP, PJ, JS)$ holds. **Decompose into 3 tables** (SP, PJ, JS) to reach 5NF.

Basis	4NF	5NF
Removes	MVD	Join dependency
Pre-condition	BCNF	4NF
Splits into	2 tables	3 or more tables

Conclusion: 4NF and 5NF remove the redundancy that BCNF cannot. They are used in supply-chain style schemas.

UNIT 4: TRANSACTIONS & CONCURRENCY CONTROL

Section A: 2-Mark Questions

Q1. Dirty read and unrepeatable read. *[2 Marks]*

Ans.

- **Dirty read (W-R conflict):** T2 reads data written by T1 **before T1 commits**. If T1 aborts, T2 has used an invalid value.
- **Unrepeatable read (R-W conflict):** T1 reads A twice and gets **different values**, because T2 updated and committed A in between.

Q2. Starvation vs. deadlock. *[2 Marks]*

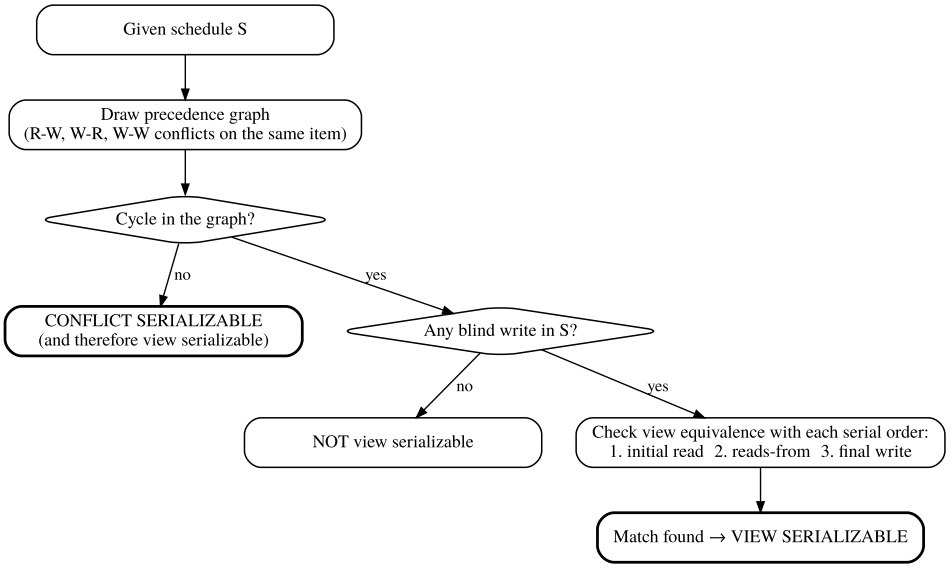
Ans.

Starvation	Deadlock
One transaction **waits indefinitely** while others proceed	Transactions **wait for each other in a cycle**, so none proceeds
Fix: aging, FIFO queue	Fix: wait-die / wound-wait, wait-for graph

Section B: 7-Mark Questions

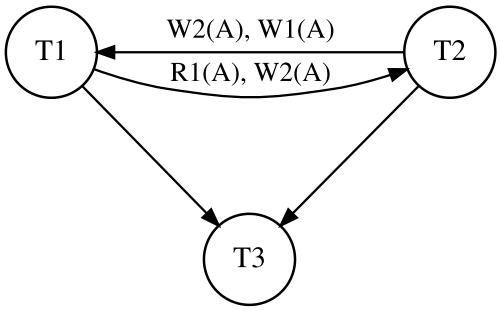
Q3. Conflict vs. view serializability; test a schedule. *[7 Marks]*

Ans. A schedule is **serializable** if it is equivalent to some **serial schedule**. **Conflict serializable:** it can be turned into a serial schedule by swapping non-conflicting operations. **View serializable:** it gives every transaction the same reads and the same final writes as a serial schedule.



Testing a schedule for conflict and view serializability

Schedule: R1(A), W2(A), W1(A), W3(A)



Precedence graph: the cycle T1 → T2 → T1 means S is NOT conflict serializable

Step 1: Conflict test. The edges T1 → T2 and T2 → T1 form a **cycle**, so S is **not conflict serializable**.

Step 2: View test against the serial order T1 → T2 → T3. W2 and W3 are **blind writes**.

Condition	S	T1 → T2 → T3
Initial read of A	T1	T1 ✓
Reads-from	none	none ✓
Final write of A	T3	T3 ✓

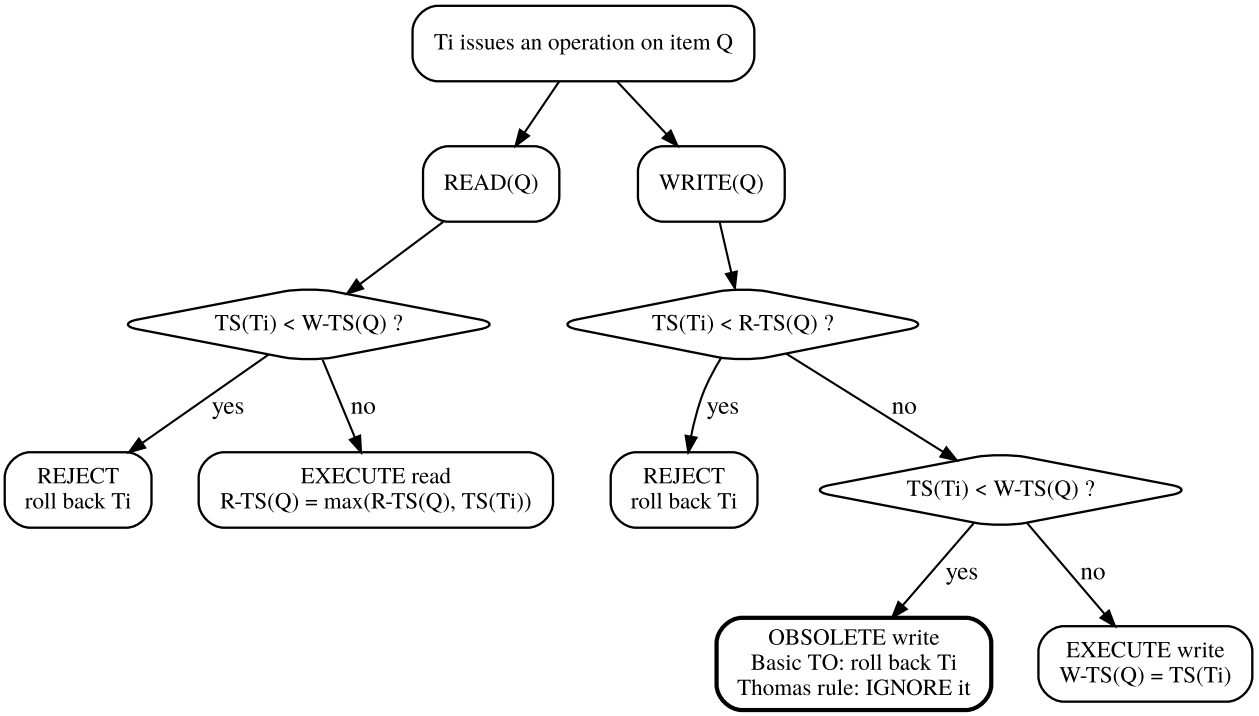
Result: VIEW serializable but not conflict serializable.

Conflict	View
Precedence graph test	Initial read, reads-from, final write
Polynomial	**NP-complete**
Subset (stricter)	Superset (allows blind writes)

Conclusion: in practice, DBMSs enforce **conflict serializability** (2PL, timestamps).

Q4. Explain the Timestamp Ordering protocol and the Thomas Write Rule. *[7 Marks]*

Ans. Timestamp Ordering (TO) is a **lock-free** protocol. Each transaction gets a unique timestamp TS(Ti), and conflicting operations run in **timestamp order**. Each item Q keeps **R-TS(Q)** and **W-TS(Q)**.



Basic timestamp ordering, with the Thomas Write Rule change

Rules:

1. **Read(Q):** if TS(Ti) < W-TS(Q), **roll back Ti**. Otherwise read, and set R-TS(Q) = max(R-TS(Q), TS(Ti)).
2. **Write(Q):** if TS(Ti) < R-TS(Q), **roll back Ti**.
3. **Write(Q):** if TS(Ti) < W-TS(Q), the write is **obsolete**. Basic TO **rolls back** Ti. The **Thomas Write Rule ignores the write** and lets Ti continue.
4. Otherwise write, and set W-TS(Q) = TS(Ti).

Example: TS(T1) = 10, TS(T2) = 20, with operations R1(Q), W2(Q), W1(Q).

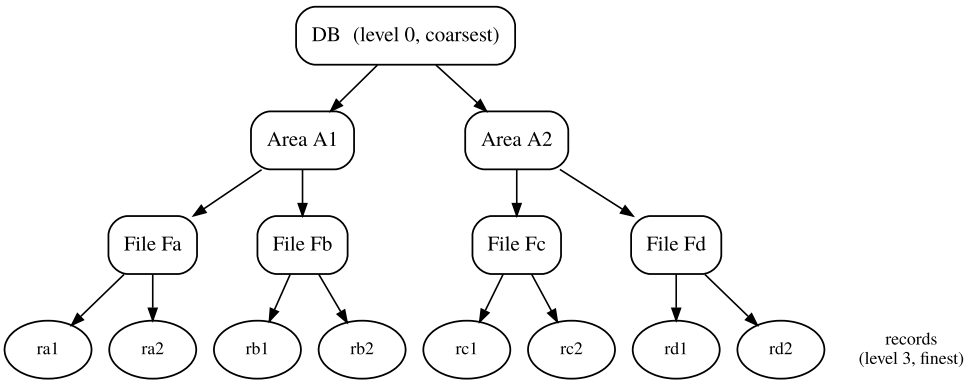
- At W1(Q): 10 < W-TS = 20, so the write is obsolete.
- **Basic TO:** roll back T1. **Thomas rule:** ignore W1(Q).

Advantages: deadlock-free, serializable. **Disadvantages:** starvation, cascading rollbacks, timestamp overhead.

Conclusion: the basis of **MVCC** in PostgreSQL and Oracle.

Q5. Explain multiple granularity locking and intention locks. *[7 Marks]*

Ans. Multiple granularity locking lets transactions lock items of **different sizes** (DB → Area → File → Record) arranged as a tree. Locking a node **implicitly locks its descendants**.



Granularity hierarchy: lock top-down, unlock bottom-up

Lock modes:

- **S** (shared) and **X** (exclusive).
- **IS / IX:** intention to take S / X locks **lower** in the tree.
- **SIX:** S on the whole subtree plus IX (read all, update a few).

Requested ↓ / Held →	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	×
IX	✓	✓	×	×	×
S	✓	×	✓	×	×
SIX	✓	×	×	×	×
X	×	×	×	×	×

Rules:

1. Lock the **root first**.
2. S or IS on a node requires IS or IX on its parent.
3. X, SIX or IX on a node requires IX or SIX on its parent.
4. Follow 2PL.
5. Unlock **bottom-up**.

- **Purpose of intention locks:** conflicts are checked only at the **ancestors**, so the whole subtree does not need to be searched.
- **Lock escalation:** many row locks (for example more than 5000) are replaced with **one table lock**, which saves memory but lowers concurrency.

Conclusion: used in **SQL Server, DB2 and MySQL InnoDB**.

UNIT 5: RECOVERY & FILE ORGANIZATION

Section A: 2-Mark Questions

Q1. What is Write-Ahead Logging (WAL)? *[2 Marks]*

Ans. WAL rule: the **log record of a change must reach stable storage before the changed data page is written to disk**, and all of a transaction's log records must be written before it commits. This makes UNDO and REDO possible after a crash.

Q2. Static vs. dynamic hashing; primary, clustering and secondary index. *[2 Marks]*

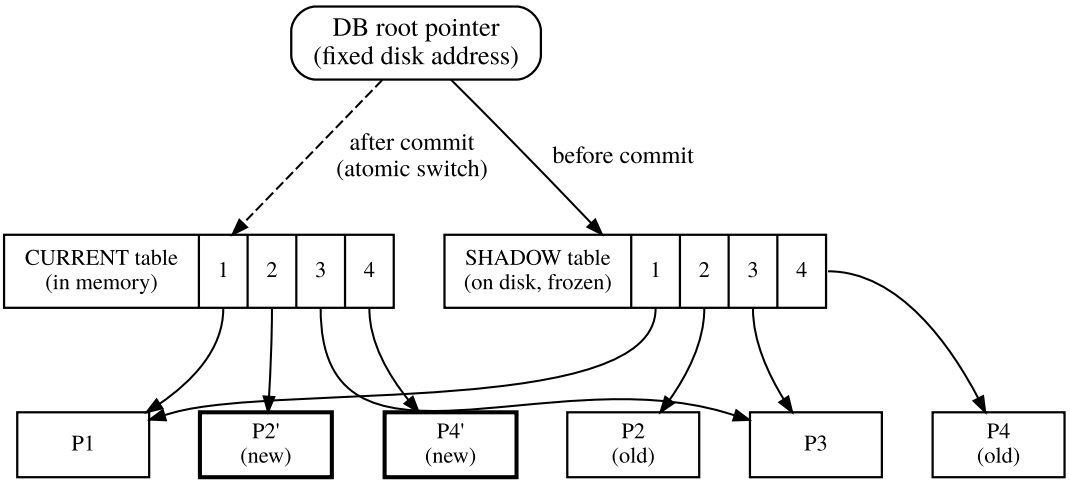
Ans.

- **Static hashing:** a **fixed** number of buckets, with overflow chains.
- **Dynamic hashing:** buckets **grow and shrink** (extendible hashing).
- **Primary index:** on the ordering **key** field (sparse).
- **Clustering index:** on an ordering **non-key** field.
- **Secondary index:** on a **non-ordering** field (dense; there can be many).

Section B: 7-Mark Questions

Q3. Explain Shadow Paging with its advantages and disadvantages. *[7 Marks]*

Ans. Shadow paging is a recovery technique that **needs no log** (for a single user). It keeps two page tables: the **current page table** and the **shadow page table**. The shadow table is never changed during the transaction.



Shadow paging: the shadow table still points to the old pages; pages 2 and 4 were copied on write

Working:

1. At the start, **copy** the current page table to make the shadow page table.
2. **On write (copy-on-write):** copy the page to a **free block**, update the copy, and point the **current table** to it.

3. **Commit:** flush the pages and the current table, then **atomically switch the DB root pointer** to the current table.
4. **Crash or abort:** the root still points to the **shadow table**, which is the old consistent state. **No UNDO or REDO** is needed.

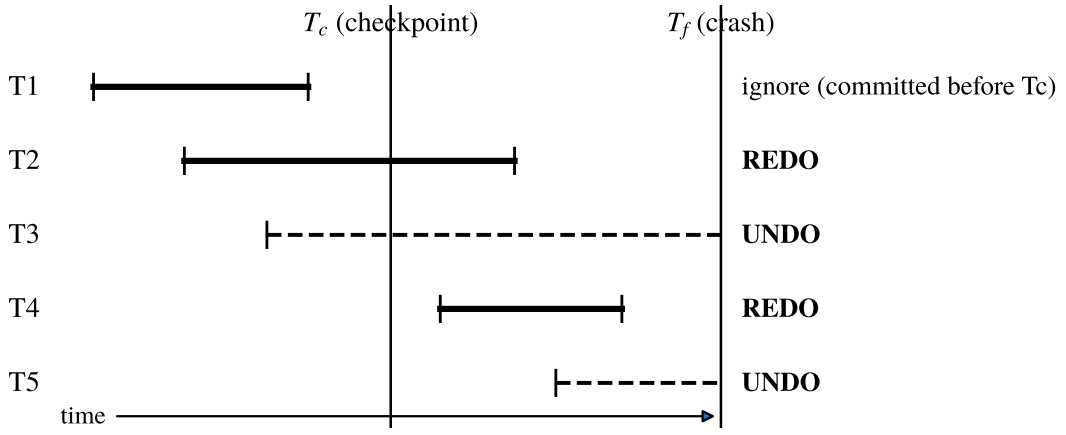
Shadow Paging	Log-based Recovery
No log	Needs a log
Recovery is very fast	Needs UNDO / REDO
High commit overhead	Low commit overhead
Hard with concurrency	Easy with concurrency

Advantages: no log overhead, fast recovery, simple atomic commit. **Disadvantages:** costly page-table copying, **data fragmentation**, garbage collection needed.

Conclusion: used in **System R** and in copy-on-write file systems (ZFS).

Q4. Explain checkpointing; deferred vs. immediate modification. *[7 Marks]*

Ans. Log-based recovery records every change: ⟨Ti start⟩, ⟨Ti, X, old, new⟩ and ⟨Ti commit⟩. A **checkpoint** limits how much of the log must be scanned after a crash.



Checkpoint recovery: transactions are classified by when they started and committed relative to the checkpoint Tc and the crash Tf

Checkpoint steps:

1. Write the log records in memory to stable storage.
2. Write the modified buffer blocks to disk.
3. Write a ⟨checkpoint L⟩ record.

On recovery, scan back only to the last checkpoint: **committed transactions are REDONE, uncommitted ones are UNDONE**, and transactions committed before Tc are ignored.

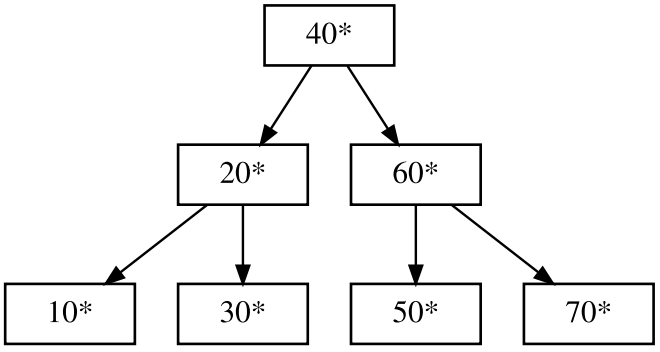
Example: T0: A = 1000 → 950, B = 2000 → 2050 (committed); T1: C = 700 → 600 (not committed).

Basis	Deferred modification	Immediate modification
DB updated	Only **after commit**	Before or after commit
Log record	⟨Ti, X, new⟩	⟨Ti, X, **old, new**⟩
Recovery	**REDO only**	**UNDO + REDO**
In the example	REDO(T0), ignore T1	REDO(T0), UNDO(T1) so C = 700

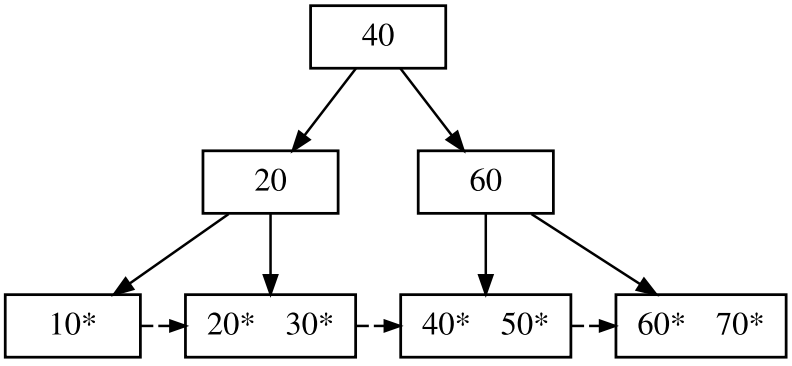
Conclusion: immediate modification with checkpoints is the basis of **ARIES** (DB2, SQL Server).

Q5. Compare B-Tree and B+ Tree; show B-Tree insertion. *[7 Marks]*

Ans. A **B-Tree of order m** is a balanced m-way search tree. Each node has at most **m children and m − 1 keys**, non-root nodes have at least **⌈m/2⌉ children**, and **all leaves are at the same level**. In a **B+ Tree**, data is stored only in the leaves, and the leaves are **linked**.



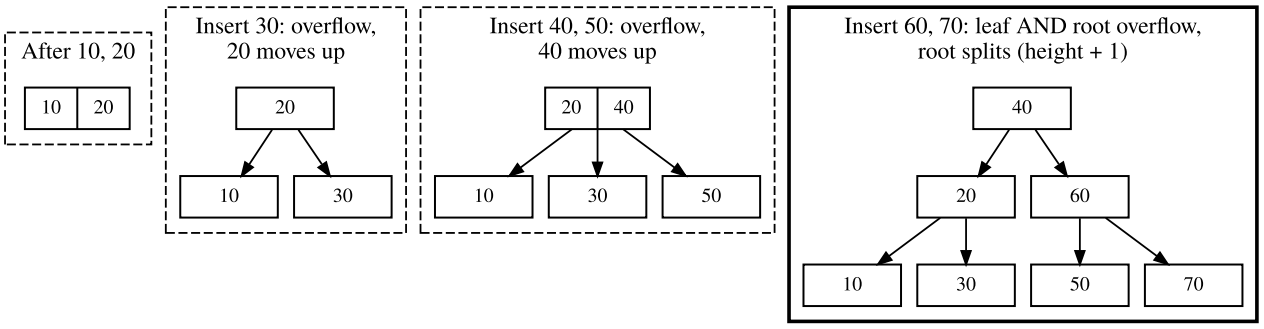
B-Tree of order 3: every node holds keys AND data pointers (*)



B+ Tree of order 3: data only in the leaves, and the leaves are linked in order

Insertion rule:

1. Insert into the correct leaf.
2. If it overflows, **split** it: the **median moves up**.
3. If the root splits, the height increases by 1.



B-Tree (order m = 3) insertion of 10, 20, ..., 70: each overflow pushes the median key up

Basis	B-Tree	B+ Tree
Data pointers	All nodes	**Leaves only**
Leaf links	No	**Yes** (sequential access)
Range queries	Slow	**Fast**
Search time	Variable	Uniform
Used in	File systems	**All DBMS indexes**

Conclusion: B+ Trees are the default index in **MySQL InnoDB, Oracle and PostgreSQL**.

Web Technology

BCS-502 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** for code questions, write a **short program with comments** and then **explain the key tags or methods** in points. For theory questions, follow this order: definition, diagram, points, table, conclusion.

UNIT 1: WEB FOUNDATIONS, HTML & XML

Section A: 2-Mark Questions

Q1. Define HTTP and mention its three basic features. *[2 Marks]*

Ans. HTTP (HyperText Transfer Protocol) is an **application-layer, request-response protocol** for transferring web resources between a **browser (client)** and a **web server**. It runs over TCP on **port 80** (HTTPS uses 443).

1. **Connectionless:** the connection closes after each response.
2. **Stateless:** the server remembers nothing between requests, so cookies and sessions are used.
3. **Media independent:** any data type can be sent, using its **MIME type**.

Q2. Differentiate between HTML and XML. *[2 Marks]*

Ans.

HTML	XML
Displays data	**Stores and transports** data
Predefined tags	**User-defined** tags
Not case-sensitive	**Case-sensitive**
Closing tags optional, lenient	Closing tags **mandatory**, strict (well-formed)

Q3. Client-server vs. peer-to-peer paradigm. *[2 Marks]*

Ans.

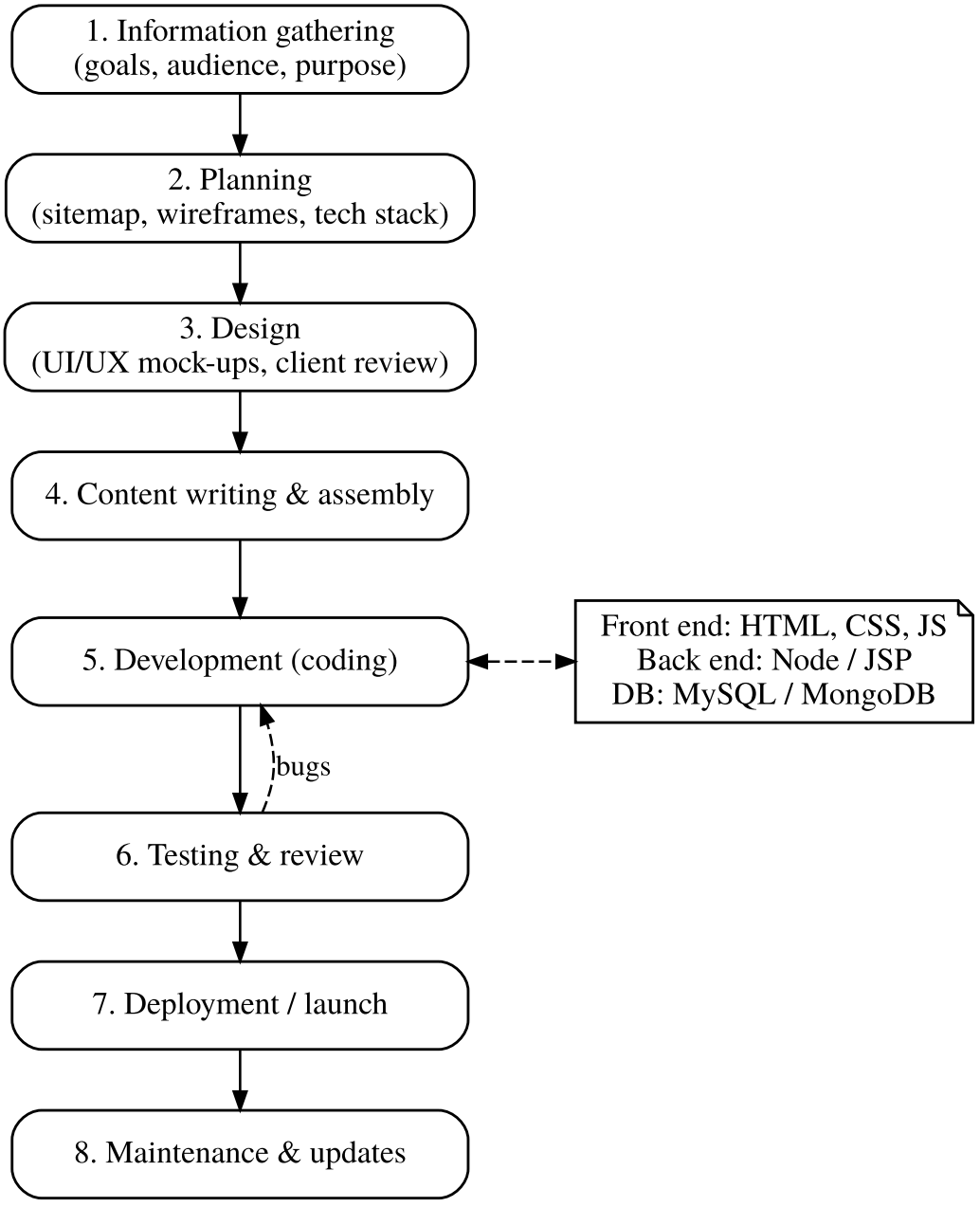
Client-Server	Peer-to-Peer
Dedicated server serves the clients	**Every node** is both client and server
Centralised; single point of failure	Decentralised; no single point of failure
Example: web browsing, email	Example: BitTorrent, blockchain

Section B: 7-Mark Questions

Q4. Explain web development strategies and the development process with a block diagram.

[7 Marks]

Ans. Web development means building, testing and maintaining websites and web applications. A **strategy** is the planned approach (goals, users, technology and method) used to deliver a site that is **usable, scalable and secure**.



Web development life cycle

Strategies:

- **Define goals and audience.**
- Choose the **architecture**: static site, dynamic site or **SPA**.
- **Mobile-first / responsive** design.
- **Methodology:** Waterfall or **Agile**.

- **Tech stack:** MERN or LAMP.
- Plan **SEO, accessibility and security** from the start.

Process (8 steps):

1. **Information gathering:** purpose and audience.
2. **Planning:** sitemap and wireframes.
3. **Design:** UI mock-ups.
4. **Content:** text, images, SEO.
5. **Development:** front end and back end.
6. **Testing:** browsers, forms, security.
7. **Deployment:** hosting and domain.
8. **Maintenance:** updates and monitoring.

Waterfall	Agile
Linear phases	Iterative sprints
Feedback at the end	Feedback every sprint
Changes costly	Changes easy

Conclusion: followed by every web team, for example when building an e-commerce site or a college ERP portal.

Q5. Write HTML for a user registration form using various input types. *[7 Marks]*

Ans. An **HTML form** collects user input. **action** gives the server URL and **method** (GET / POST) decides how the data is sent.

USER REGISTRATION FORM

Full Name [text]

Email [email]

Password [••••••••]

Date of Birth [dd-mm-yyyy (date)]

Gender (•) Male ( ) Female ( ) Other

Course [B.Tech CSE]

Skills [x] HTML [ ] CSS [x] JS

Photo [Choose File]

Address [ ]

[x] I agree to the terms

[Register] [Reset]

method="post" → action="/register" → server (Servlet / Node.js)

Layout of the registration form and where its data goes

```
<form action="/register" method="post" enctype="multipart/form-data">
  Name:     <input type="text" name="name" required><br>
  Email:    <input type="email" name="email" required><br>
  Password: <input type="password" name="pwd" minlength="8"><br>
  DOB:      <input type="date" name="dob"><br>
  Mobile:   <input type="tel" name="mob" pattern="[0-9]{10}"><br>
  Gender:   <input type="radio" name="g" value="M" checked> Male
            <input type="radio" name="g" value="F"> Female<br>
  Course:   <select name="course">
              <option>B.Tech CSE</option><option>B.Tech IT</option>
            </select><br>
  Skills:   <input type="checkbox" name="sk" value="html"> HTML
            <input type="checkbox" name="sk" value="js"> JS<br>
  Photo:    <input type="file" name="photo" accept="image/*"><br>
  Address:  <textarea name="addr" rows="3"></textarea><br>
  <input type="submit" value="Register"> <input type="reset">
</form>
```

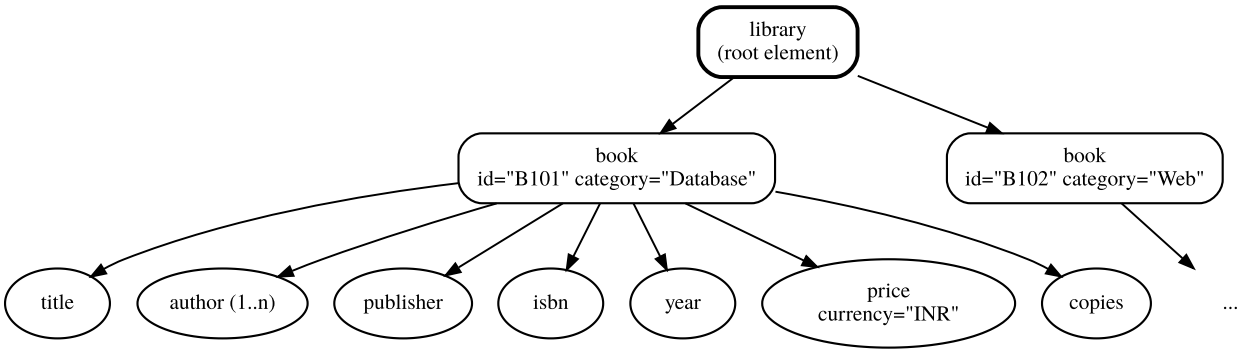
Key points:

1. **method=“post”** sends the data in the request body, which is secure for passwords.
2. **enctype=“multipart/form-data”** is required for **file upload**.
3. **name** is the key sent to the server, for example email=abc@x.com.
4. **required, pattern, minlength** give HTML5 validation.
5. **radio:** choose one option (same name). **checkbox:** choose many.

Conclusion: used on every sign-up page. Server-side validation is still required.

Q6. Write an XML file for library books and a DTD / XML Schema. *[7 Marks]*

Ans. XML stores data with **user-defined tags**. A **DTD or XSD** defines the legal structure, and a document that follows it is **valid**.



XML tree of library.xml, validated against a DTD or XSD

```
<?xml version="1.0"?>
<!DOCTYPE library [
  <!ELEMENT library (book+)>
  <!ELEMENT book (title, author+, isbn, year, price)>
  <!ELEMENT title (#PCDATA)>  <!ELEMENT author (#PCDATA)>
  <!ELEMENT isbn (#PCDATA)>   <!ELEMENT year (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
  <!ATTLIST book id ID #REQUIRED category CDATA #IMPLIED>
]>
<library>
  <book id="B101" category="Database">
    <title>Database System Concepts</title>
    <author>Silberschatz</author> <author>Korth</author>
    <isbn>9780073523323</isbn> <year>2019</year> <price>850</price>
  </book>
</library>
```

XSD for one element (sample):

```
<xs:element name="book" maxOccurs="unbounded">
  <xs:complexType><xs:sequence>
    <xs:element name="title" type="xs:string"/>
    <xs:element name="year"  type="xs:gYear"/>
    <xs:element name="price" type="xs:decimal"/>
  </xs:sequence>
  <xs:attribute name="id" type="xs:ID" use="required"/></xs:complexType>
</xs:element>
```

Symbols:

- **+** = one or more, ***** = zero or more, **?** = optional.
- **#PCDATA** = text.
- **#REQUIRED / #IMPLIED** = mandatory / optional attribute.

DTD	XML Schema
Non-XML syntax	Written in XML
Text only	**44+ data types** (int, date)
No namespaces	Namespaces supported

Conclusion: XML with schemas is used in SOAP web services, configuration files and library systems.

UNIT 2: CASCADING STYLE SHEETS (CSS)

Section A: 2-Mark Questions

Q1. Three ways to integrate CSS; remove the underline from a link. *[2 Marks]*

Ans.

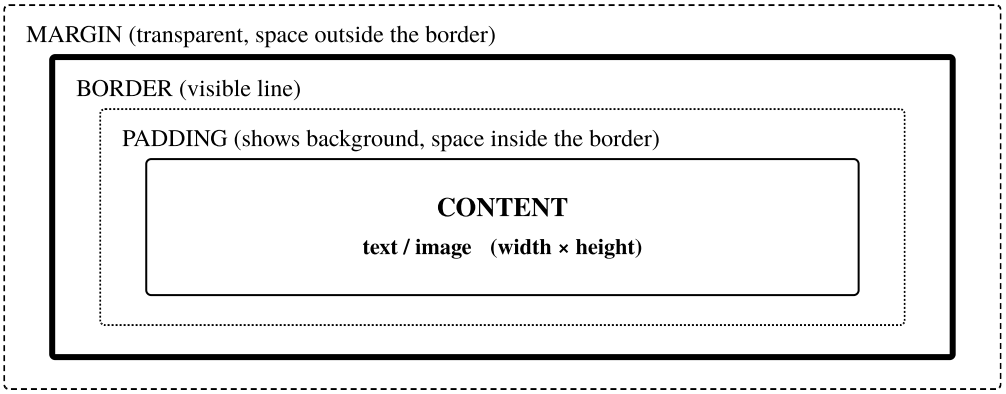
1. **Inline:** `<p style="color:blue">`. This has the **highest priority**.
2. **Internal:** a `<style>` tag in `<head>`, for one page.
3. **External:** `<link rel="stylesheet" href="style.css">`, for many pages.

To remove the underline: `a { text-decoration: none; }`

Section B: 7-Mark Questions

Q2. Illustrate the CSS Box Model with a diagram. *[7 Marks]*

Ans. The **CSS box model** says that **every element is a rectangular box** made of **content, padding, border and margin** (from the inside out).



CSS box model: margin → border → padding → content (outside to inside)

1. **Content:** text or image. Its size is set by `width` and `height`.
2. **Padding:** the space **inside the border**. It shows the background. Example: `padding: 10px;`
3. **Border:** the line around the padding. Example: `border: 2px solid black;`
4. **Margin:** the space **outside the border**. It is transparent, can be negative, and vertical margins **collapse**.

Size example: width 300, padding 20, border 5, margin 30:

$$\text{Total width} = 300 + 2(20) + 2(5) + 2(30) = 410 \text{ px}$$

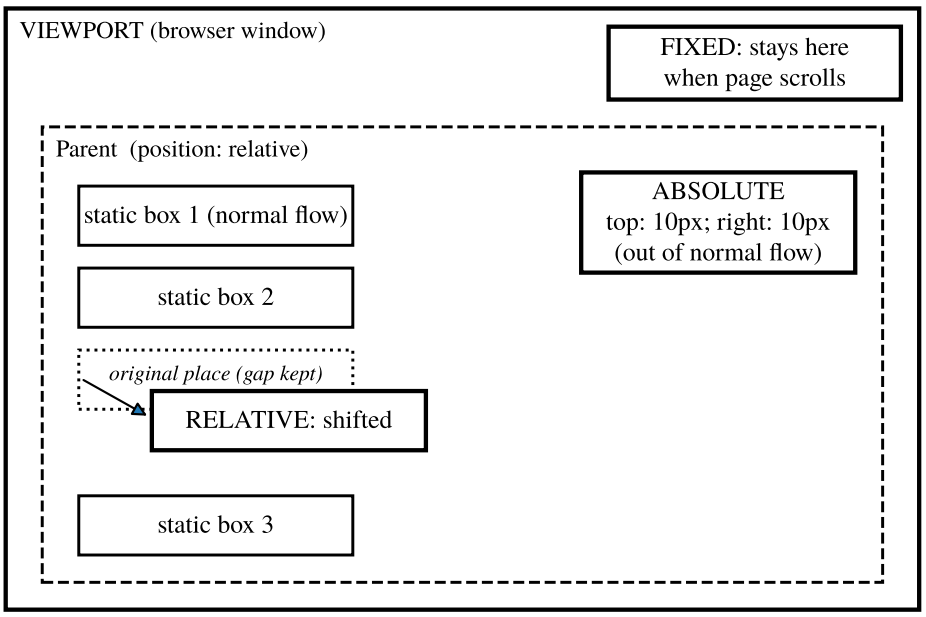
box-sizing: border-box makes `width` include the padding and the border.

Padding	Margin
Inside the border	Outside the border
Shows the background	Transparent
Cannot be negative	Can be negative

Conclusion: the box model is the base of every CSS layout (Flexbox, Grid).

Q3. Explain CSS positioning: static, relative, absolute, fixed. *[7 Marks]*

Ans. The **position** property decides how an element is placed. It works with **top, right, bottom, left** and **z-index**.



CSS positioning: static and relative stay in the normal flow; absolute is placed inside the nearest positioned parent; fixed is placed relative to the viewport

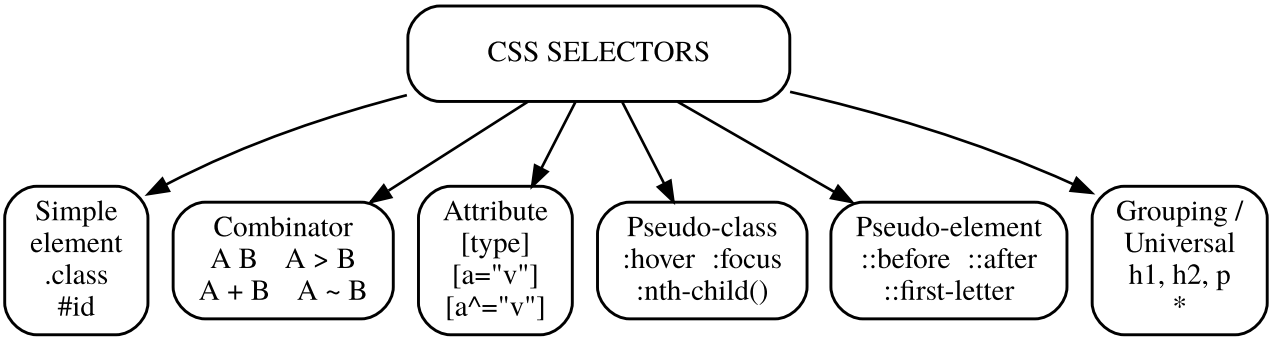
1. **static** (default): normal document flow. Offsets have no effect.
2. **relative:** moved **relative to its own normal position**. Its **original space is kept**. Example: `.box { position: relative; top: 20px; }`
3. **absolute: removed from the flow**. Placed relative to the **nearest positioned ancestor**. Example: a badge in a corner.
4. **fixed:** placed relative to the **viewport**. It **does not scroll**. Example: a sticky navbar.
5. **sticky:** acts as relative until a scroll point, then acts as fixed.

Value	In normal flow?	Placed relative to	Scrolls?
static	Yes	Normal flow	Yes
relative	Yes (gap kept)	Itself	Yes
absolute	**No**	Positioned ancestor	Yes
fixed	**No**	**Viewport**	**No**

Conclusion: used for navigation bars, dropdowns, modals and badges.

Q4. Discuss CSS selectors and pseudo-classes with examples. *[7 Marks]*

Ans. A **selector** chooses the HTML elements to style. A **pseudo-class** (`:`) selects an element in a **special state or position**.



Types of CSS selectors

1. **Simple:**
   - `*` (universal)
   - `p` (element)
   - `.cls` (class)
   - `#id` (id)
   - `h1, h2` (group)
2. **Combinators:**
   - `div p` (descendant)
   - `ul > li` (child)
   - `h1 + p` (adjacent sibling)
   - `h1 ~ p` (general sibling)
3. **Attribute:**
   - `input[type="text"]`
   - `a[href^="https"]` (starts with)
   - `a[href$=".pdf"]` (ends with)
4. **Pseudo-classes:**
   - `a:hover { color: red; }`, `input:focus`, `li:first-child`, `tr:nth-child(even)`, `:not(.x)`.
   - Write the link states in **LVHA order**: `:link`, `:visited`, `:hover`, `:active`.
5. **Pseudo-elements** (`::`): `p::first-letter`, `h2::before`, `::after`.
6. **Specificity:** inline (1000) > id (100) > class / pseudo-class (10) > element (1).

Pseudo-class	Pseudo-element
`:hover` (single colon)	`::before` (double colon)
A **state** of an element	A **part** of an element

Conclusion: used for zebra tables, hover menus and form-focus effects.

UNIT 3: JAVASCRIPT, AJAX & NETWORKING

Section A: 2-Mark Questions

Q1. What is AJAX and its main advantage? *[2 Marks]*

Ans. AJAX (Asynchronous JavaScript and XML) uses **XMLHttpRequest / fetch()** to exchange data with the server **in the background**. **Main advantage: partial page update without reloading**, so pages are faster and smoother. Examples: Google suggestions, Gmail refresh.

Q2. Java vs. JavaScript; define a Datagram. *[2 Marks]*

Ans.

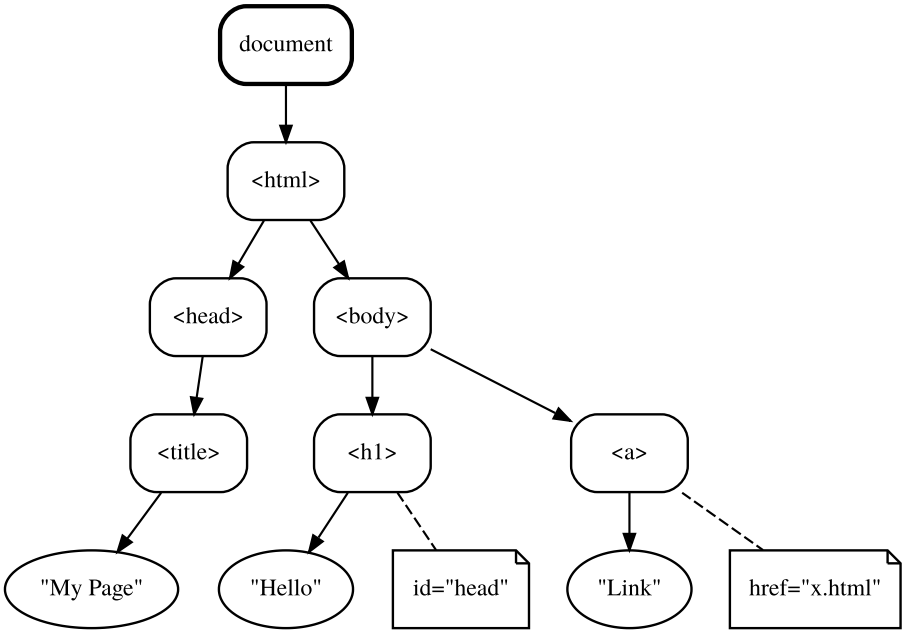
Java	JavaScript
Compiled to bytecode, runs on the **JVM**	Interpreted / JIT in the **browser**
Static typing, multi-threaded	**Dynamic** typing, single-threaded event loop

A **datagram** is an **independent, self-addressed packet** sent **without a connection** (UDP). It is **unreliable** (no guarantee of order or delivery), **fast** and of limited size. In Java: `DatagramSocket` and `DatagramPacket`.

Section B: 7-Mark Questions

Q3. Explain the JavaScript DOM. *[7 Marks]*

Ans. The **Document Object Model (DOM)** is a **W3C, language-independent interface** that represents an HTML document as a **tree of nodes**. JavaScript uses it to **read and change** content, structure and style **dynamically**.



DOM tree of the page: element, attribute and text nodes

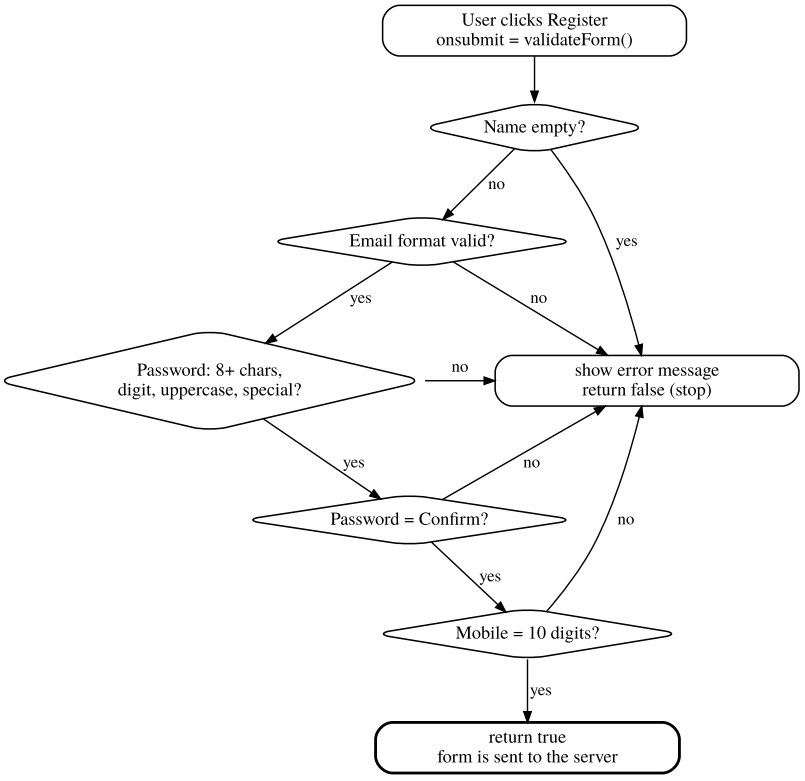
1. **Node types:** document, **element**, **attribute**, **text** and comment nodes.
2. **Select:** `getElementById()`, `getElementsByClassName()`, `querySelector()`, `querySelectorAll()`.
3. **Modify:** `el.innerHTML`, `el.textContent`, `el.setAttribute()`, `el.style.color = "blue"`.
4. **Create / remove:** `document.createElement("p")`, `appendChild()`, `removeChild()`.
5. **Traverse:** `parentNode`, `childNodes`, `firstChild`, `nextSibling`.
6. **Events:** `btn.addEventListener("click", fn)`. Events go through capturing, then the target, then **bubbling**.

DOM	BOM
Page content	Browser window
Root: `document`	Root: `window`
W3C standard	No standard

Conclusion: the DOM powers live validation, sliders, menus and AJAX updates.

Q4. Write a JavaScript program for form validation. *[7 Marks]*

Ans. Client-side validation checks input **in the browser before submission**, through the form's **onsubmit** event. Returning **false stops** the submission.



Client-side validation flow

```
        <form onsubmit="return validate()">
          Name: <input id="name"> Email: <input id="email">
          Password: <input type="password" id="pwd">
          Confirm: <input type="password" id="cpwd">
          <input type="submit" value="Register">
        </form>
        <script>
        function validate() {
          let n = document.getElementById("name").value.trim();
          let e = document.getElementById("email").value.trim();
          let p = document.getElementById("pwd").value;
          let c = document.getElementById("cpwd").value;
          if (n === "") { alert("Name is required"); return false; }          // empty
check
          if (!/^[\w.]+@[\w.]+\.[a-z]{2,}$/i.test(e)) {                        // email
format
              alert("Invalid email"); return false; }
          if (!/^(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%]).{8,}$/.test(p)) {            // strong
password
              alert("Password: 8+ chars, 1 capital, 1 digit, 1 symbol"); return false; }
          if (p !== c) { alert("Passwords do not match"); return false; }     // confirm
          return true;                                                         // submit
form
        }
        </script>
```

Explanation:

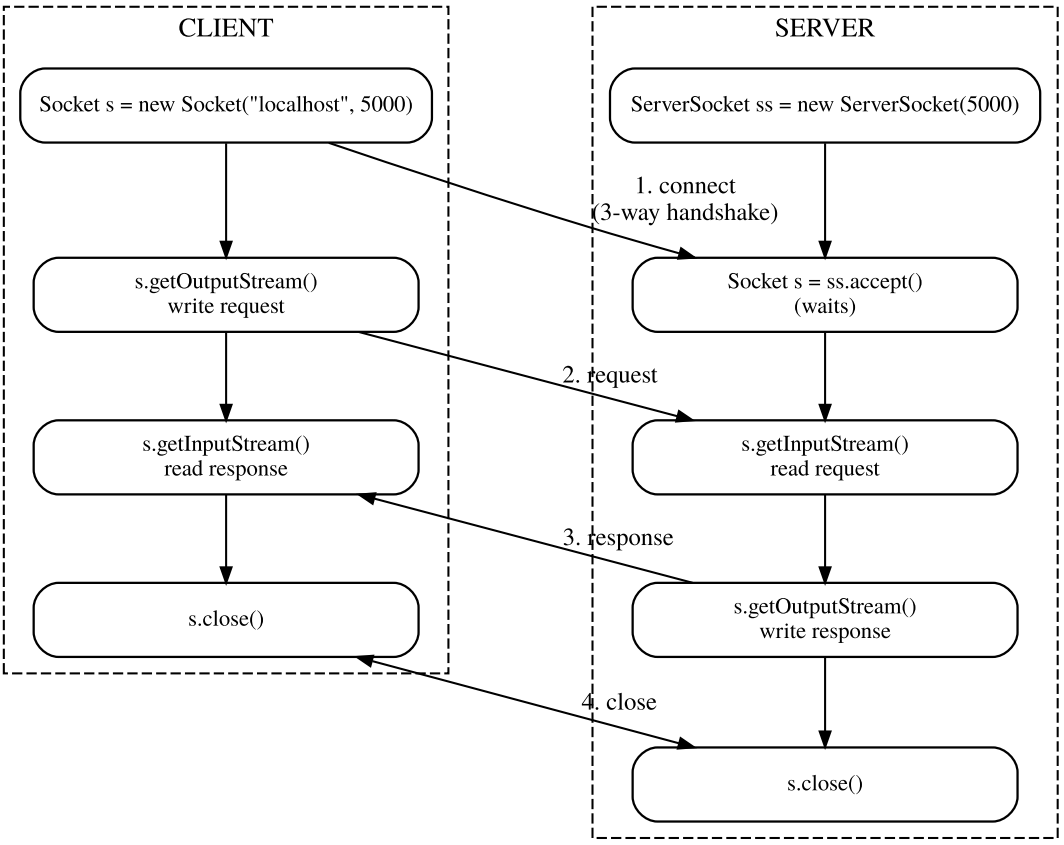
1. `.value.trim()` reads the input and removes extra spaces.
2. `RegExp.test()` checks the format.
3. `return false` cancels the submission.

Client-side	Server-side
Instant feedback	Needs a round trip
Can be bypassed	**Secure, mandatory**

Conclusion: used on login and sign-up forms, always together with server-side checks.

Q5. Explain TCP/IP client sockets with their constructors and methods. *[7 Marks]*

Ans. A **socket** is one **endpoint of a two-way link**, identified by an **IP address and a port**. A Java **TCP client socket (`java.net.Socket`)** gives a **reliable, connection-oriented stream**.



TCP client-server socket communication

Constructors:

1. `Socket(String host, int port)`: connects to a host and port.
2. `Socket(InetAddress addr, int port)`
3. `Socket(String host, int port, InetAddress localAddr, int localPort)`
4. `Socket()`: unconnected. Call `connect()` later.

Methods:

- `getInputStream()`, `getOutputStream()`
- `getInetAddress()`, `getPort()`, `getLocalPort()`
- `setSoTimeout()`, `isConnected()`, `close()`

```
        Socket s = new Socket("localhost", 5000);
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
        out.println("Hello Server");
        System.out.println("Server: " + in.readLine());
        s.close();
```

TCP Socket	UDP Datagram
Connection-oriented, reliable	Connectionless, unreliable
Byte stream	Packets
HTTP, FTP	DNS, video streaming

Conclusion: used by browsers, chat apps and JDBC drivers.

UNIT 4: JAVA BEANS & EJB

Section A: 2-Mark Questions

Q1. What is a Java Bean? What is introspection? *[2 Marks]*

Ans. A **Java Bean** is a **reusable software component** that follows these conventions:

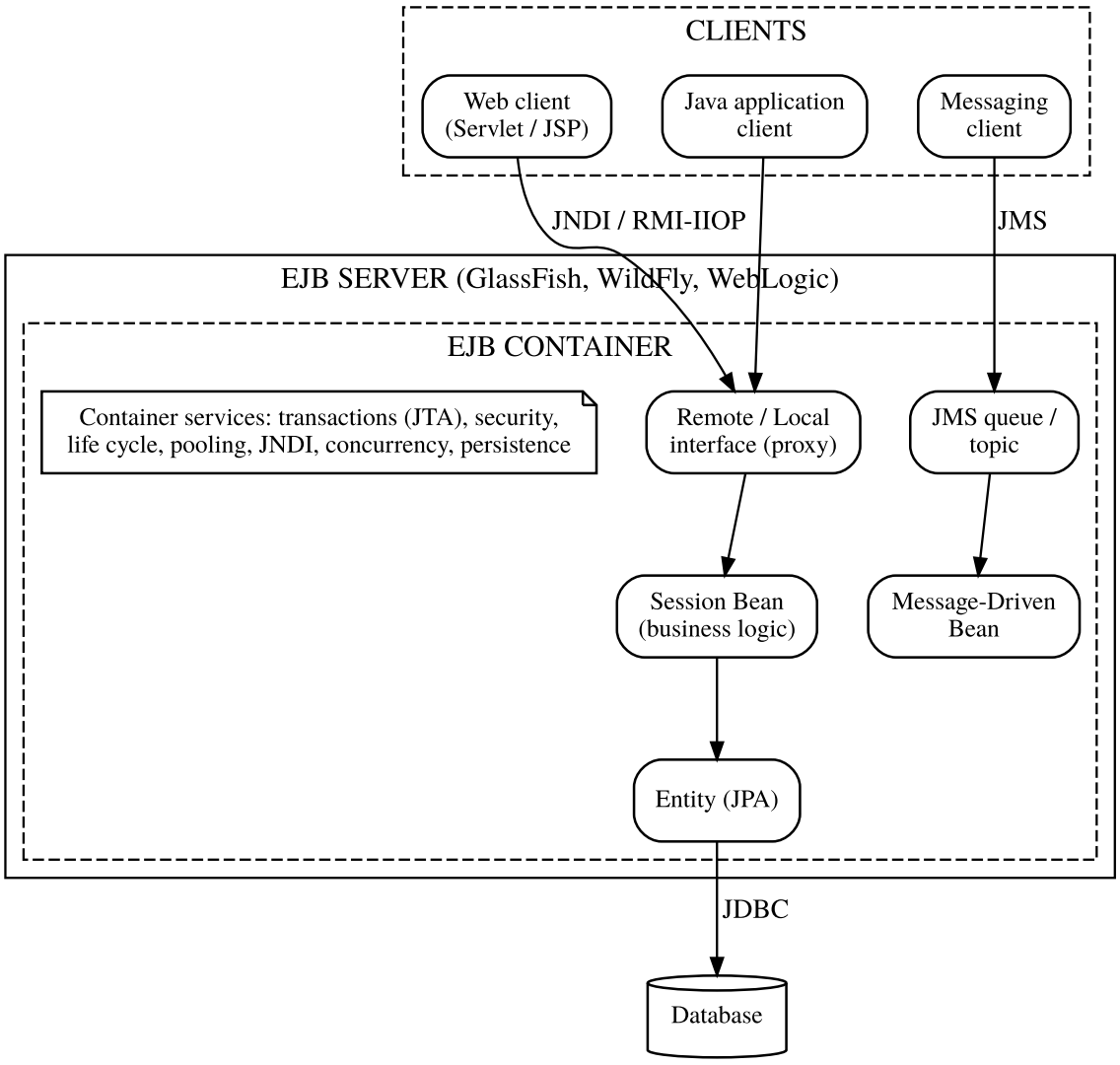
1. A **public no-arg constructor**.
2. **Private properties with getters and setters**.
3. It implements **Serializable**.

Introspection is the process by which builder tools **automatically discover a bean's properties, methods and events** at run time, using `java.beans.Introspector` (reflection plus naming conventions) or a `BeanInfo` class.

Section B: 7-Mark Questions

Q2. Explain the complete EJB architecture. *[7 Marks]*

Ans. Enterprise JavaBeans (EJB) is a **server-side component architecture** (Jakarta EE) for **distributed, transactional, secure** business applications. Beans run inside an **EJB container**, which gives them system services.



EJB architecture

1. **EJB server:** the application server (GlassFish, WildFly).
2. **EJB container:** manages the beans. **Every call passes through it.**
3. **Beans:**
   - **Session beans:** business logic.
   - **Message-driven beans:** asynchronous JMS messages.
   - **JPA entities:** persistence.
4. **Interfaces:** `@Remote` (other JVM, RMI-IIOP) and `@Local` (same JVM).
5. **JNDI:** clients look up beans by name.
6. **Container services:** transactions (JTA), security, pooling, life cycle, concurrency, persistence.

Request flow:

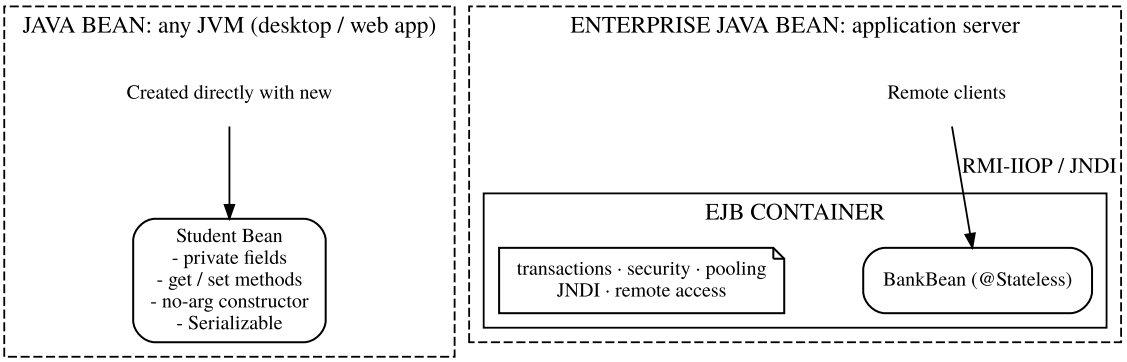
1. The client looks up the bean through JNDI.
2. The container returns a proxy.
3. The client calls a method on the proxy.
4. The container starts a transaction and checks security.
5. A pooled bean runs the method.

6. The container commits and returns the result.

Conclusion: EJB is used in banking and airline reservation systems that need distributed transactions.

Q3. Differentiate between a Java Bean and an EJB. *[7 Marks]*

Ans. A **Java Bean** is a simple reusable component. An **EJB** is a **server-side distributed business component** managed by a container.



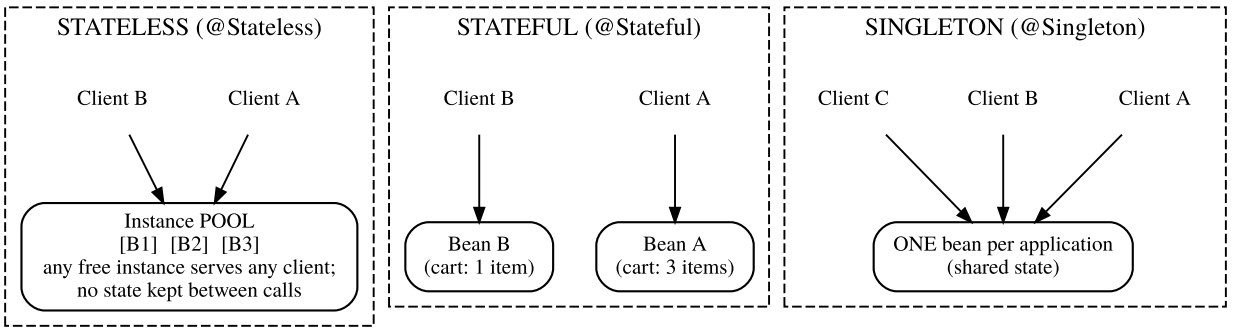
Where a Java Bean and an EJB run

Basis	Java Bean	EJB
Runs in	Any JVM	**EJB container** only
Created by	Programmer (`new`)	**Container**
Package	`java.beans`	`jakarta.ejb`
Remote access	No	Yes (RMI-IIOP)
Transactions / security	Coded manually	**Provided by the container**
Interfaces	None needed	`@Local` / `@Remote`
GUI use	Yes (builder tools)	No
Example	Swing button, Student DTO	Bank service bean
Complexity	Low	High

Conclusion: Java Beans are used for UI and data components. EJBs are used for enterprise business services.

Q4. Explain session beans: stateful vs. stateless. *[7 Marks]*

Ans. A **session bean** performs **business tasks for a client**. It is **not persistent**. There are three types: **Stateless, Stateful and Singleton**.



Session bean types



Life cycle of a stateful session bean (a stateless bean has only Does Not Exist and Ready)

1. **Stateless (`@Stateless`):**
   - Keeps **no client state** between calls.
   - Instances are **pooled and shared**, so it is highly scalable.
   - Example: a tax calculator.
2. **Stateful (`@Stateful`):**
   - Keeps **conversational state for one client**.
   - Can be **passivated** to disk when idle. Ends at `@Remove`.
   - Example: a **shopping cart**.
3. **Singleton (`@Singleton`): one instance per application**, shared by everyone. Example: a cache.

Stateless	Stateful
No conversational state	Keeps state per client
Shared pool	One bean per client
Very scalable	Uses more memory
No passivation	Passivation / activation

Conclusion: stateless beans for payment services, stateful beans for carts and booking wizards.

UNIT 5: SERVLETS, JSP, NODE.JS & MONGODB

Section A: 2-Mark Questions

Q1. What is Node.js? Purpose of HttpServletRequest? *[2 Marks]*

Ans.

- **Node.js** is a **JavaScript run time built on Chrome's V8 engine** for the **server side**. It is **single-threaded, event-driven and non-blocking**. **Use:** fast, scalable I/O apps such as REST APIs and

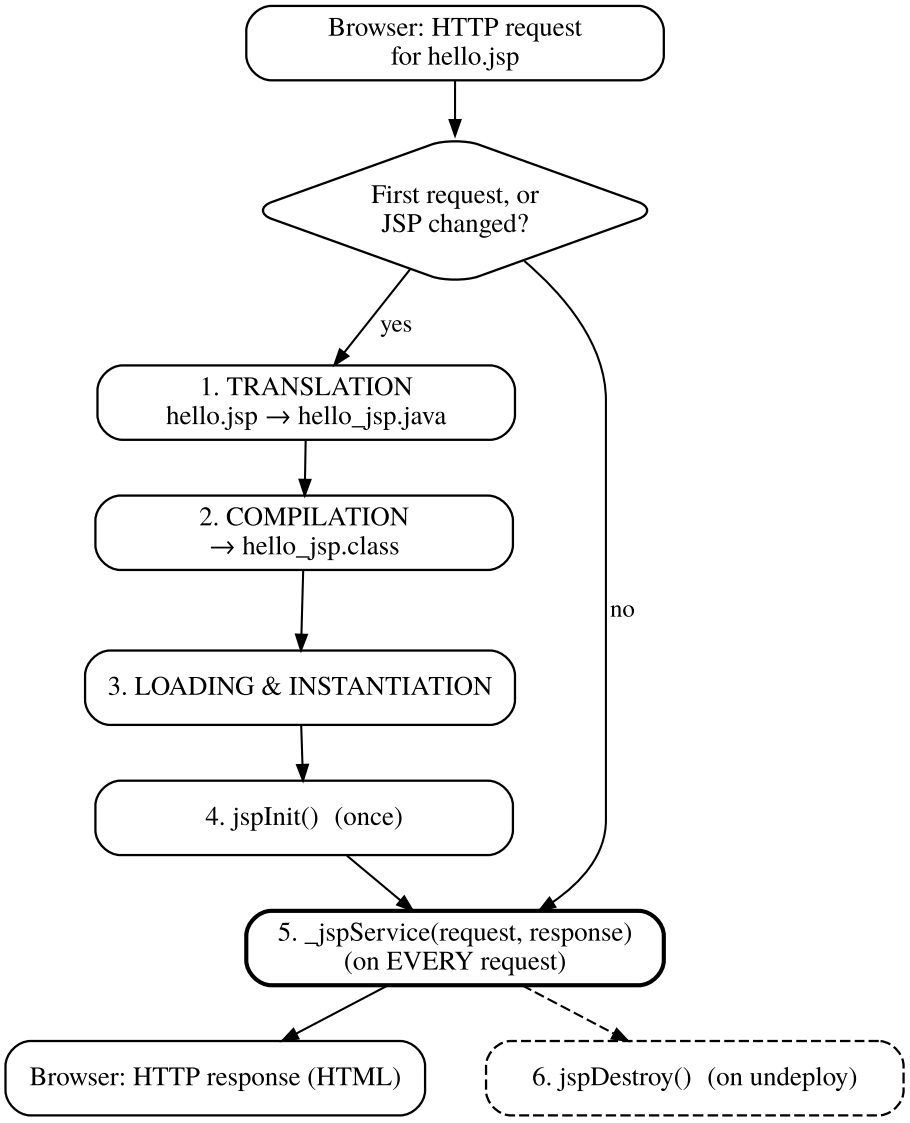
real-time chat.

- **HttpServletRequest** represents the **client's HTTP request**. It lets the servlet read parameters (`getParameter()`), headers, cookies, the session (`getSession()`) and the request method.

Section B: 7-Mark Questions

Q2. Explain the compilation of JSP into Servlets and its execution. *[7 Marks]*

Ans. A **JSP** page is not run directly. The **JSP container (Jasper in Tomcat) translates it into a servlet, compiles it and then executes it**.



JSP processing: translation and compilation happen only on the first request (or after the JSP changes)

1. **Translation:** hello.jsp becomes **hello_jsp.java**.
   - HTML becomes `out.write()`.
   - `<% %>` code goes into `_jspService()`.
   - `<%= %>` becomes `out.print()`.
2. **Compilation:** produces **hello_jsp.class**.
3. **Loading and instantiation:** one object is created.

4. **jspInit(): called once**.
5. **_jspService(req, res):** called for **every request**, in a new thread. Implicit objects are available: `request, response, out, session, application`.
6. **jspDestroy(): called once, on undeploy.

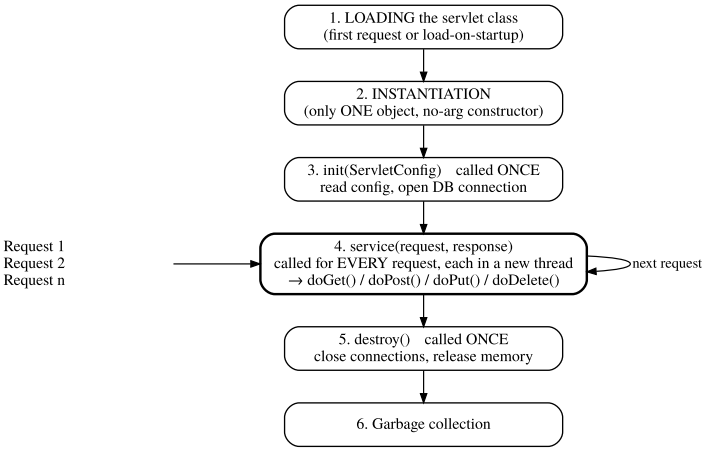
> **Remember:** only the **first request** is slow (translation and compilation). Later requests call `_jspService()` directly.

JSP	Servlet
Java inside HTML (view)	HTML inside Java (controller)
Auto-recompiles	Manual compile
Implicit objects	Get the objects manually

Conclusion: JSP is the **View** in MVC web applications running on Tomcat.

Q3. Explain the Servlet life cycle methods: init, service, destroy. *[7 Marks]*

Ans. A **servlet** is a server-side Java class that handles HTTP requests. Its **life cycle** is controlled by the **container** through `init()`, `service()` and `destroy()`.



Servlet life cycle

1. **Loading and instantiation:** the class is loaded on the first request (or at start-up), and **one object** is created.
2. **init(ServletConfig): called once**. Used for set-up, such as opening a DB connection.
3. **service(req, res): called for **every request**, each in a **new thread**. It dispatches to `doGet()` / `doPost()`.
4. **destroy(): called once**, before removal. Releases resources.
5. **Garbage collection.**

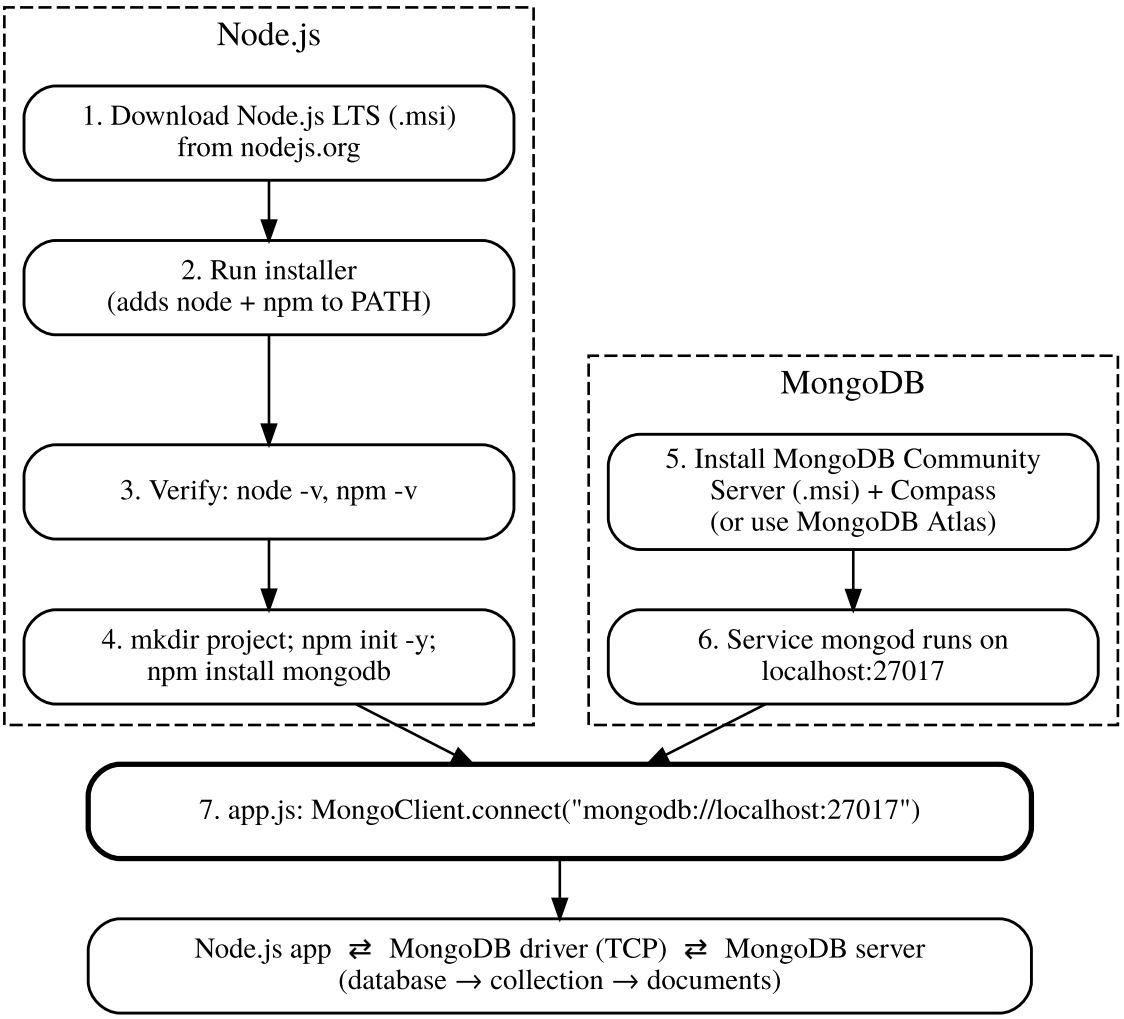
```
        public class Hello extends HttpServlet {
          public void init() { System.out.println("init once"); }
          protected void doGet(HttpServletRequest rq, HttpServletResponse rs) throws
IOException {
              rs.getWriter().println("<h2>Hello " + rq.getParameter("name") + "</h2>"); }
          public void destroy() { System.out.println("destroy once"); }
        }
```

init()	**service()**	**destroy()**
Once	Per request	Once
Set up	Process the request	Clean up

Conclusion: the basis of Spring MVC's DispatcherServlet and all Java web apps.

Q4. Steps to set up Node.js on Windows and connect to MongoDB. *[7 Marks]*

Ans. Node.js runs JavaScript on the server. **MongoDB** is a **NoSQL document database**. They are connected through the **mongodb driver** (or Mongoose).



Setting up Node.js and MongoDB on Windows

1. Download the **Node.js LTS (.msi)** from nodejs.org and install it (adds it to PATH).
2. Verify: `node -v` and `npm -v`.

3. `mkdir myapp && cd myapp`, then `npm init -y`, then `npm install mongodb`.
4. Install **MongoDB Community Server** (as a service). It runs on **localhost:27017**. (Or use MongoDB Atlas.)
5. Write `app.js` and run it with `node app.js`:

```
const { MongoClient } = require("mongodb");
const client = new MongoClient("mongodb://127.0.0.1:27017");
async function main() {
  await client.connect();                              // connect
  const db = client.db("collegeDB");                   // database
  await db.collection("students").insertOne({ roll: 101, name: "Harsh" });
  console.log(await db.collection("students").find().toArray());
  await client.close();
}
main();
```

MongoDB	MySQL
Documents (JSON)	Tables
Flexible schema	Fixed schema
Horizontal scaling	Mainly vertical scaling

Conclusion: Node.js with MongoDB forms the back end of the **MERN** stack.

Design and Analysis of Algorithms

BCS-503 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** DAA papers mostly ask for **traces**. Always write the **algorithm or formula first**, then a **step table**, then the **final answer in a box**. Draw every tree or graph.

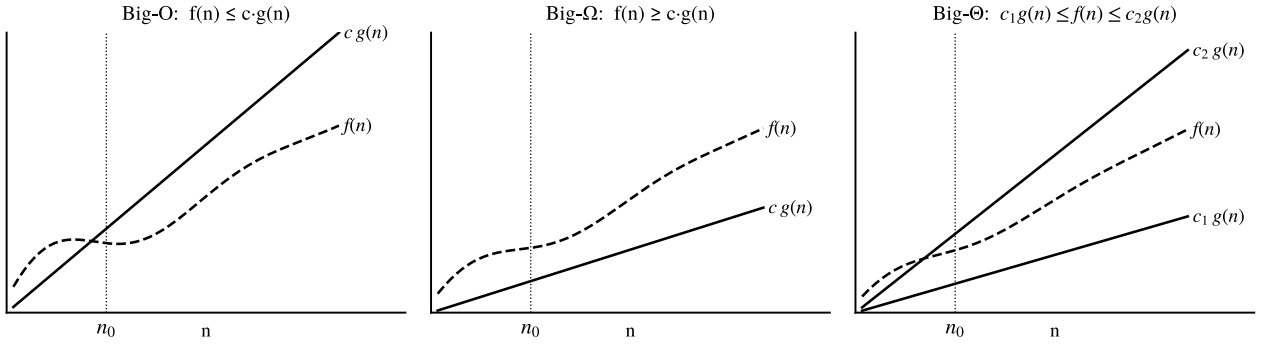
UNIT 1: COMPLEXITY, RECURRENCES & SORTING

Section A: 2-Mark Questions

Q1. Define Big-O, Big-Omega and Big-Theta. *[2 Marks]*

Ans.

- **Big-O (upper bound):** $f(n) = O(g(n))$ if $0 \le f(n) \le c\,g(n)$ for all $n \ge n_0$.
- **Big-Ω (lower bound):** $f(n) = \Omega(g(n))$ if $0 \le c\,g(n) \le f(n)$ for all $n \ge n_0$.
- **Big-Θ (tight bound):** $c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0$.
- Example: $3n + 2 = \Theta(n)$.



Asymptotic bounds: O gives an upper bound, Ω a lower bound and Θ a tight bound (for all $n \ge n_0$)

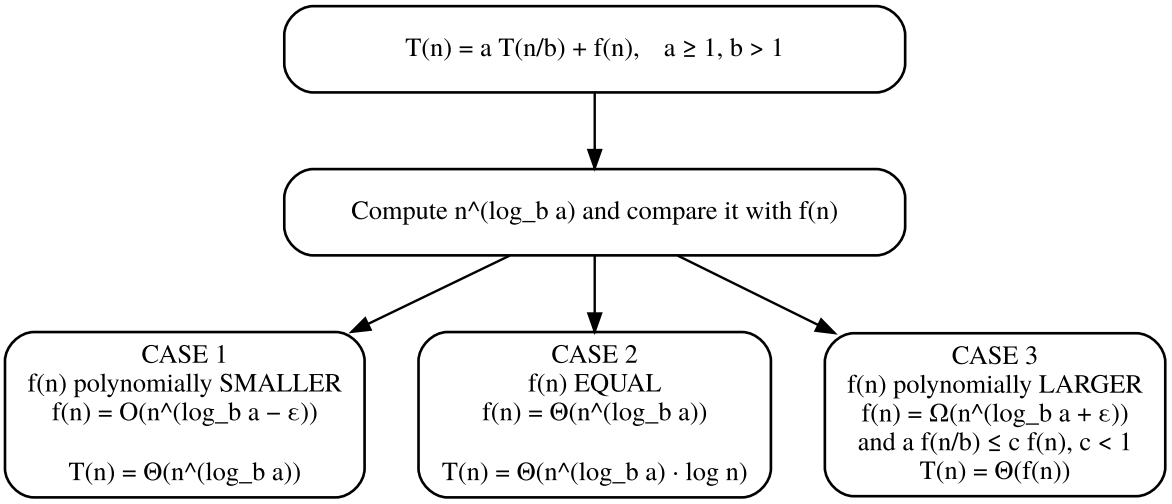
Q2. Stable vs. unstable sorting. *[2 Marks]*

Ans. A **stable** sort keeps the **relative order of equal keys** (for example 5a before 5b). **Stable:** Bubble, Insertion, **Merge**, Counting, Radix. **Unstable: Quick**, **Heap**, Selection.

Section B: 7-Mark Questions

Q3. Solve T(n) = 2T(n/2) + n and T(n) = 3T(n/9) + n. *[7 Marks]*

Ans. Master Theorem: for $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$.



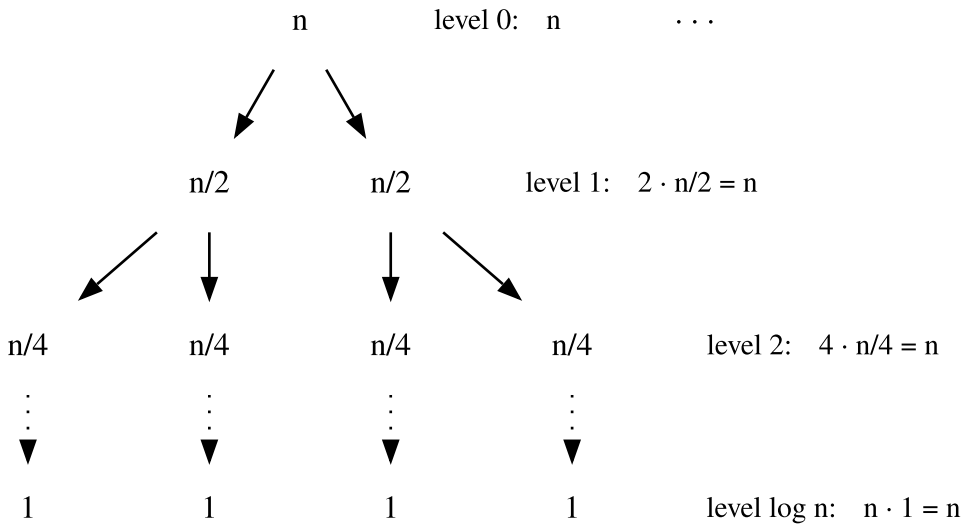
Master Theorem: compare f(n) with n^(log_b a)

(i) T(n) = 2T(n/2) + n

- a = 2, b = 2, so $n^{\log_2 2} = n$, and $f(n) = n$. They are equal, so this is **Case 2**.

$$T(n) = \Theta(n \log n)$$

Recursion tree check: every level costs n, and there are $\log_2 n + 1$ levels.



Recursion tree for T(n) = 2T(n/2) + n: every level costs n, and there are $\log_2 n + 1$ levels

(ii) T(n) = 3T(n/9) + n

- a = 3, b = 9, so $n^{\log_9 3} = n^{1/2}$. Since $f(n) = n = \Omega\left(n^{1/2+\varepsilon}\right)$, f(n) is larger, so check **Case 3**.
- **Regularity:** $3f(n/9) = n/3 \le cn$ with $c = 1/3 < 1$ ✓

$$T(n) = \Theta(n)$$

Recursion tree check: the level costs are $n + n/3 + n/9 + \cdots \le 1.5n$.

> **Exam Tip:** the Master Theorem does not apply to $T(n) = T(n-1) + n$ or to the "gap" cases.

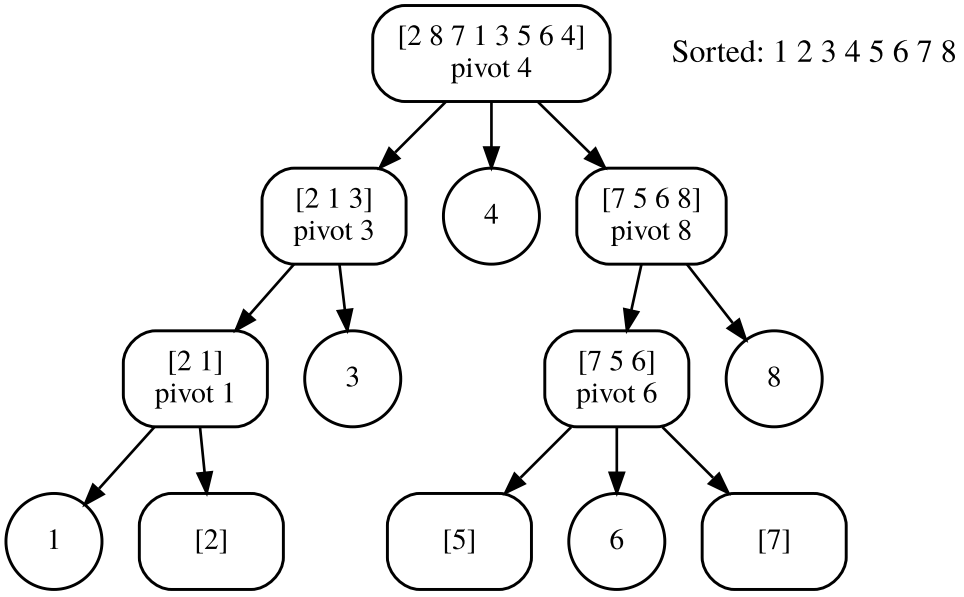
Q4. Explain Quick Sort, trace it, and say why it is preferred over Merge Sort. *[7 Marks]*

Ans. Quick Sort (Hoare) is a **divide-and-conquer, in-place** sort. It **partitions** the array around a **pivot** and then sorts both parts recursively.

```
PARTITION(A, p, r):  x = A[r]; i = p - 1
   for j = p to r - 1:  if A[j] <= x:  i = i + 1; swap A[i], A[j]
   swap A[i + 1], A[r];  return i + 1
```

Trace on A = [2, 8, 7, 1, 3, 5, 6, 4] (pivot = 4)

j	A[j]	≤ 4?	i	Array
1	2	Yes	1	2 8 7 1 3 5 6 4
4	1	Yes	2	2 1 7 8 3 5 6 4
5	3	Yes	3	2 1 3 8 7 5 6 4
end	swap A4, A8			2 1 3 **[4]** 7 5 6 8



Recursive calls of Quick Sort on [2, 8, 7, 1, 3, 5, 6, 4] (circled = pivot placed in its final position)

Complexity:

- Best and average: $\Theta(n \log n)$.
- **Worst** (sorted input): $\Theta(n^2)$.
- Space: $O(\log n)$.

Why it is preferred over Merge Sort:

1. **In-place:** no O(n) extra array is needed.
2. **Better cache locality.**
3. **Smaller constant factors.**
4. With a **randomized pivot**, the worst case is rare.

Quick Sort	Merge Sort
Worst case $O(n^2)$	Always $O(n \log n)$
$O(\log n)$ space	$O(n)$ space
Not stable	Stable

Conclusion: used in C's `qsort` and Java's primitive-array sort.

Q5. Trace Counting Sort and Heap Sort. *[7 Marks]*

Ans.

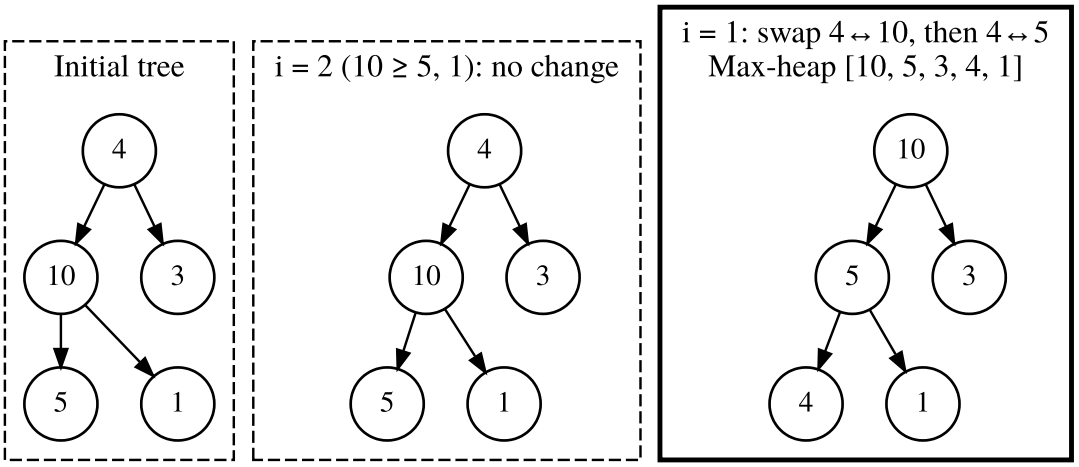
(a) Counting Sort on A = [2, 5, 3, 0, 2, 3, 0, 3], k = 5. This is a **non-comparison, stable** sort that runs in $O(n + k)$.

i	0	1	2	3	4	5
Count C[i]	2	0	2	3	0	1
Cumulative C[i]	2	2	4	7	7	8

Scan A from the **right**: put $B[C[A[j]]] = A[j]$, then decrease C. For example A[8] = 3 goes to B[7], and A[7] = 0 goes to B[2].

$$B = [0, 0, 2, 2, 3, 3, 3, 5]$$

(b) Heap Sort on A = [4, 10, 3, 5, 1]. **Build a max-heap**, then repeatedly swap the root with the last element and heapify. It runs in $O(n \log n)$, in place.



BUILD-MAX-HEAP on [4, 10, 3, 5, 1]

Pass	Swap	Heap after heapify / sorted part
1	10 ↔ 1	[5, 4, 3, 1 / 10]
2	5 ↔ 1	[4, 1, 3 / 5, 10]
3	4 ↔ 3	[3, 1 / 4, 5, 10]
4	3 ↔ 1	[1 / 3, 4, 5, 10]

Sorted: $[1, 3, 4, 5, 10]$

UNIT 2: ADVANCED DATA STRUCTURES

Section A: 2-Mark Questions

Q1. Properties of a Red-Black Tree. *[2 Marks]*

Ans.

1. Every node is **red or black**.
2. The **root is black**.
3. Every **leaf (NIL) is black**.
4. A **red node has black children**: no two reds in a row.
5. **Every path from a node to its leaves has the same number of black nodes** (the black-height).

So the height is at most $2 \log_2 (n + 1)$.

Q2. Define a B-Tree of order m. *[2 Marks]*

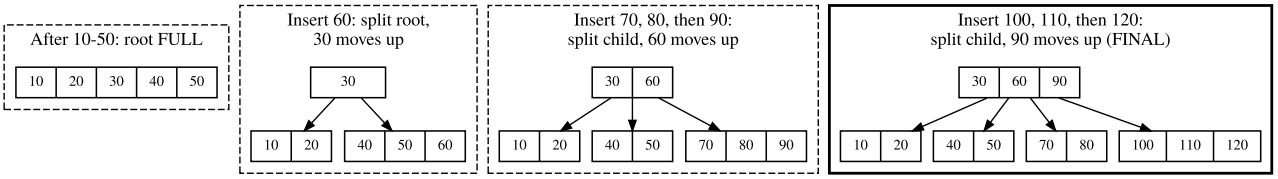
Ans. A balanced m-way search tree in which:

- Each node has at most **m children and m − 1 keys**.
- Each non-root node has at least **⌈m/2⌉ children**.
- The root has at least 2 children.
- **All leaves are at the same level.**
- Keys in each node are sorted.

Section B: 7-Mark Questions

Q3. Insert keys into an empty B-Tree of minimum degree t = 3. *[7 Marks]*

Ans. With **t = 3**, each node holds **2 to 5 keys** (the root 1 to 5). A **full node (5 keys) is split**, and its **median moves up**. CLRS splits full nodes **on the way down**.



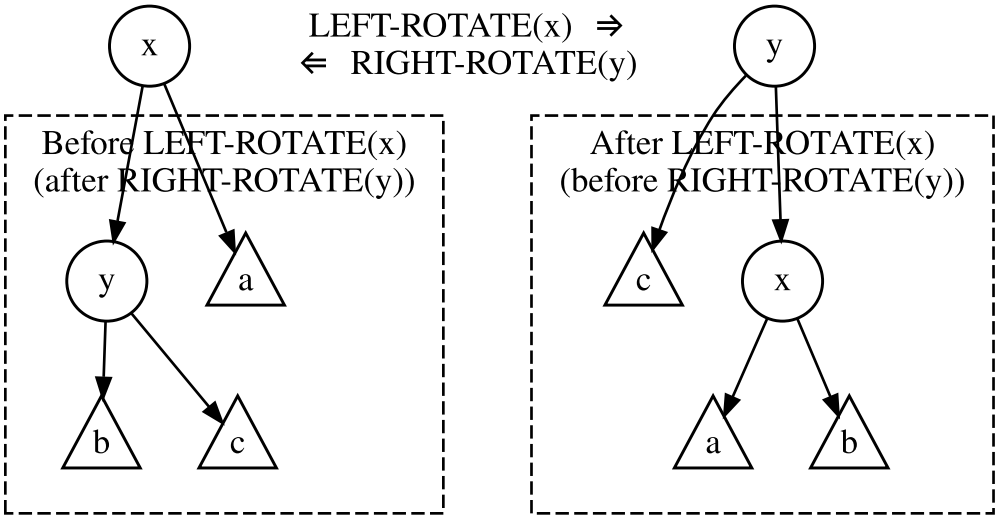
B-Tree (t = 3, max 5 keys per node): inserting 10, 20, ..., 120

> **Exam Tip:** for t = 3: max keys = 2t − 1 = 5, min keys = t − 1 = 2, max children = 2t = 6.

Conclusion: B-Trees keep **all leaves at the same level**, giving $O(\log n)$ disk accesses. They are used in DBMS and file systems.

Q4. Show insertions and rotations in a Red-Black Tree. *[7 Marks]*

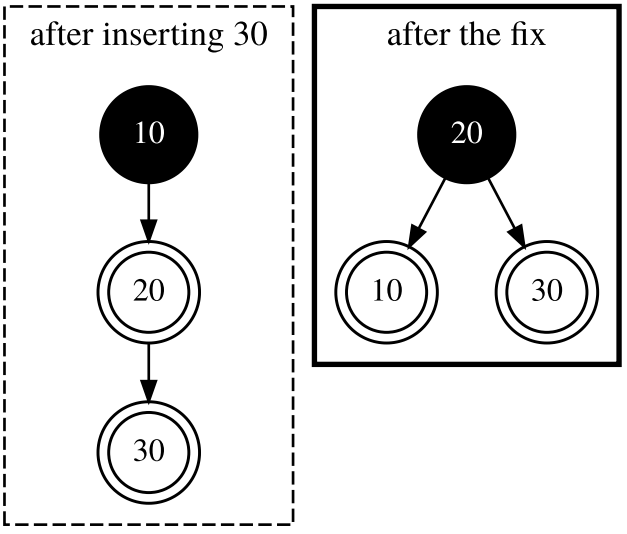
Ans. A new node is inserted as **RED**. If its parent is also red, fix it with **recolouring or rotations**.



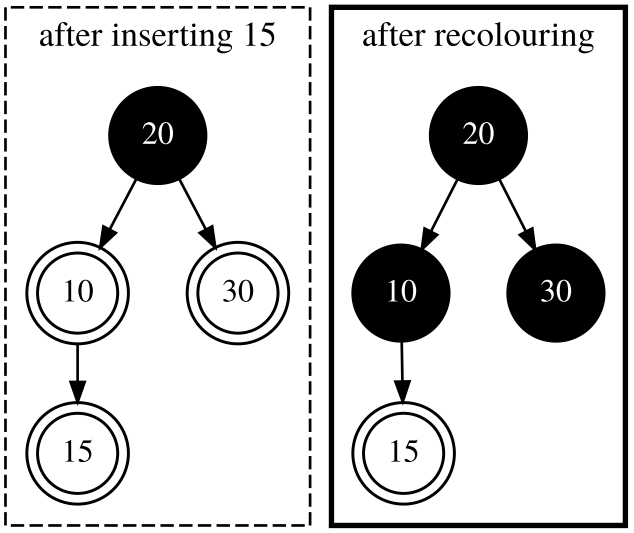
Left and right rotation (in-order sequence a, x, b, y, c is unchanged; O(1) time)

Case	Condition	Fix
1	Uncle is **RED**	Recolour: parent and uncle black, grandparent red
2	Uncle black, **triangle** (LR / RL)	Rotate at the parent (becomes Case 3)
3	Uncle black, **line** (LL / RR)	Rotate at the grandparent, swap colours

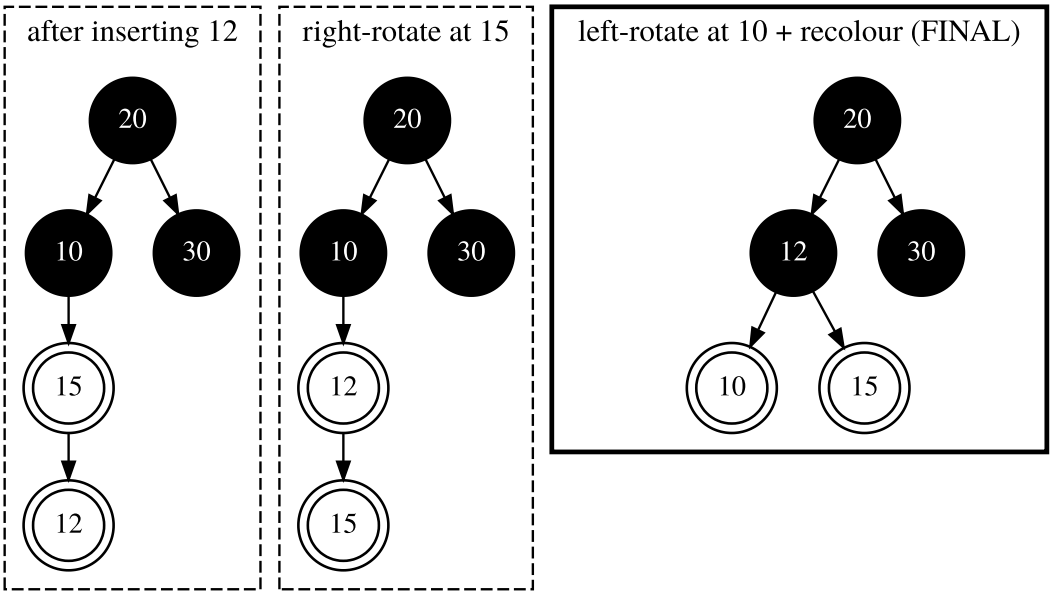
Insert 10, 20, 30, 15, 12 (filled circle = BLACK, double circle = RED)



Insert 30: Case 3 (RR line). Left-rotate at 10 and swap the colours of 10 and 20



Insert 15: Case 1 (uncle 30 is RED). Recolour 10 and 30 BLACK, 20 RED, then the root becomes BLACK again

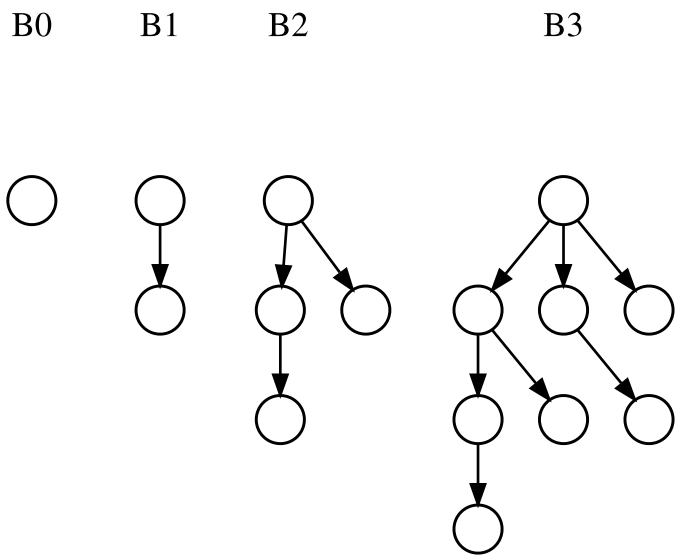


Insert 12: RL triangle. Case 2 (right-rotate at 15), then Case 3 (left-rotate at 10, recolour 12 BLACK and 10 RED)

Conclusion: at most **2 rotations per insert**. Used in Java TreeMap and C++ `std::map`.

Q5. Explain the structure, properties and union of a Binomial Heap. *[7 Marks]*

Ans. A **binomial heap** is a set of **binomial trees**. Each tree is **min-heap ordered**, and there is **at most one tree of each degree**. Its **union takes O(log n)**.



Binomial trees B0 to B3: Bk has 2^k nodes and height k; it is formed by linking two B(k−1) trees

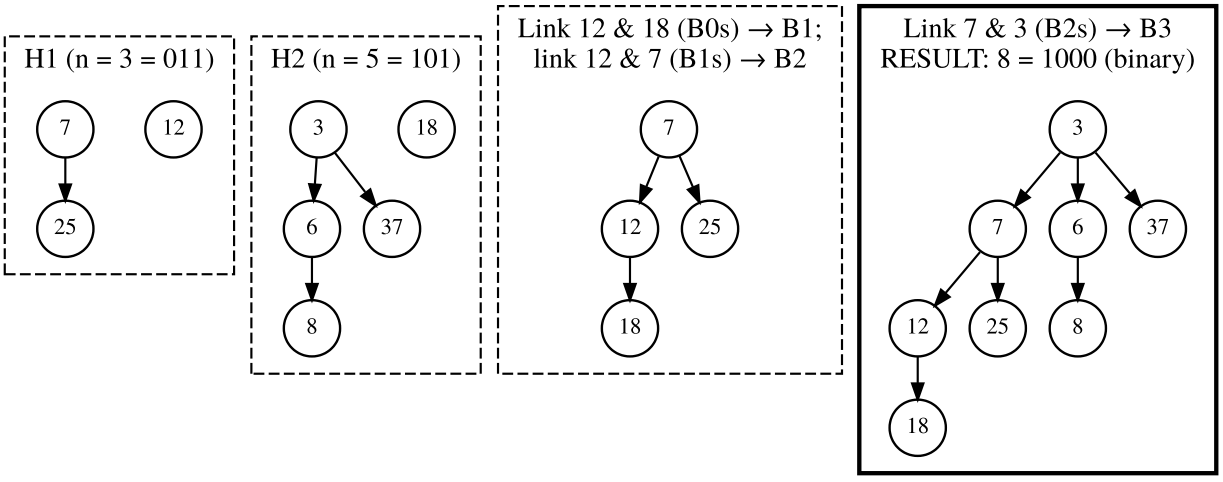
Properties of B_k:

- 2^k nodes and height k.
- $\binom{k}{i}$ nodes at depth i.
- The root has degree k.

Properties of the heap: a heap with n nodes has trees matching the **1-bits of n in binary**. For example $13 = 1101_2$ gives B_3, B_2, B_0.

Union steps:

1. **Merge** the root lists by degree.
2. **Link** trees of equal degree; the smaller key becomes the root. This works like binary addition.



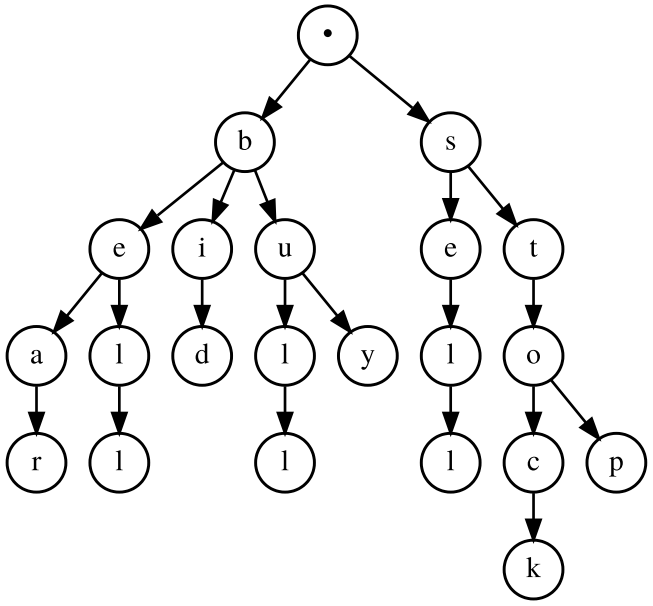
BINOMIAL-HEAP-UNION of H1 (3 nodes) and H2 (5 nodes)

Operation	Binary heap	Binomial heap
Insert	$O(\log n)$	$O(\log n)$
Extract-min	$O(\log n)$	$O(\log n)$
Union	**O(n)**	**O(log n)**

Conclusion: a mergeable priority queue, and the basis of Fibonacci heaps.

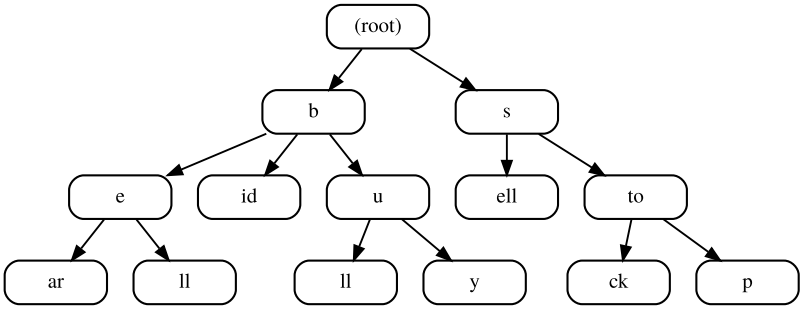
Q6. Construct a compressed trie for {bear, bell, bid, bull, buy, sell, stock, stop}. *[7 Marks]*

Ans. A **trie** stores strings character by character. A **compressed trie** merges **single-child chains into one edge**, so every internal node has **at least 2 children**.



Standard trie for {bear, bell, bid, bull, buy, sell, stock, stop} (22 nodes)

Compress: a→r becomes “ar”, i→d becomes “id”, e→l→l becomes “ell”, t→o becomes “to”, c→k becomes “ck”.



Compressed trie: single-child chains merged into one edge (14 nodes)

Standard trie	Compressed trie
Edge = 1 character	Edge = substring
22 nodes here	**14 nodes** here
O(total length) nodes	O(number of strings) nodes

Conclusion: used in autocomplete, IP routing and suffix trees.

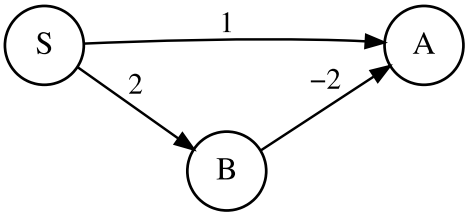
UNIT 3: GREEDY ALGORITHMS & GRAPHS

Section A: 2-Mark Questions

Q1. Greedy choice property; when Dijkstra fails. *[2 Marks]*

Ans.

- **Greedy choice property:** a **globally optimal solution can be reached by locally optimal choices**. Example: activity selection, always pick the activity that finishes earliest.
- **Dijkstra fails with negative edges**, because it finalizes a vertex too early.

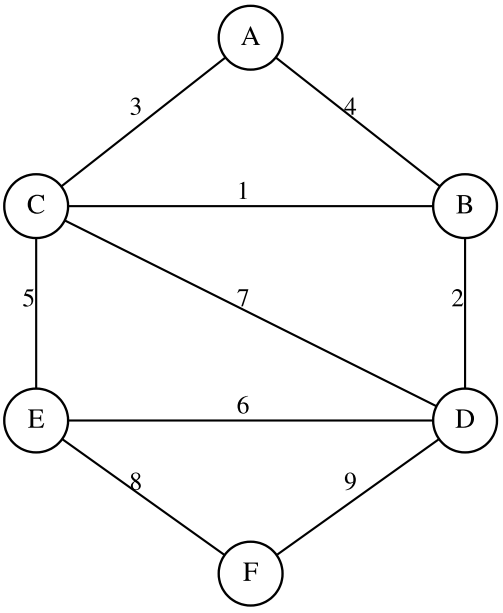


Dijkstra fails with a negative edge: it finalizes $d(A) = 1$, but the true shortest distance is $S \to B \to A = 2 + (-2) = 0$

Section B: 7-Mark Questions

Q2. Trace Kruskal's and Prim's algorithms for the MST. *[7 Marks]*

Ans. An **MST** connects all V vertices with **V – 1 edges of minimum total weight**. Kruskal and Prim are **greedy** algorithms.



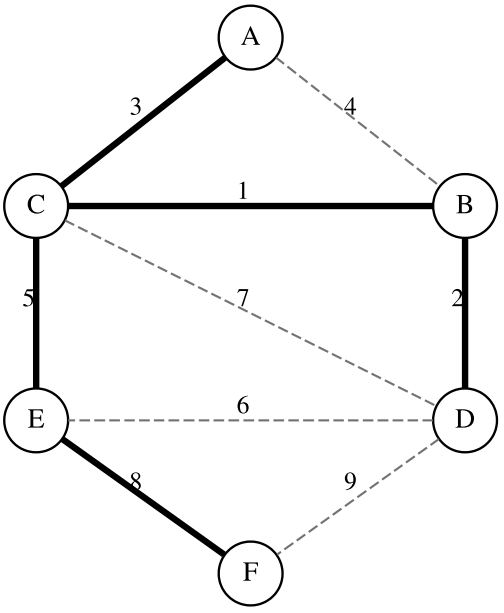
Given weighted graph (6 vertices, 9 edges)

Kruskal (sort the edges, add an edge if it makes no cycle, checked with Union-Find)

Edge	B-C 1	B-D 2	A-C 3	A-B 4	C-E 5	D-E 6	C-D 7	E-F 8
Action	✓	✓	✓	× cycle	✓	×	×	✓

Prim (from A, always add the cheapest edge leaving the tree)

Tree so far	{A}	{A,C}	{A,C,B}	{A,B,C,D}	{A,B,C,D,E}
Edge chosen	A-C 3	C-B 1	B-D 2	C-E 5	E-F 8



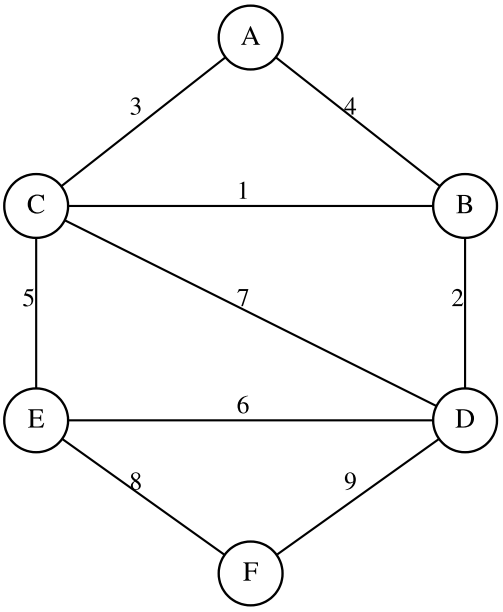
Minimum spanning tree (bold edges), total weight = 1 + 2 + 3 + 5 + 8 = 19

Kruskal	Prim
Edge-based; forest grows	Vertex-based; one tree grows
Union-Find	Priority queue
$O(E \log E)$; best for sparse graphs	$O(E \log V)$; best for dense graphs

Conclusion: both give MST weight = **19**. Used to design cable and road networks.

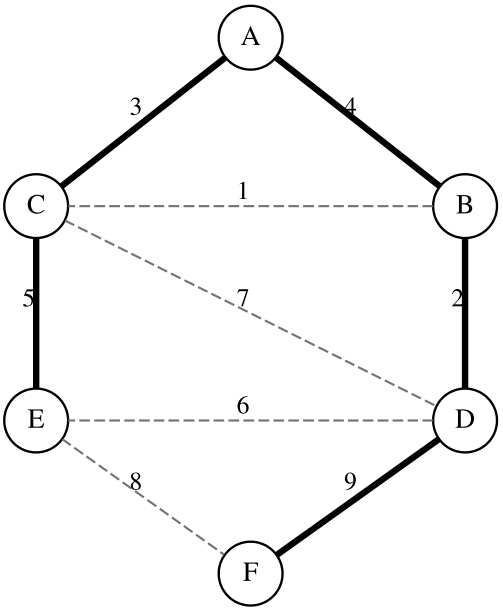
Q3. Find shortest paths using Dijkstra's algorithm. *[7 Marks]*

Ans. Dijkstra is a **greedy single-source shortest-path** algorithm for **non-negative** weights. It repeatedly picks the vertex with the minimum distance and **relaxes** its edges: if $d[u] + w(u, v) < d[v]$ then $d[v] = d[u] + w(u, v)$.



Graph used (source = A)

Picked	A	B	C	D	E	F
Init	0	∞	∞	∞	∞	∞
A (0)	**0**	4	3	∞	∞	∞
C (3)	0	4	**3**	10	8	∞
B (4)	0	**4**	3	6	8	∞
D (6)	0	4	3	**6**	8	15
E (8)	0	4	3	6	**8**	15
F (15)	0	4	3	6	8	**15**



Shortest-path tree from A (bold edges); distances A0, B4, C3, D6, E8, F15

Answer: A → B = 4, A → C = 3, A → B → D = 6, A → C → E = 8, A → B → D → F = 15.

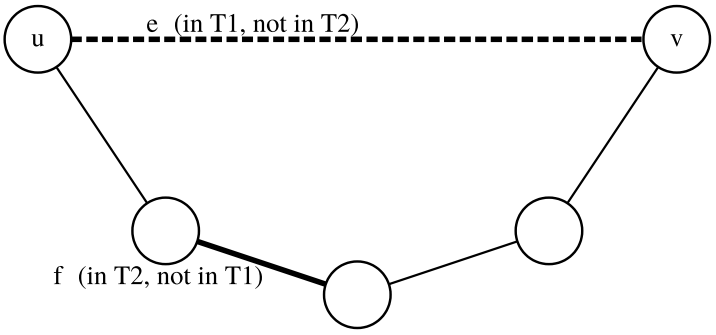
Complexity: $O\left(V^2\right)$ with an array, or $O\left(\left(V+E\right)\log V\right)$ with a heap.

Conclusion: used in GPS navigation and OSPF routing. For negative edges, use **Bellman-Ford**.

Q4. Prove that distinct edge weights give a unique MST. *[7 Marks]*

Ans. Proof by contradiction (exchange argument):

1. **Assume** there are two different MSTs, $T_1 \neq T_2$.
2. Let **e** be the **minimum-weight edge** in $T_1 \oplus T_2$ (the edges in only one of the trees). Say $e \in T_1$ and $e \notin T_2$.
3. Adding e to T_2 creates exactly **one cycle C**.
4. C must contain an edge $f \in T_2$ with $f \notin T_1$, because otherwise T_1 would contain a cycle.
5. f is in $T_1 \oplus T_2$, and the weights are distinct, so $w(e) < w(f)$.
6. $T_2' = T_2 + e - f$ is a spanning tree with $w(T_2') = w(T_2) + w(e) - w(f) < w(T_2)$. **Contradiction.**
7. **Therefore the MST is unique.** ∎



path in T2 from u to v

Exchange argument: adding e to T2 creates a cycle; removing the heavier edge f gives a lighter spanning tree

> **Remember:** the converse is false. With equal weights there may be many MSTs (for example a triangle with all weights 1 has 3).

UNIT 4: DYNAMIC PROGRAMMING

Section A: 2-Mark Questions

Q1. Principle of Optimality; Divide and Conquer vs. DP. *[2 Marks]*

Ans. Principle of optimality (Bellman): an optimal solution contains **optimal solutions to its sub-problems**.

Divide & Conquer	Dynamic Programming
Independent sub-problems	**Overlapping** sub-problems
Solves the same sub-problem again	Solves each once and **stores** it
Merge Sort, Quick Sort	Knapsack, LCS, MCM

Section B: 7-Mark Questions

Q2. Solve the 0/1 Knapsack problem with DP. W = 5, (w, v) = (2,3), (3,4), (4,5), (5,6). *[7 Marks]*

Ans.

$$V[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ V[i-1][w] & w_i > w \\ \max\left(V[i-1][w],\ v_i + V[i-1][w-w_i]\right) & \text{otherwise} \end{cases}$$

i w	0	1	2	3	4	5
0	0	0	0	0	0	0
1 (2, 3)	0	0	3	3	3	3
2 (3, 4)	0	0	3	4	4	**7**
3 (4, 5)	0	0	3	4	5	7
4 (5, 6)	0	0	3	4	5	**7**

Sample: $V[2][5] = \max\left(V[1][5], 4 + V[1][2]\right) = \max\left(3, 7\right) = 7.$

Traceback:

- V[4][5] = V[3][5] = V[2][5], so items 4 and 3 are not taken.
- V[2][5] ≠ V[1][5], so **item 2 is taken**. Remaining capacity w = 2.
- V[1][2] ≠ V[0][2], so **item 1 is taken**.

$$\text{Items } \{1, 2\},\ \text{ maximum value} = 7,\ \ x = (1, 1, 0, 0)$$

Complexity: $O(nW)$, which is pseudo-polynomial.

Q3. Matrix Chain Multiplication: find the optimal parenthesization. p = ⟨5, 4, 6, 2, 7⟩. *[7 Marks]*

Ans.

$$m[i][j] = \min_{i \le k < j} \{m[i][k] + m[k+1][j] + p_{i-1} p_k p_j\}, \qquad m[i][i] = 0$$

L = 2:

- m[1][2] = 5·4·6 = 120
- m[2][3] = 4·6·2 = 48
- m[3][4] = 6·2·7 = 84

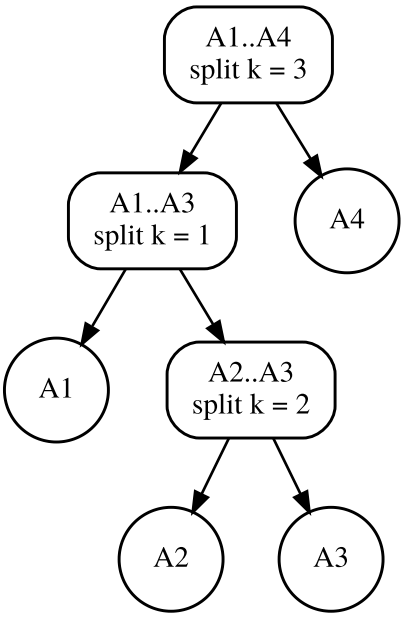
L = 3:

- m[1][3] = min(0 + 48 + 40, 120 + 0 + 60) = **88** (k = 1)
- m[2][4] = min(0 + 84 + 168, 48 + 0 + 56) = **104** (k = 3)

L = 4:

- m[1][4] = min(0 + 104 + 140, 120 + 84 + 210, 88 + 0 + 70) = **158** (k = 3)

m	1	2	3	4
1	0	120	88	**158**
2		0	48	104
3			0	84



Parenthesization tree from the s table: ((A1 (A2 A3)) A4)

$$\left(\left(A_1\left(A_2A_3\right)\right)A_4\right), \quad \text{cost} = 158$$

Complexity: $O\left(n^3\right)$ time and $O\left(n^2\right)$ space.

Q4. Find the LCS of X = ABCBDAB and Y = BDCABA. *[7 Marks]*

Ans.

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & X_i = Y_j \\ \max\left(c[i-1][j], c[i][j-1]\right) & X_i \neq Y_j \end{cases}$$

	Ø	B	D	C	A	B	A
A	0	0	0	0	1↖	1	1↖
B	0	1↖	1	1	1	2↖	2
C	0	1	1	2↖	2	2	2
B	0	1↖	1	2	2	3↖	3
D	0	1	2↖	2	2	3	3
A	0	1	2	2	3↖	3	4↖
B	0	1↖	2	2	3	4↖	**4**

(↖ marks a match, where 1 is added from the diagonal.)

Traceback from c[7][6], following the ↖ cells, picks B, C, B, A.

$$\text{LCS} = \text{BCBA}, \quad \text{length} = 4$$

Complexity: $O(mn)$.

UNIT 5: BACKTRACKING & NP

Section A: 2-Mark Questions

Q1. Backtracking; N-Queens problem. *[2 Marks]*

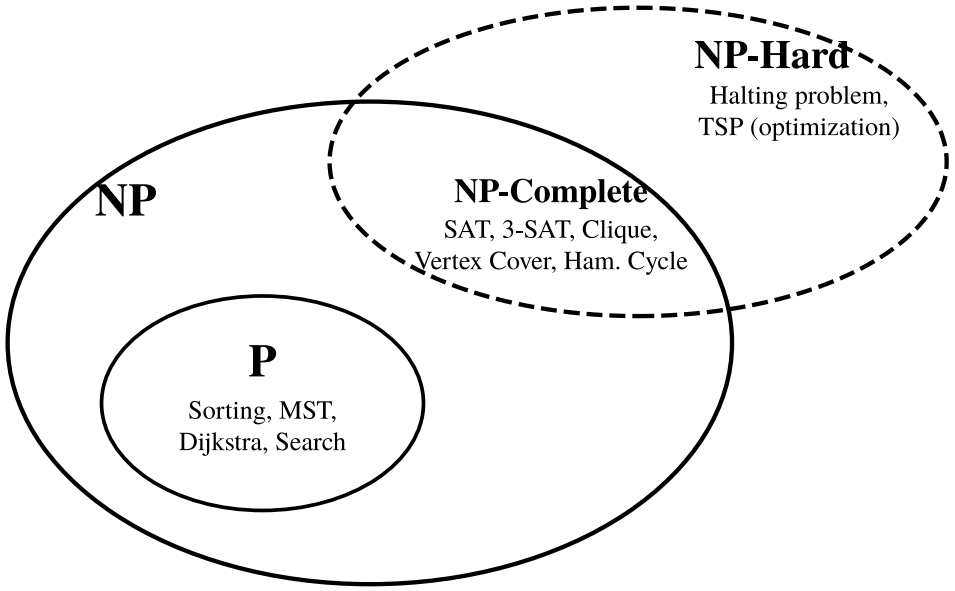
Ans.

- **Backtracking:** a **DFS of the state-space tree**. Build the solution step by step, and **undo (backtrack)** as soon as a partial solution violates a constraint.
- **N-Queens:** place N queens on an N×N board so that **no two share a row, column or diagonal**. Two queens attack if $x_i = x_j$ or $|x_i - x_j| = |i - j|$. 4-Queens has 2 solutions; 8-Queens has 92.

Section B: 7-Mark Questions

Q2. Explain P, NP, NP-Complete and NP-Hard with a diagram. *[7 Marks]*

Ans.

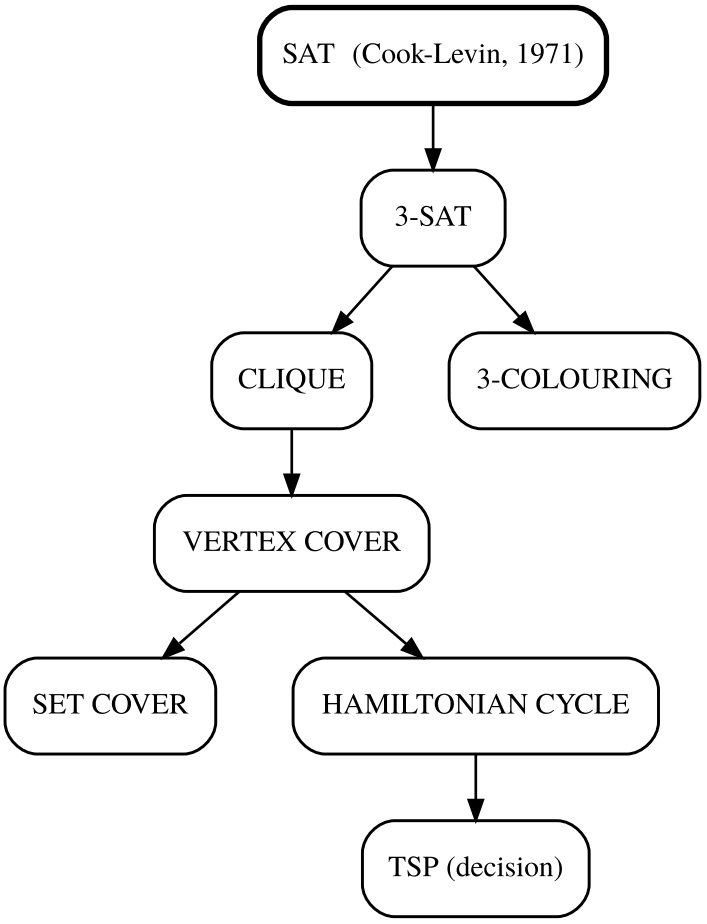


$NP\text{-}Complete = NP \cap NP\text{-}Hard; \quad P \subseteq NP$

Relationship between P, NP, NP-Complete and NP-Hard (assuming $P \neq NP$)

1. **P:** problems **solvable** in polynomial time. Examples: sorting, MST, Dijkstra.
2. **NP:** problems whose solutions are **verifiable** in polynomial time. **P ⊆ NP.** Example: Sudoku.
3. **NP-Hard: every NP problem reduces to it** ($L \leq_p H$). It **need not be in NP**. Example: the Halting problem.
4. **NP-Complete = NP ∩ NP-Hard.** Examples: **SAT (Cook-Levin, the first)**, 3-SAT, Clique, Vertex Cover.

5. **To prove X is NP-Complete:** (i) show $X \in NP$, and (ii) reduce a known NPC problem to X.
6. If **any NPC problem is solved in polynomial time, then P = NP**. This question is still unsolved.



Chain of polynomial-time reductions between NP-Complete problems

	P	NP	NPC	NP-Hard
Solvable in poly time	Yes	?	?	?
Verifiable in poly time	Yes	Yes	Yes	Not necessarily
In NP?	Yes	Yes	Yes	Not necessarily

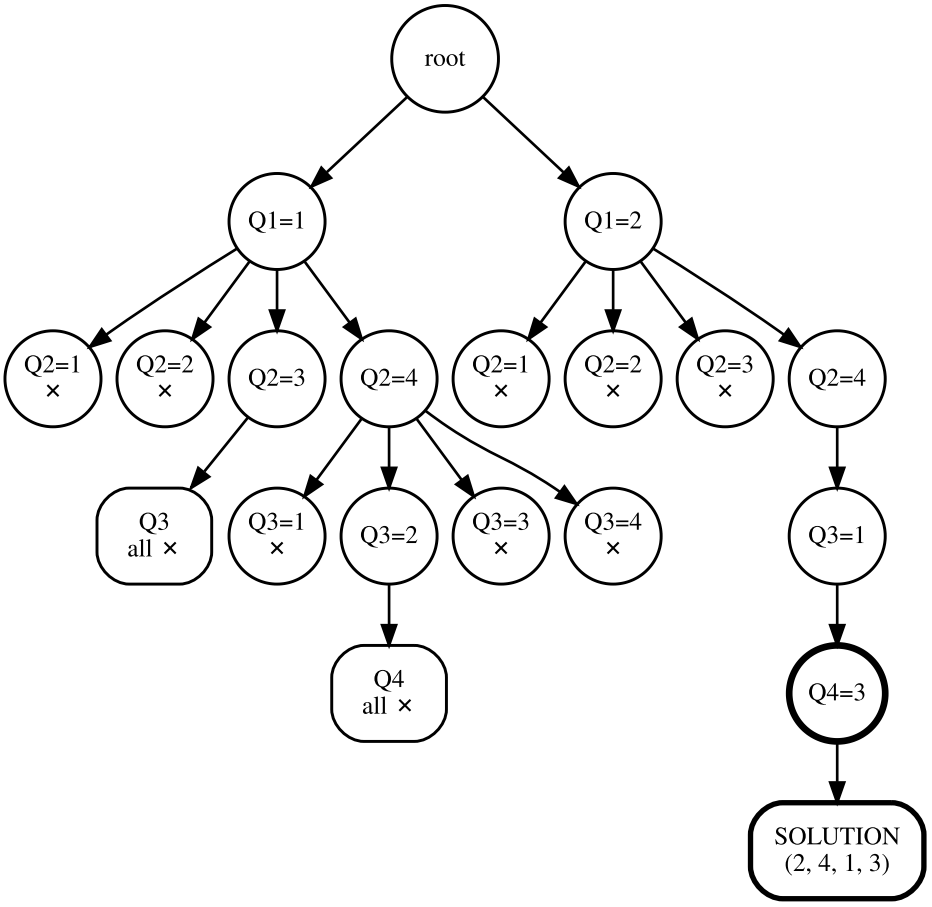
Conclusion: for NPC problems (TSP, scheduling) we use **approximation and heuristics**.

Q3. Solve the 4-Queens problem using backtracking. *[7 Marks]*

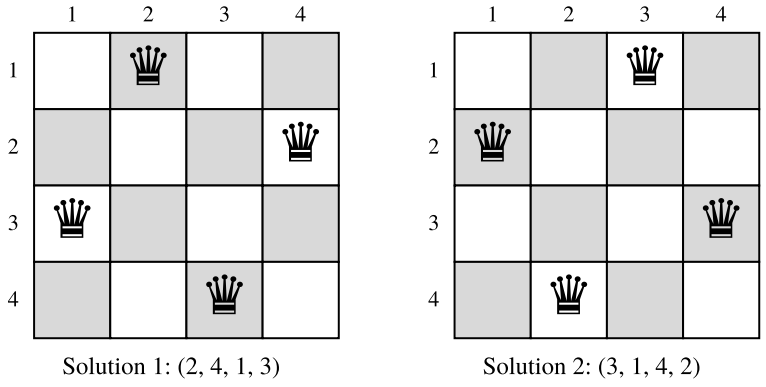
Ans. Place the queens **row by row**. For each row, try columns 1 to 4. **PLACE(k, i)** checks the column and the diagonals, and the algorithm **backtracks** when no column is safe.

```
NQUEENS(k, n): for i = 1 to n:
    if PLACE(k, i): x[k] = i;  if k == n: print x  else: NQUEENS(k+1, n)
PLACE(k, i): for j = 1 to k-1:  if x[j] == i or |x[j]-i| == |j-k|: return false
             return true
```

Step	Try	Result
1	Q1 = 1, Q2 = 3	Q3 has no safe column, so **backtrack**
2	Q2 = 4, Q3 = 2	Q4 has no safe column, so **backtrack** to Q1
3	Q1 = 2, Q2 = 4, Q3 = 1, Q4 = 3	**Solution**



Pruned state-space tree for 4-Queens (× = killed by the bounding function)



The two solutions of 4-Queens (Solution 2 is the mirror image of Solution 1)

Answer: (2, 4, 1, 3) and its mirror (3, 1, 4, 2). **8-Queens:** 92 solutions; the first found is (1, 5, 8, 6, 3, 7, 2, 4).

Conclusion: backtracking prunes the search space. It is used in Sudoku solvers and timetabling.

Machine Learning Techniques

BCS-055 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** ML numericals (ID3, Naive Bayes, Candidate Elimination, K-Means, Backpropagation) come up **every year**. Write the **formula, then the substitution, then the boxed answer**. Each one is solved step by step below, inside its unit.

UNIT 1: INTRODUCTION & LEARNING SYSTEMS

Section A: 2-Mark Questions

Q1. Define a well-posed learning problem. *[2 Marks]*

Ans. Tom Mitchell: *"A program learns from **experience E** with respect to **task T** and **performance measure P** if its performance at T, measured by P, improves with E."* A problem is **well-posed** when T, P and E are all defined. Example (checkers): **T** = play checkers, **P** = % of games won, **E** = games played against itself.

Q2. ML vs. Deep Learning; Occam's Razor. *[2 Marks]*

Ans.

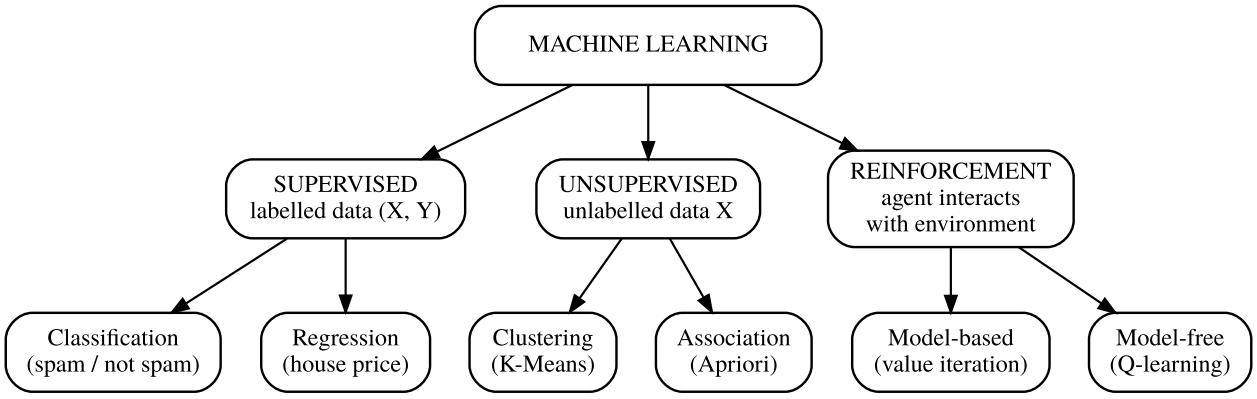
Machine Learning	Deep Learning
Manual feature extraction	**Automatic** feature learning
Works on small data, runs on a CPU	Needs **big data** and a **GPU**
Interpretable (decision tree)	Black box (CNN, RNN)

Occam's Razor: *prefer the **simplest hypothesis** that fits the data*. Simpler models **generalize better** and overfit less. For example, ID3 prefers shorter trees.

Section B: 7-Mark Questions

Q3. Compare supervised, unsupervised and reinforcement learning with examples. *[7 Marks]*

Ans. The three paradigms differ in the **kind of feedback** the learner gets.



The three learning paradigms

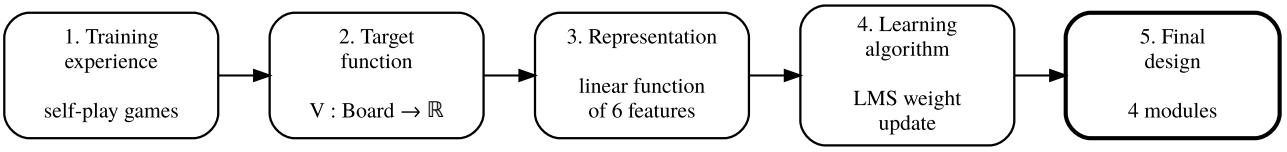
1. **Supervised:**
   - Learns from **labelled data (X, Y)** a function $f: X \to Y$.
   - **Classification** and **regression**.
   - Algorithms: SVM, Decision Tree, k-NN.
   - Example: **spam filtering**, house-price prediction.
2. **Unsupervised:**
   - **Unlabelled data**; it finds hidden structure.
   - **Clustering**, association, PCA.
   - Algorithms: K-Means, Apriori.
   - Example: **customer segmentation**, market-basket analysis.
3. **Reinforcement:**
   - An **agent** acts in an **environment** and gets **rewards**. It learns a **policy** by trial and error.
   - Algorithms: Q-learning, DQN.
   - Example: **AlphaGo**, self-driving cars.

Basis	Supervised	Unsupervised	Reinforcement
Data	Labelled	Unlabelled	Interaction
Feedback	Correct answer	None	**Delayed reward**
Goal	Predict outputs	Find patterns	Maximize total reward
Example	Spam filter	Customer segments	Game playing

Conclusion: ChatGPT uses all three: supervised training, self-supervised learning, and **RLHF**.

Q4. Design a learning system: the checkers case study. *[7 Marks]*

Ans. Designing a learner means choosing the **training experience, the target function, its representation and the learning algorithm**.



Design choices for the checkers learner (the option chosen at each step)

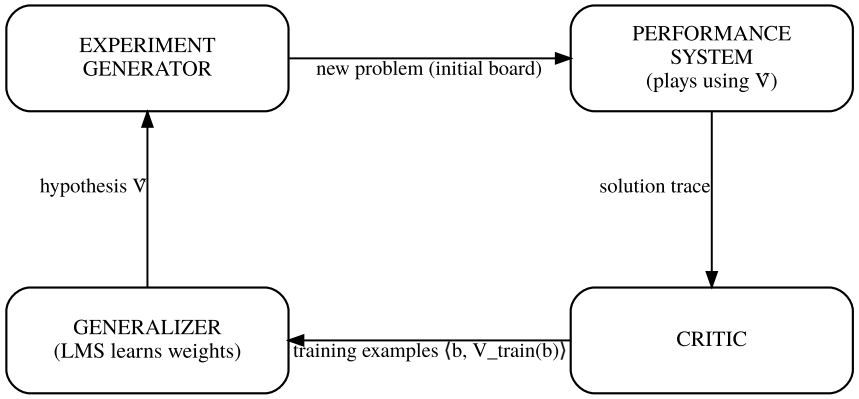
1. **Training experience:** the program plays **games against itself**. This gives **indirect feedback** (only the final win or loss), which raises the credit-assignment problem.
2. **Target function:** V: Board $\rightarrow \mathbb{R}$, where V = +100 for a win, −100 for a loss and 0 for a draw.
3. **Representation:** a linear combination of 6 features:

$$\widehat{V}(b) = w_0 + w_1 x_1 + \cdots + w_6 x_6$$

Here x_1, x_2 = black and red pieces, x_3, x_4 = black and red kings, x_5, x_6 = pieces threatened. 4. **Learning algorithm (LMS):** estimate the training value as $V_{train}(b) \leftarrow \widehat{V}(\text{Successor}(b))$, then update each weight:

$$w_i \leftarrow w_i + \eta \left(V_{train}(b) - \widehat{V}(b)\right) x_i$$

5. **Final design** (four modules in a loop):



Final design: four modules working in a loop

Conclusion: this framework led to Samuel's checkers player and the ideas behind AlphaZero.

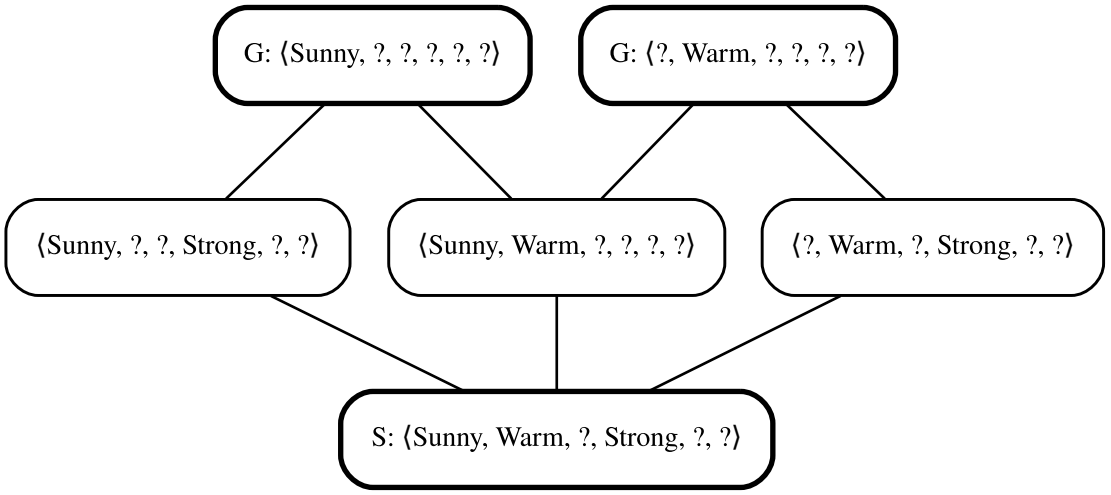
Q5. Trace the Candidate Elimination algorithm (EnjoySport). *[7 Marks]*

Ans. Candidate Elimination finds the **version space** using two boundaries. **S** is the most specific boundary and **G** is the most general. **A positive example generalizes S; a negative example specializes G.**

Ex	Sky	Temp	Humid	Wind	Water	Forecast	Sport
1	Sunny	Warm	Normal	Strong	Warm	Same	**Yes**
2	Sunny	Warm	High	Strong	Warm	Same	**Yes**
3	Rainy	Cold	High	Strong	Warm	Change	**No**
4	Sunny	Warm	High	Strong	Cool	Change	**Yes**

Step	S	G
0	⟨ø,ø,ø,ø,ø,ø⟩	⟨?,?,?,?,?,?⟩
1 (+)	⟨Sunny, Warm, Normal, Strong, Warm, Same⟩	⟨?,?,?,?,?,?⟩
2 (+)	⟨Sunny, Warm, **?**, Strong, Warm, Same⟩	⟨?,?,?,?,?,?⟩
3 (−)	unchanged	⟨Sunny,?,?,?,?,?⟩, ⟨?,Warm,?,?,?,?⟩, ⟨?,?,?,?,?,Same⟩
4 (+)	⟨Sunny, Warm, ?, Strong, **?, ?**⟩	⟨Sunny,?,?,?,?,?⟩, ⟨?,Warm,?,?,?,?⟩

*At step 3, only specializations that are **more general than S** are kept. At step 4, ⟨?,?,?,?,?,Same⟩ is **removed** because it does not cover the positive example (Forecast = Change).*



Final version space: the S boundary, the three hypotheses between, and the G boundary

S = ⟨Sunny, Warm, ?, Strong, ?, ?⟩ G = {⟨Sunny,?,?,?,?,?⟩, ⟨?,Warm,?,?,?,?⟩}

UNIT 2: REGRESSION, BAYESIAN LEARNING & SVM

Section A: 2-Mark Questions

Q1. Maximum Likelihood hypothesis; two uses of logistic regression. *[2 Marks]*

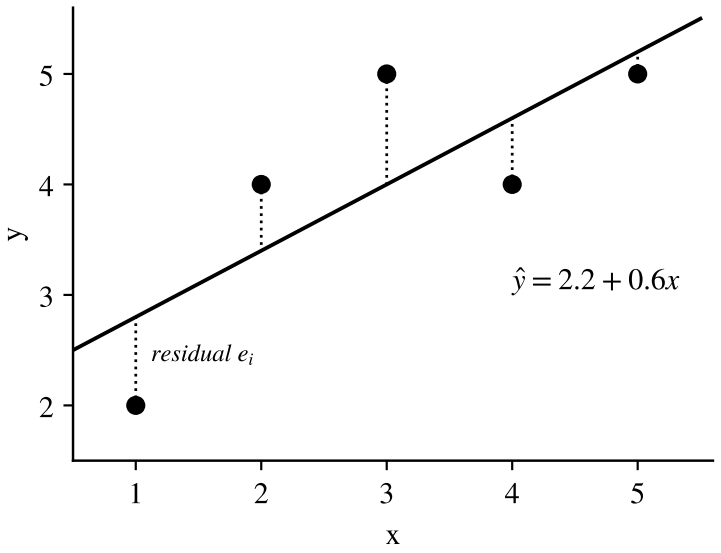
Ans.

- $h_{ML} = \arg\max_{h \in H} P(D \mid h)$: the hypothesis that makes the observed data **most probable**. It is the same as MAP when all hypotheses are equally likely.
- **Logistic regression uses:** (1) **disease prediction** (Yes/No), (2) **spam detection**. Also fraud detection and churn prediction.

Section B: 7-Mark Questions

Q2. Derive the least-squares solution for linear regression. *[7 Marks]*

Ans. For the line $\hat{y} = \beta_0 + \beta_1 x$, **least squares** minimizes $SSE = \sum \left(y_i - \beta_0 - \beta_1 x_i\right)^2$.



Least-squares line through the five data points; each residual is the vertical gap between a point and the line

Step 1:

$$\frac{\partial SSE}{\partial \beta_0} = -2\sum \left(y_i - \beta_0 - \beta_1 x_i\right) = 0 \;\Rightarrow\; \beta_0 = \bar{y} - \beta_1 \bar{x}$$

Step 2:

$$\frac{\partial SSE}{\partial \beta_1} = -2\sum x_i\left(y_i - \beta_0 - \beta_1 x_i\right) = 0 \;\Rightarrow\; \sum x_i y_i = \beta_0 \sum x_i + \beta_1 \sum x_i^2$$

Step 3: substitute β_0 and simplify:

$$\beta_1 = \frac{\sum (x_i - \bar{x})\left(y_i - \bar{y}\right)}{\sum (x_i - \bar{x})^2}, \qquad \beta_0 = \bar{y} - \beta_1 \bar{x} \qquad \text{Matrix form: } \beta = \left(X^T X\right)^{-1} X^T y$$

Example: x = 1, 2, 3, 4, 5 and y = 2, 4, 5, 4, 5. Here $\bar{x} = 3$, $\bar{y} = 4$, $\sum (x - \bar{x})\left(y - \bar{y}\right) = 6$ and $\sum (x - \bar{x})^2 = 10$.

$$\beta_1 = 0.6, \qquad \beta_0 = 4 - 1.8 = 2.2 \qquad \Rightarrow \qquad \hat{y} = 2.2 + 0.6x$$

Conclusion: used for sales forecasting and price prediction. It is sensitive to outliers.

Q3. Naive Bayes classifier: classify X = (Confident = Yes, Studied = Yes, Sick = No). *[7 Marks]*

Ans. Naive Bayes uses Bayes' theorem and assumes the **features are independent given the class**:

$$P(C \mid X) \propto P(C) \prod_i P(x_i \mid C)$$

No.	Confident	Studied	Sick	Result
1	Yes	No	No	Fail
2	Yes	No	Yes	Pass
3	No	Yes	Yes	Fail
4	No	Yes	No	Pass
5	Yes	Yes	Yes	Pass

Step 1: Priors. $P(\text{Pass}) = \frac{3}{5}$ and $P(\text{Fail}) = \frac{2}{5}$.

Step 2: Likelihoods.

	Pass (3 rows)	Fail (2 rows)
Confident = Yes	2/3	1/2
Studied = Yes	2/3	1/2
Sick = No	1/3	1/2

Step 3: Multiply.

$$\text{Pass: } \frac{3}{5} \cdot \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = 0.0889 \qquad \text{Fail: } \frac{2}{5} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = 0.0500$$

Step 4: Posterior.

$$P(\text{Pass} \mid X) = \frac{0.0889}{0.1389} = 0.64$$

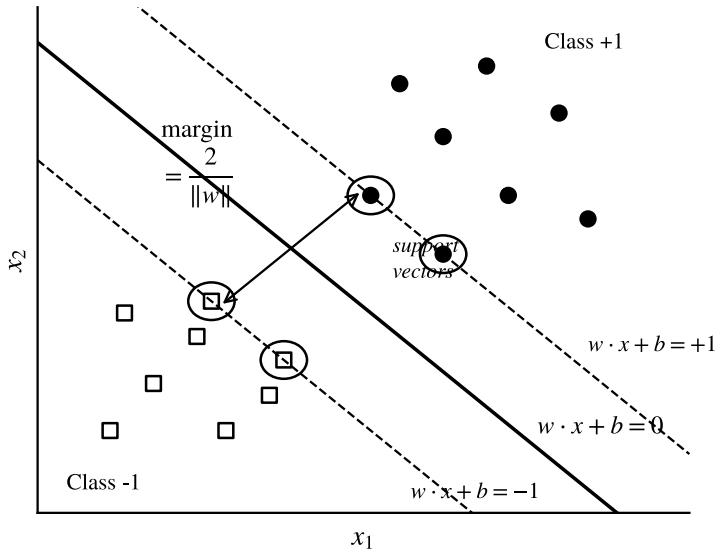
Result=**PASS**

If only (Confident, Sick) is given: Pass = 0.6 × 2/3 × 1/3 = 0.133, Fail = 0.4 × 1/2 × 1/2 = 0.100, so the answer is still **PASS**.

> **Exam Tip:** if any count is 0, use Laplace smoothing: $\frac{n+1}{N+k}$.

Q4. Discuss the SVM: architecture, margin and core issues. *[7 Marks]*

Ans. An **SVM** finds the **hyperplane** $w \cdot x + b = 0$ that separates the classes with the **maximum margin**. The closest points are the **support vectors**.



Maximum-margin hyperplane with support vectors (circled)

1. **Margin** $= \frac{2}{\| w \|}$, so maximizing the margin means minimizing $\| w \|$.
2. **Hard margin:**

$$\min \tfrac{1}{2} \| w \|^2 \quad \text{s.t.} \quad y_i (w \cdot x_i + b) \geq 1$$

3. **Soft margin:** add slack variables ξ_i and a penalty $C \sum \xi_i$. A large C gives a narrow margin.
4. **Kernel trick:** $K\left(x_i, x_j\right) = \phi (x_i) \cdot \phi \left(x_j \right)$ handles non-linear data. Kernels: linear, polynomial, **RBF**.
5. **Core issues:**
   - Choice of kernel, and tuning **C and** γ.
   - **Slow** on big data ($O\left(n^2\right)$ to $O\left(n^3\right)$).
   - Needs **feature scaling**.
   - Binary only (use OvO / OvR for more classes).
   - No probability output.

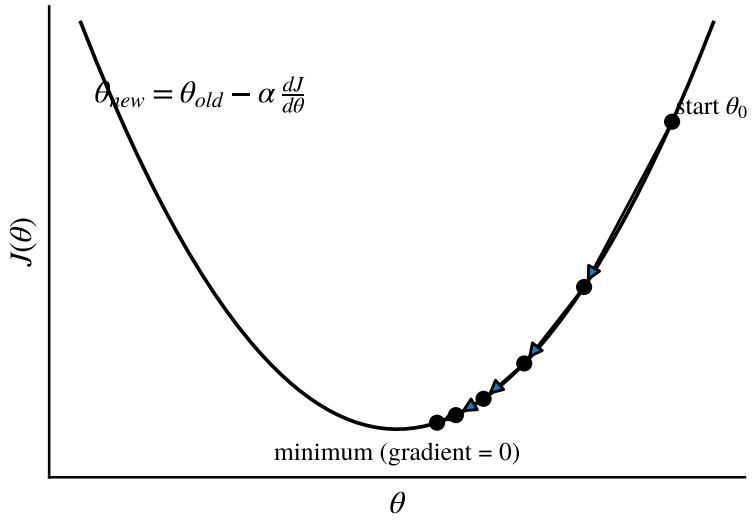
Hard margin	Soft margin
No errors allowed	Errors allowed (slack ξ)
Sensitive to outliers	Robust

Conclusion: used for text classification, face detection and handwriting recognition.

Q5. Explain Gradient Descent. *[7 Marks]*

Ans. Gradient descent is an **iterative** method that minimizes a cost $J (\theta)$ by moving **against the gradient**:

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}, \qquad J (\theta) = \frac{1}{2m} \sum (h_\theta (x_i) - y_i)^2$$



Gradient descent steps down the cost curve towards the minimum; steps get smaller as the slope flattens

1. The gradient points **uphill**, so subtracting it moves θ **downhill**.
2. Near the minimum the gradient is small, so the **steps shrink**. At the minimum the gradient is 0.
3. **Learning rate α:** too small is slow; too large **overshoots or diverges**.
4. **Example:** $J = \theta^2$ with $\theta_0 = 4$ and α = 0.1 gives θ = 4 → 3.2 → 2.56 → 2.05 → … → 0.

Batch GD	Stochastic GD	Mini-batch GD
Uses all data per step	One sample per step	Small batch per step
Stable but slow	Fast but noisy	**Best of both**

Conclusion: gradient descent trains regression models and every neural network (with Adam or SGD).

UNIT 3: DECISION TREES & INSTANCE-BASED LEARNING

Section A: 2-Mark Questions

Q1. Define Entropy; concept of instance-based learning. *[2 Marks]*

Ans.

- **Entropy** is the impurity of a set: $\text{Entropy}(S) = -\sum p_i \log_2 p_i$. It is 0 when the set is pure and 1 for a 50:50 split. Example: [9+, 5−] gives 0.940.
- **Instance-based (lazy) learning stores the training examples** and builds no model. At query time it predicts from the **most similar stored instances**. Example: **k-NN**.

Section B: 7-Mark Questions

Q2. Trace ID3 on the PlayTennis dataset. *[7 Marks]*

Ans. ID3 picks the attribute with the **highest Information Gain** at each node:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_v \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Step 1: with S = [9+, 5−]:

$$\text{Entropy}(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$$

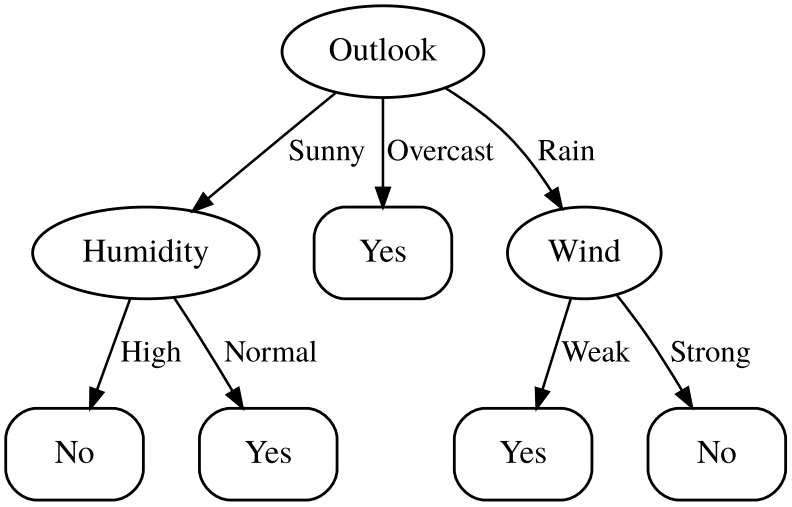
Step 2: Gain at the root

Attribute	Split [+, −]	Gain
Outlook	Sunny [2,3] 0.971, Overcast [4,0] 0, Rain [3,2] 0.971	0.940 − 0.694 = **0.246**
Humidity	High [3,4] 0.985, Normal [6,1] 0.592	0.151
Wind	Weak [6,2] 0.811, Strong [3,3] 1.0	0.048
Temperature	Hot [2,2], Mild [4,2], Cool [3,1]	0.029

Root = Outlook. Overcast is pure, so it becomes the leaf **Yes**.

Step 3: Sunny branch [2+, 3−]. Gain(Humidity) = **0.971** (both subsets pure), Temp = 0.571, Wind = 0.020. So **Humidity** is chosen.

Step 4: Rain branch [3+, 2−]. Gain(Wind) = **0.971**, Humidity = Temp = 0.020. So **Wind** is chosen.



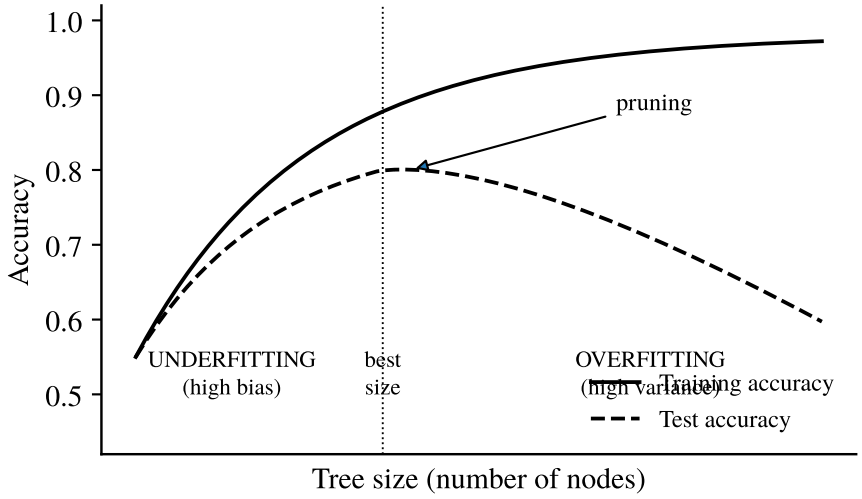
Final decision tree for PlayTennis

Conclusion: Temperature is never used. ID3 gives readable rules.

Q3. Overfitting and underfitting in decision trees; tree pruning. *[7 Marks]*

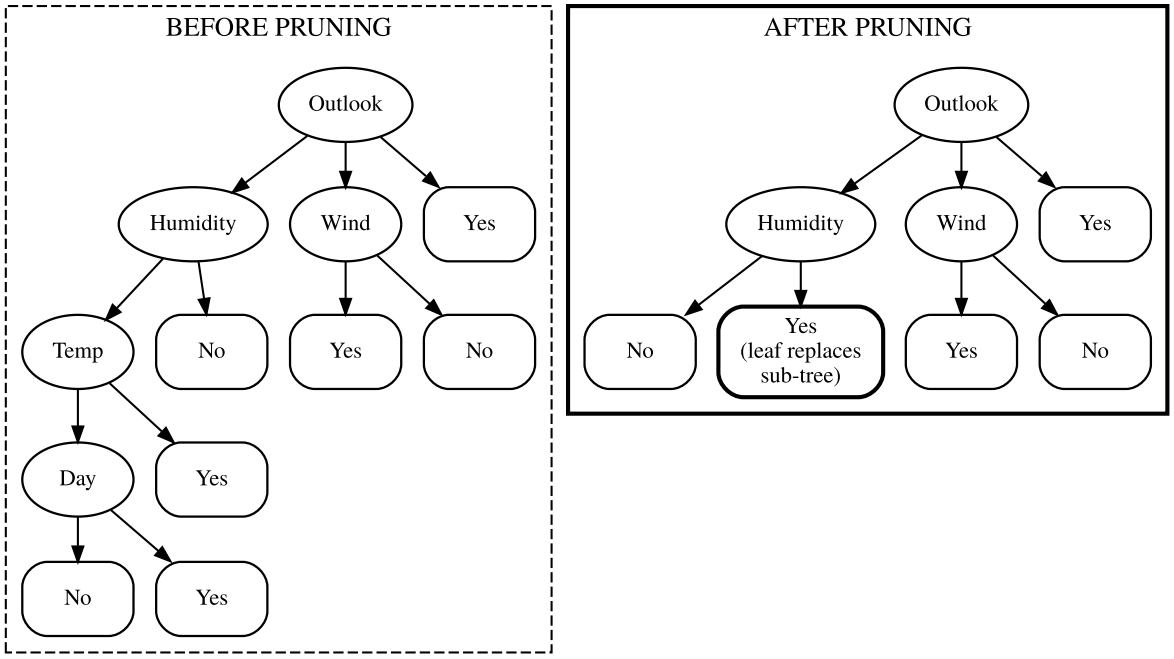
Ans.

- **Overfitting:** the tree is **too deep** and memorizes **noise**. Training accuracy is high but **test accuracy is low**.
- **Underfitting:** the tree is **too shallow**, so accuracy is low on both.
- **Pruning** removes branches that do not help generalization.



Accuracy vs. tree size: training accuracy keeps rising while test accuracy peaks and then falls (overfitting); pruning moves the tree back to the best size

1. **Causes of overfitting:**
   - Noisy data, too few examples.
   - Growing the tree until every leaf is pure.
   - Irrelevant attributes.
2. **Pre-pruning (early stopping):** stop at a max depth, a minimum number of samples, a gain below a threshold, or a chi-square test.
3. **Post-pruning:** grow the full tree, then prune it. This works better.
   - **Reduced Error Pruning:** replace a sub-tree with a leaf if **validation accuracy does not drop**.
   - **Rule post-pruning (C4.5):** convert the tree into rules and drop preconditions.
   - **Cost-complexity pruning (CART):** minimize $R(T) + \alpha|T|$.

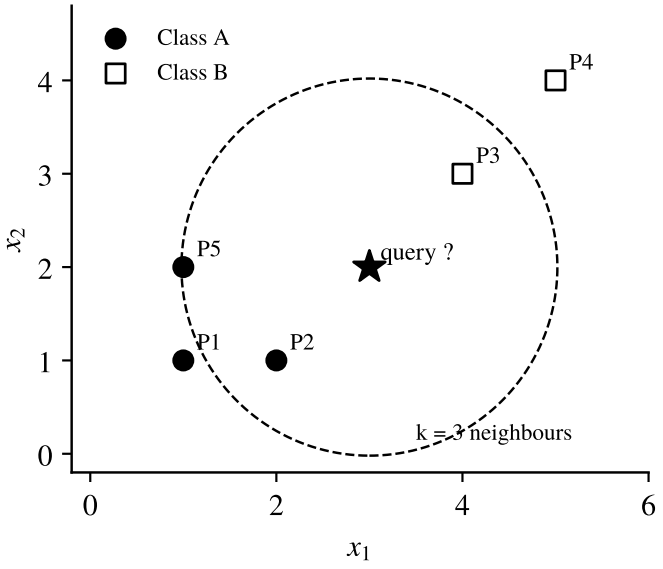


Before and after pruning: the noisy sub-tree under Humidity = Normal is replaced by its majority leaf

Underfitting	Overfitting
High bias	High variance
Low train and test accuracy	High train, low test accuracy
Fix: deeper tree	Fix: **pruning**, more data, Random Forest

Q4. Explain the k-NN algorithm and its limitations. *[7 Marks]*

Ans. k-NN is a **lazy, instance-based** classifier. It finds the **k nearest training points** and takes a **majority vote**.



k-NN with k = 3: the three nearest neighbours of the query (?) are A, A and B, so the majority class is A

1. Store all the training data.
2. Compute the distance to every point: $d = \sqrt{\sum \left(x_i - y_i\right)^2}$ (Euclidean).
3. Pick the k nearest.
4. Take a majority vote (or the mean, for regression).

Example: classify Q(3, 2) with k = 3, using P1(1,1) A, P2(2,1) A, P3(4,3) B, P4(5,4) B, P5(1,2) A.

- Distances: P2 = 1.41, P3 = 1.41, P5 = 2.0, P1 = 2.24, P4 = 2.83.
- The nearest 3 are A, B, A.

$$\text{Class} = A$$

Limitations:

1. **Slow prediction:** $O(n)$ distances per query.
2. Must **store all the data**.
3. **Curse of dimensionality.**
4. Needs **feature scaling**.
5. **Sensitive to noise** and irrelevant features.
6. The choice of k matters (use an odd k, chosen by cross-validation).

Q5. K-Means: cluster (185,72), (170,56), (168,60), (179,68), (182,72), (188,77) with K = 2.

[7 Marks]

Ans. Start with C1 = (185, 72) and C2 = (170, 56). Use $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

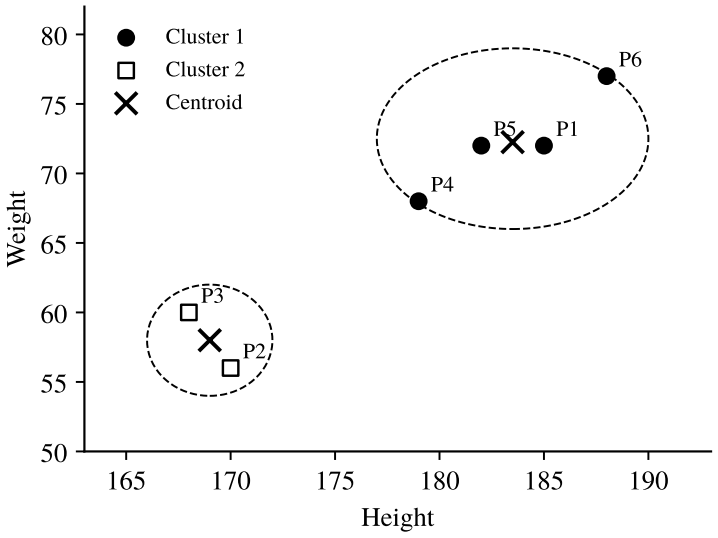
Iteration 1

Point	d to C1	d to C2	Cluster
(168,60)	$\sqrt{17^2 + 12^2} = 20.81$	$\sqrt{2^2 + 4^2} = 4.47$	C2
(179,68)	$\sqrt{6^2 + 4^2} = 7.21$	$\sqrt{9^2 + 12^2} = 15.0$	C1
(182,72)	3.00	20.00	C1
(188,77)	5.83	27.66	C1

New centroids:

$$C_1 = \left(\frac{185 + 179 + 182 + 188}{4}, \frac{72 + 68 + 72 + 77}{4}\right) = (183.5, 72.25), \qquad C_2 = (169, 58)$$

Iteration 2: every point keeps its cluster, so the algorithm has **converged**.



Final clusters after convergence (x = centroid)

$$C_1 = \{P1, P4, P5, P6\} \text{ at } (183.5, 72.25), \qquad C_2 = \{P2, P3\} \text{ at } (169, 58)$$

Q6. Compare Information Gain, Gain Ratio and Gini Index.

[7 Marks]

Ans. These are **attribute selection measures** for decision trees.

$$\text{GainRatio} = \frac{\text{Gain}}{\text{SplitInfo}}, \quad \text{SplitInfo} = -\sum \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}, \quad \text{Gini} = 1 - \sum p_i^2$$

Worked on Outlook (PlayTennis):

- Gain = 0.246.
- SplitInfo = 1.577, so GainRatio = 0.246 / 1.577 = **0.156**.
- Gini(S) = 0.459 and Gini_Outlook = 0.343, so the Gini gain = **0.116**.

Basis	Information Gain	Gain Ratio	Gini Index
Algorithm	**ID3**	**C4.5**	**CART**
Choose	Maximum	Maximum	**Minimum**
Bias	Many-valued attributes	Unbalanced splits	Larger partitions
Logarithm used?	Yes	Yes	**No (fastest)**
Split type	Multi-way	Multi-way	**Binary**

UNIT 4: ARTIFICIAL NEURAL NETWORKS

Section A: 2-Mark Questions

Q1. What is backpropagation? Inductive bias in ANN? *[2 Marks]*

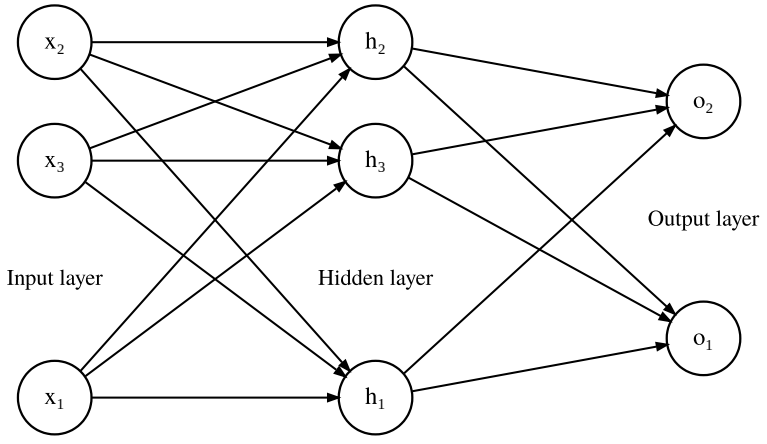
Ans.

- **Backpropagation** trains multilayer networks. A **forward pass** computes the outputs, then a **backward pass** sends the error δ from the output back to the hidden layers using the **chain rule**. Weights are updated by $\Delta w = \eta\, \delta\, x$.
- **Inductive bias of ANNs: smooth interpolation between data points**. The network architecture also adds bias (CNNs assume spatial locality).

Section B: 7-Mark Questions

Q2. Derive the backpropagation algorithm. *[7 Marks]*

Ans. Minimize $E = \frac{1}{2} \sum_k \left(t_k - o_k\right)^2$ by gradient descent, $\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}$, with sigmoid units $o = \sigma\left(net\right)$.



Forward pass computes the outputs; the backward pass sends the error terms δ from the output layer back to the hidden layer

Step 1: Chain rule.

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial net_j} \cdot x_{ji}$$

Step 2: Sigmoid derivative.

$$\frac{\partial o_j}{\partial net_j} = o_j\left(1 - o_j\right)$$

Step 3: Output unit.

$$\frac{\partial E}{\partial net_j} = \frac{\partial E}{\partial o_j} \cdot \frac{\partial o_j}{\partial net_j} = -\left(t_j - o_j\right) o_j\left(1 - o_j\right)$$

$$\delta_k = o_k\left(1 - o_k\right)\left(t_k - o_k\right)$$

Step 4: Hidden unit. It affects E through its downstream units k:

$$\frac{\partial E}{\partial net_h} = \sum_k \frac{\partial E}{\partial net_k} \cdot \frac{\partial net_k}{\partial o_h} \cdot \frac{\partial o_h}{\partial net_h} = \sum_k \left(-\delta_k\right) w_{kh}\, o_h\left(1 - o_h\right)$$

$$\delta_h = o_h\left(1 - o_h\right) \sum_k w_{kh}\, \delta_k$$

Step 5: Update.

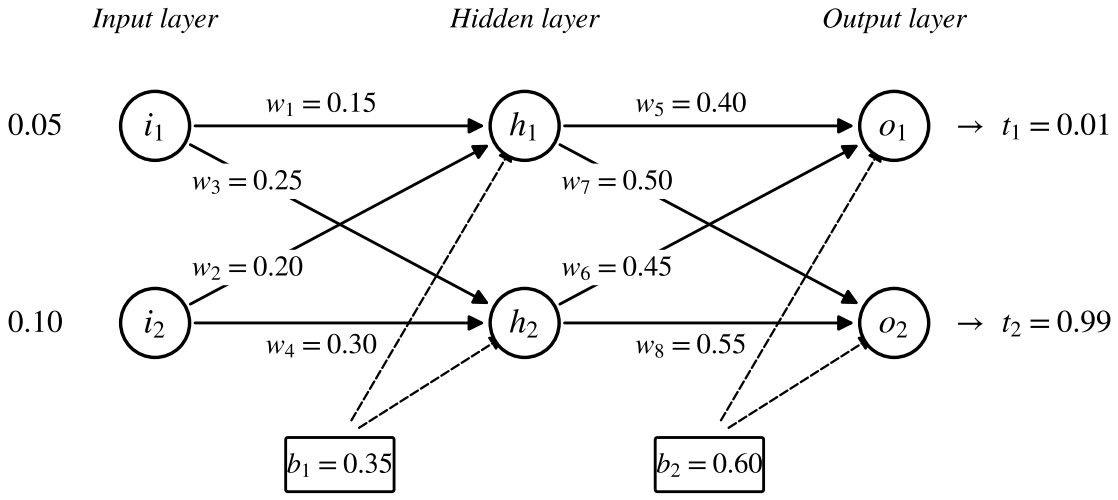
$$w_{ji} \leftarrow w_{ji} + \eta\, \delta_j\, x_{ji}$$

Algorithm: initialize small random weights; then repeat for each example: forward pass, compute δ for the outputs, compute δ for the hidden units, update all weights.

> **Remember:** the hidden δ is the **weighted sum of the downstream δs**. That is what "back-propagation" means.

Q3. Numerical: one forward and backward pass with η = 0.5. *[7 Marks]*

Ans.



Network with initial weights, biases and targets

Forward pass:

- $net_{h_1} = 0.15\,(0.05) + 0.20\,(0.10) + 0.35 = 0.3775$, so $out_{h_1} = \sigma\,(0.3775) = 0.5933$, and $out_{h_2} = 0.5969$.
- $net_{o_1} = 0.40\,(0.5933) + 0.45\,(0.5969) + 0.60 = 1.1059$, so $out_{o_1} = 0.7514$, and $out_{o_2} = 0.7729$.

$$E = \tfrac{1}{2}\,(0.01 - 0.7514)^2 + \tfrac{1}{2}\,(0.99 - 0.7729)^2 = 0.2984$$

Backward pass (output layer):

$$\delta_{o_1} = (0.01 - 0.7514)(0.7514)(0.2486) = -0.1385, \qquad \delta_{o_2} = +0.0381$$

- $w_5 = 0.40 + 0.5(-0.1385)(0.5933) = 0.3589$, $w_6 = 0.4087$, $w_7 = 0.5113$, $w_8 = 0.5614$.

Backward pass (hidden layer, using the OLD w5 to w8):

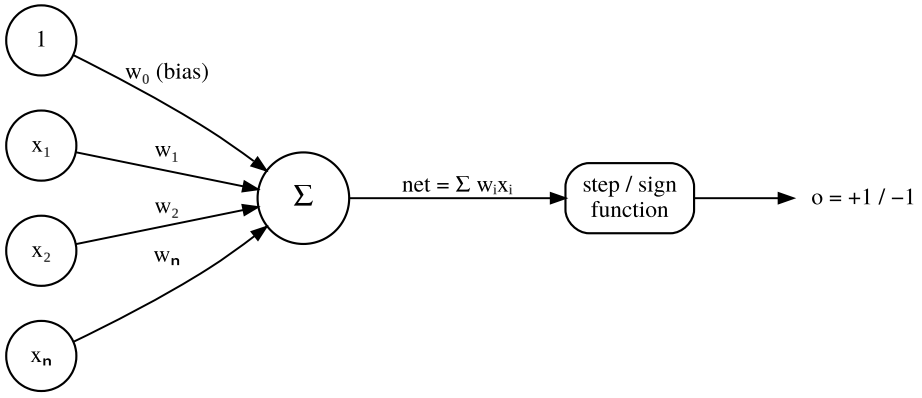
$$\delta_{h_1} = 0.2413\,[0.40(-0.1385) + 0.50(0.0381)] = -0.00877, \qquad \delta_{h_2} = -0.00995$$

- $w_1 = 0.15 + 0.5(-0.00877)(0.05) = 0.14978$, $w_2 = 0.19956$, $w_3 = 0.24975$, $w_4 = 0.29950$.

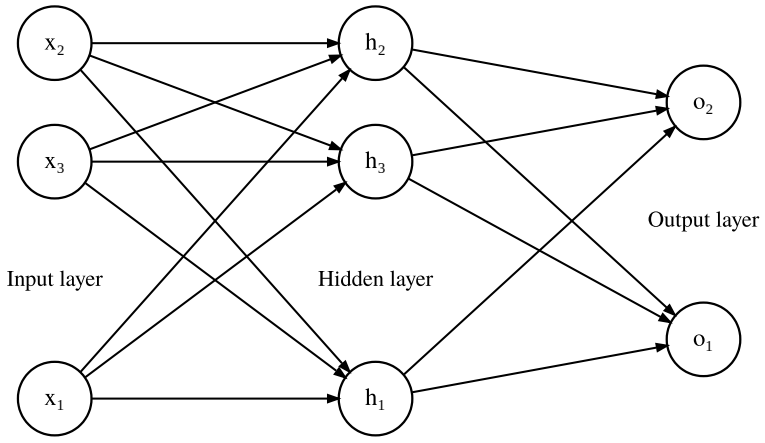
w	w1	w2	w3	w4	w5	w6	w7	w8
New	0.14978	0.19956	0.24975	0.29950	0.3589	0.4087	0.5113	0.5614

Q4. Single-layer perceptron vs. multilayer perceptron. *[7 Marks]*

Ans. An **SLP** has one layer of weights and a **step activation**, so it learns only **linearly separable** functions. An **MLP** has **hidden layers** with non-linear activations and is trained by **backpropagation**.

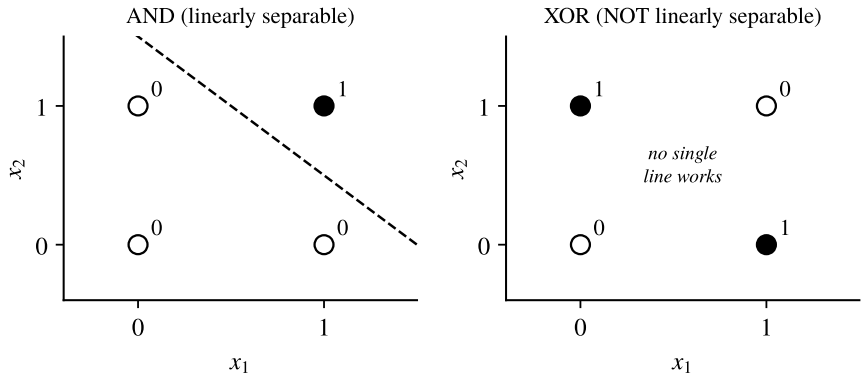


Single-layer perceptron: weighted sum followed by a step function



Multilayer perceptron with one hidden layer (sigmoid / ReLU units), trained by backpropagation

- **Perceptron rule:** $w_i \leftarrow w_i + \eta\,(t - o)\,x_i$. It converges if the data is linearly separable.
- **An SLP cannot learn XOR** (Minsky and Papert, 1969). An MLP can.



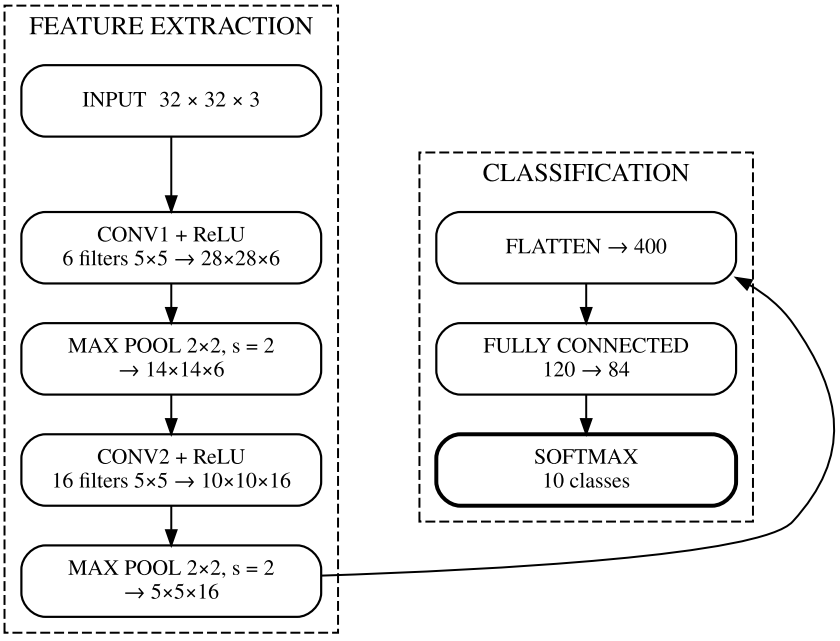
AND is linearly separable (one line separates the classes); XOR is not, so a single-layer perceptron cannot learn it

Basis	SLP	MLP
Hidden layer	None	One or more
Activation	Step	Sigmoid / ReLU
Learning	Perceptron rule	Backpropagation
Boundary	Linear	Non-linear
XOR	**No**	**Yes**

Q5. Explain the CNN layered architecture.

[7 Marks]

Ans. A **CNN** learns **spatial features** from images using **convolution, pooling and fully connected** layers.



CNN architecture (LeNet-style): feature extraction (left), then classification (right)

1. **Convolution:** a filter slides over the input and takes **dot products**, producing **feature maps**. It uses **weight sharing** and local connectivity. Output size:

$$\frac{W - F + 2P}{S} + 1 \qquad \text{for example } \frac{32 - 5}{1} + 1 = 28$$

2. **ReLU:** $\max(0, x)$ adds non-linearity.
3. **Pooling:** down-samples, has **no learnable weights**, and gives translation invariance. Example:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 9 & 1 \\ 3 & 4 & 0 & 8 \end{bmatrix} \xrightarrow{\max\ 2\times 2} \begin{bmatrix} 6 & 4 \\ 7 & 9 \end{bmatrix}$$

4. **Flatten → Fully connected → Softmax:** $P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$.

Convolution	Pooling	Fully Connected
Extracts features	Down-samples	Classifies
Has learnable weights	No weights	Most of the weights

Conclusion: used for face recognition, medical imaging and self-driving cars.

UNIT 5: REINFORCEMENT LEARNING & GENETIC ALGORITHMS

Section A: 2-Mark Questions

Q1. What is Q-learning? Crowding in GA? *[2 Marks]*

Ans.

- **Q-learning** is **model-free, off-policy** RL. It learns $Q(s, a)$ with:

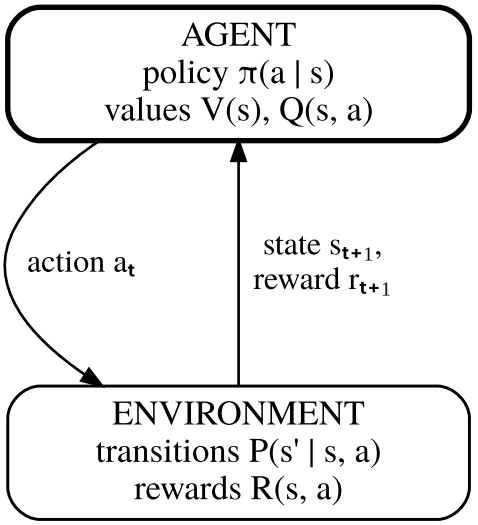
$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s,a)]$$

- **Crowding:** a very fit individual and its copies **take over the population**. Diversity is lost and the GA **converges prematurely**. Fixes: **fitness sharing**, crowding replacement, tournament selection.

Section B: 7-Mark Questions

Q2. Explain RL through the Markov Decision Process. *[7 Marks]*

Ans. RL is modelled as an **MDP (S, A, P, R, γ)**. An agent chooses actions to **maximize the discounted return** $G_t = \sum_k \gamma^k r_{t+k+1}$.

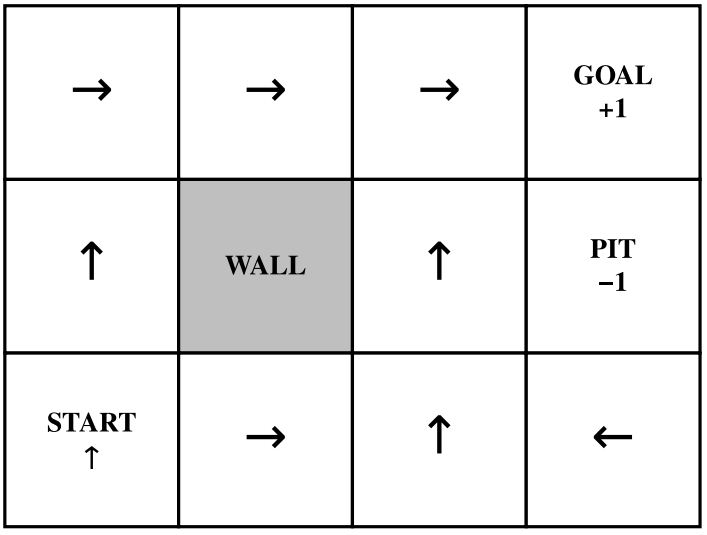


Agent-environment interaction in an MDP

1. **S, A:** the states and actions.
2. **P(s' | s, a):** the transition probability.
3. **R:** the reward.
4. **γ:** the discount factor.
5. **Markov property:** the next state depends **only on the current state and action**.
6. **Policy π(s):** maps states to actions. **Value:** $V^{\pi}(s) = E\left[\sum \gamma^k r\right]$.
7. **Bellman optimality equation:**

$$V^{*}(s) = \max_{a} \sum_{s'} P\left(s'|s,a\right)\left[R + \gamma V^{*}\left(s'\right)\right]$$

8. **Solving it: value iteration or policy iteration** (when the model is known), **Q-learning or SARSA** (when it is not). Use **ε-greedy** exploration.

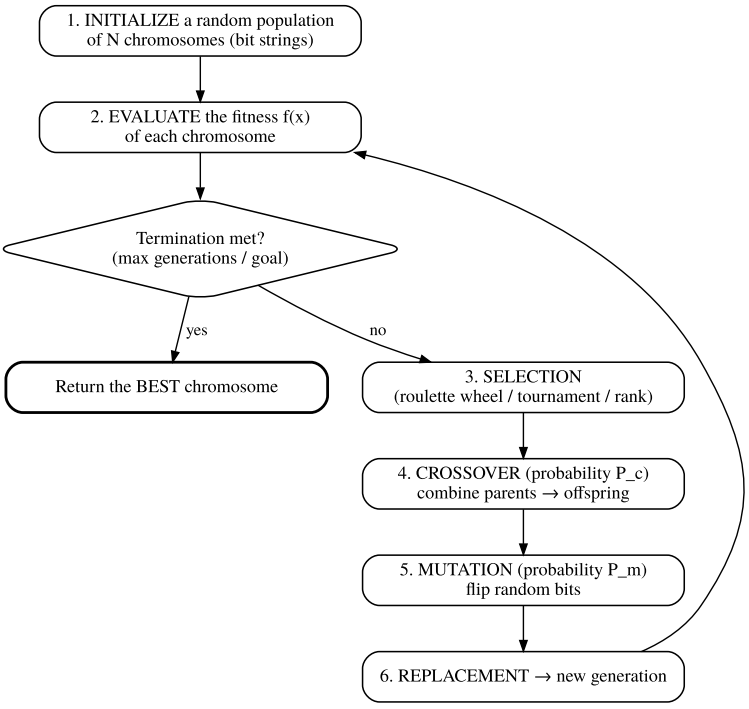


Grid-world MDP ($\gamma = 0.9$; +1 at GOAL, −1 at PIT, −0.04 per step; 0.8 chance of the intended move). Arrows show the optimal policy π^*

Conclusion: used in AlphaGo, robotics and RLHF.

Q3. Trace the Genetic Algorithm cycle (fitness, crossover, mutation). *[7 Marks]*

Ans. A **GA** evolves a population of chromosomes through **selection, crossover and mutation**.



Genetic Algorithm cycle

Trace: maximize $f(x) = x^2$ for x in 0 to 31 (5 bits).

Chromosome	x	f(x)	P(select)
01101	13	169	0.14
11000	24	576	**0.49**
01000	8	64	0.06
10011	19	361	0.31

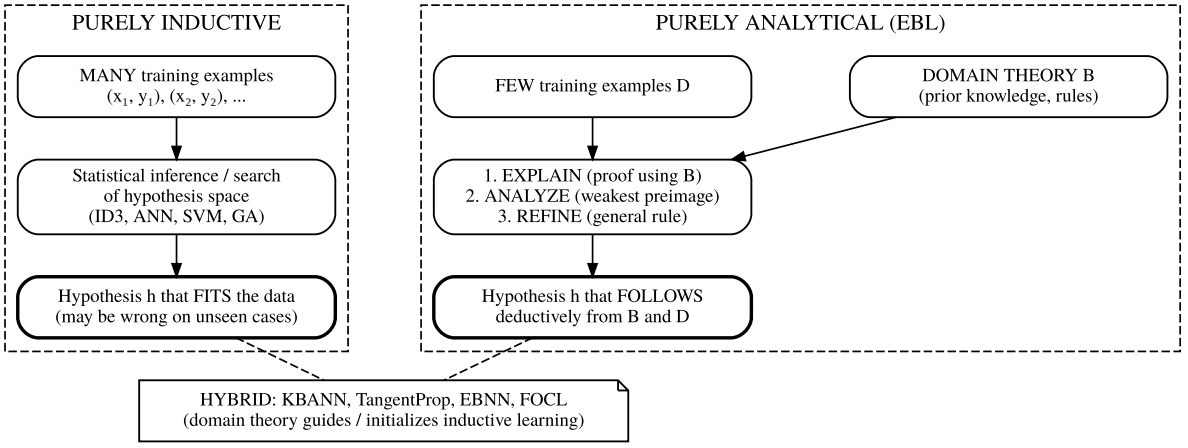
- Sum = 1170, average = 293, max = 576.
- **Crossover:**
  - 0110|1 × 1100|0 gives 01100 (144) and 11001 (625).
  - 11|000 × 10|011 gives **11011 (729)** and 10000 (256).
- After one generation: **average 293 → 439, max 576 → 729**.
- **Mutation:** flip a random bit with small probability P_m (for example 10000 → 10100). This keeps diversity.

Roles:

- **Fitness:** measures quality and guides selection.
- **Crossover: exploitation** (combines good building blocks).
- **Mutation: exploration** (prevents premature convergence).

Q4. Compare purely analytical and purely inductive learning. *[7 Marks]*

Ans. Inductive learning generalizes from **many examples**, with no prior knowledge. **Analytical learning (EBL)** uses a **domain theory plus a few examples** and **explains** them deductively.



Purely inductive vs. purely analytical learning

Basis	Inductive	Analytical
Input	Many examples	Domain theory + few examples
Reasoning	**Induction** (statistical)	**Deduction** (logical proof)
Prior knowledge	Not needed	**Essential**
Risk	Too little data gives wrong hypotheses	**Wrong domain theory** gives wrong hypotheses
Examples	ID3, ANN, SVM	**Prolog-EBG**

EBL steps:

1. **Explain** the example using the theory.
2. **Analyze** it (weakest preimage).
3. **Refine** it into a general rule.

Hybrid methods: KBANN (builds a network from rules, then trains it).

Data Analytics

BCS-052 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** theory answers are short and diagram-based. The **numericals (PCA, regression, Flajolet-Martin, windows, Apriori, K-Means)** are solved inside their own units, in the format: formula, then substitution, then boxed answer.

UNIT 1: DATA ANALYTICS LIFECYCLE & BIG DATA

Section A: 2-Mark Questions

Q1. Structured, semi-structured and unstructured data. *[2 Marks]*

Ans.

Structured	Semi-structured	Unstructured
Fixed schema (rows and columns)	Tags / keys, flexible schema	No format
RDBMS: bank records	XML, JSON, email	Images, videos, social posts (**about 80%** of all data)

Q2. The V's of Big Data; analysis vs. reporting. *[2 Marks]*

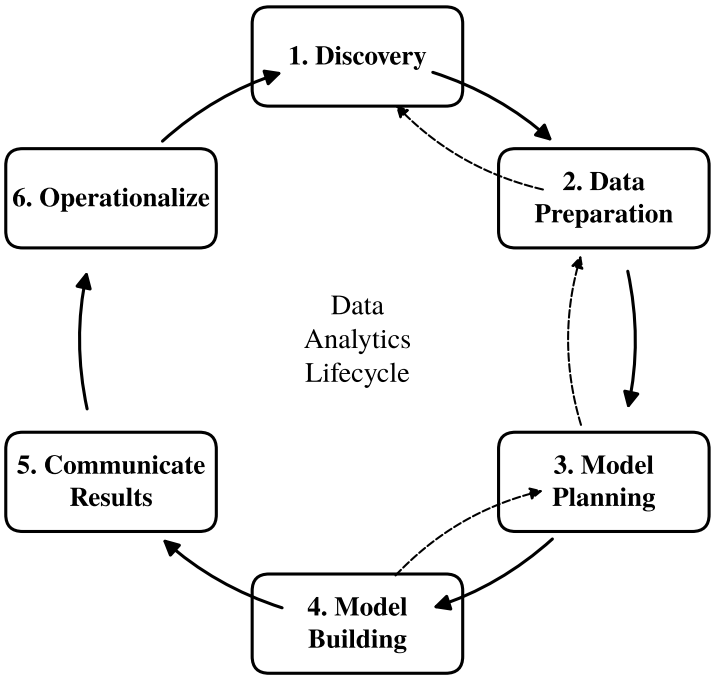
Ans.

- **V's: Volume** (size), **Velocity** (speed), **Variety** (types), **Veracity** (trustworthiness), **Value** (useful insight).
- **Reporting** shows **what happened** (organised summaries, dashboards). **Analysis** explains **why it happened and what will happen** (insights, predictions).

Section B: 7-Mark Questions

Q3. Explain the 6 phases of the Data Analytics Lifecycle with a diagram. *[7 Marks]*

Ans. The **Data Analytics Lifecycle** is an **iterative six-phase process** for data science projects. The team can **go back** to an earlier phase whenever needed.



Data Analytics Lifecycle: solid arrows show the normal flow, dashed arrows show going back to an earlier phase

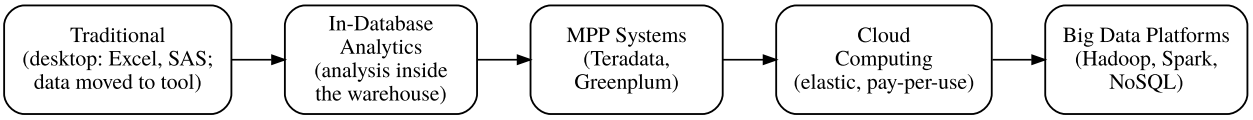
1. **Discovery:**
   - Understand the business problem, the stakeholders and the resources.
   - Form **initial hypotheses**.
   - *Exit question:* is there enough information to draft a plan?
2. **Data preparation:**
   - Set up an **analytic sandbox** and run **ETLT**.
   - Clean the data: missing values, outliers.
   - Usually **50 to 80% of the project time**.
   - Tools: Hadoop, OpenRefine.
3. **Model planning:**
   - Choose techniques and variables with **EDA**.
   - Tools: R, SQL, SAS.
4. **Model building:**
   - Split into train and test sets, build and tune the models.
   - Check that the model is robust.
   - Tools: R, Python, Spark MLlib.
5. **Communicate results:** compare with the success criteria and present the findings with visuals.
6. **Operationalize:** run a pilot, **deploy**, monitor and retrain. Deliver the reports and code.

Phase	Output
Discovery	Problem statement + hypotheses
Data preparation	Clean dataset
Model planning	Model plan
Model building	Working model
Communicate results	Findings
Operationalize	Deployed system

Conclusion: used in churn prediction, fraud detection and demand forecasting projects.

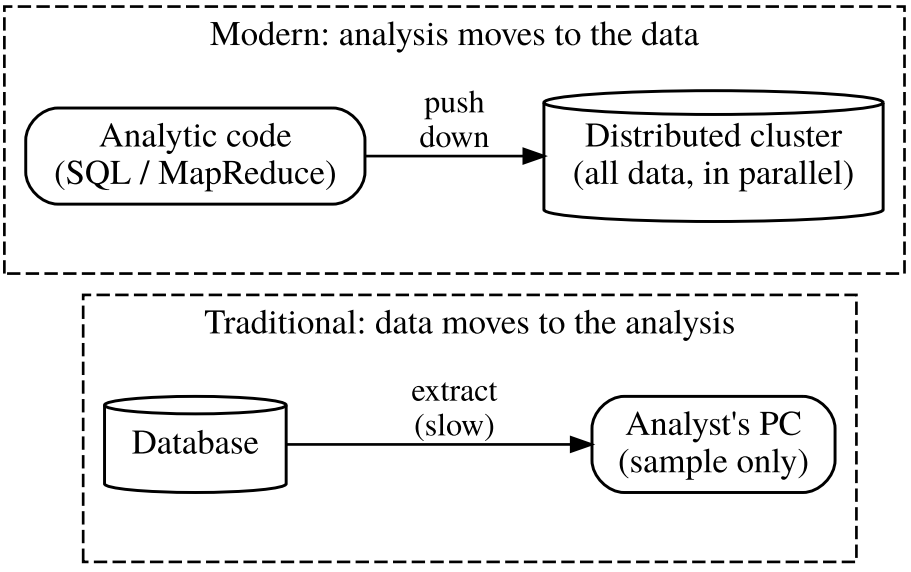
Q4. Explain the evolution of analytic scalability and the need for Big Data platforms. *[7 Marks]*

Ans. Analytic scalability is the ability to handle **growing data** by adding resources. Analytics has moved from desktop tools to **distributed platforms**.



Evolution of analytic scalability

1. **Traditional:** data is **extracted to the analyst's PC** (SAS, Excel). This is slow and only **samples** can be analysed.
2. **In-database analytics:** the analysis runs **inside the warehouse**, so there is no data movement.
3. **MPP** (Teradata, Greenplum): data is partitioned across nodes and processed **in parallel**.
4. **Cloud: elastic, pay-per-use** resources (AWS EMR, BigQuery).
5. **Big Data platforms** (Hadoop, Spark): **distributed storage and processing on commodity hardware**, with data locality.



Old approach vs. modern approach: move the analysis to the data

Need for Big Data platforms:

- **Volume:** petabytes.
- **Variety:** unstructured data.
- **Velocity:** real-time streams.
- **Cost:** cheap horizontal scaling.
- **Fault tolerance:** replication.
- **ML on the full data**, not just samples.

Traditional	Big Data platform
Vertical scaling	**Horizontal** scaling
Structured data only	All types
Schema-on-write	**Schema-on-read**

UNIT 2: ADVANCED DATA ANALYSIS TECHNIQUES

Section A: 2-Mark Questions

Q1. What is PCA? Define a Bayesian Network. Core idea of Fuzzy Logic. *[2 Marks]*

Ans.

- **PCA:** reduces dimensions by converting correlated variables into **uncorrelated principal components** (the eigenvectors of the covariance matrix), ordered by variance (eigenvalue).
- **Bayesian network:** a **DAG** whose nodes are random variables, with **conditional probability tables**: $P(X_1..X_n) = \prod P(X_i \mid \text{Parents}(X_i))$.
- **Fuzzy logic:** truth is a **degree from 0 to 1** (membership value μ), not just true or false. Example: 30°C is "Hot" with μ = 0.7.

Section B: 7-Mark Questions

Q2. Explain regression modelling and multivariate analysis. *[7 Marks]*

Ans. Regression estimates how a **dependent variable Y depends on predictors X**, so that Y can be predicted. **Multivariate analysis** studies **many variables together**.



Types of regression and multivariate techniques

1. **Simple linear regression:** $\widehat{Y} = b_0 + b_1X$, with $b_1 = \dfrac{\sum (X - \bar{X})(Y - \bar{Y})}{\sum (X - \bar{X})^2}$ and $b_0 = \bar{Y} - b_1\bar{X}$.
2. **Multiple regression:** $\widehat{Y} = b_0 + b_1X_1 + \ldots + b_nX_n$, with $b = (X^TX)^{-1}X^TY$.

3. **Logistic regression:** $P(Y=1) = \frac{1}{1+e^{-(b_0+b_1X)}}$, for a binary Y.
4. **Assumptions:** linearity, independent errors, homoscedasticity, normal residuals, no multicollinearity.
5. **Fit:** $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$, plus RMSE and p-values.
6. **Multivariate techniques:** multiple regression, **PCA / factor analysis**, **cluster analysis**, discriminant analysis, **MANOVA**.

Univariate	Bivariate	Multivariate
1 variable	2 variables	3 or more variables
Mean, histogram	Correlation	Regression, PCA

Q3. Numerical: PCA. (a) Eigenvalues 4.5, 2.8, 1.2, 0.4, 0.1. (b) 2-D points. *[7 Marks]*

Ans. (a) $\sum \lambda = 9.0$, and the variance of each component is $\frac{\lambda_k}{9.0} \times 100$.

PC	1	2	3	4	5
% variance	50.0	31.1	13.3	4.4	1.1
Cumulative %	50.0	81.1	**94.4**	98.9	100

3 components are needed for $\geq 90\%$ variance

(b) Points (2,1), (3,5), (4,3), (5,6), (6,7), (7,8)

1. **Mean:** $\bar{x} = 4.5$, $\bar{y} = 5$.
2. **Sums of deviations:** $\sum (x-\bar{x})^2 = 17.5$, $\sum (y-\bar{y})^2 = 34$, $\sum (x-\bar{x})(y-\bar{y}) = 22$.
3. **Covariance matrix** (dividing by n = 6):

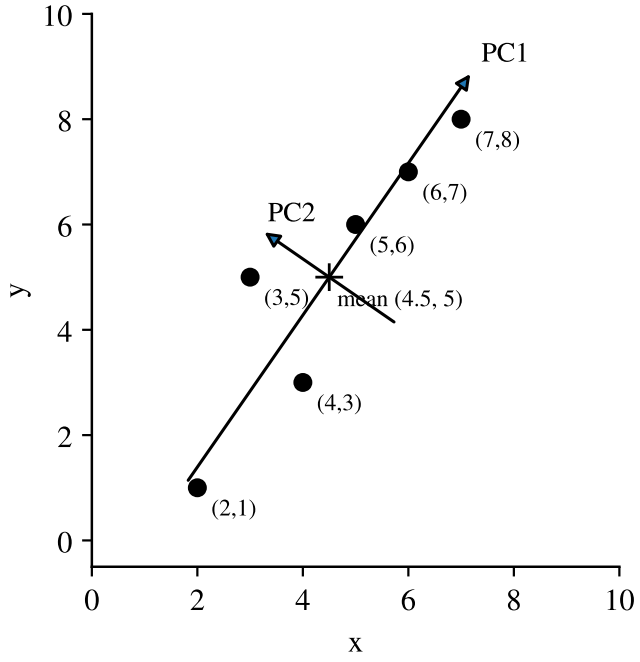
$$\text{Cov} = \begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix}$$

4. **Eigenvalues:**

$$\lambda^2 - 8.58\lambda + 3.08 = 0 \Rightarrow \lambda_1 = 8.21,\ \lambda_2 = 0.38$$

5. **Eigenvector for λ_1:** $(2.92 - 8.21)e_1 + 3.67e_2 = 0$, so $e_2 = 1.44e_1$.

$$PC_1 = (0.57,\ 0.82) \text{ explains } \frac{8.21}{8.59} = 95.6\% \text{ of the variance}$$



Data points with the first principal component direction

Q4. Numerical: regression (hours vs. scores) and standard deviation. *[7 Marks]*

Ans. (a) X = 2, 4, 6, 8, 10 and Y = 50, 55, 65, 70, 78.

- $\bar{X} = 6$, $\bar{Y} = 63.6$.
- $\sum \left(X - \bar{X}\right)\left(Y - \bar{Y}\right) = 54.4 + 17.2 + 0 + 12.8 + 57.6 = 142$ and $\sum \left(X - \bar{X}\right)^2 = 40$.

$$b_1 = \frac{142}{40} = 3.55, \qquad b_0 = 63.6 - 3.55\,(6) = 42.3$$

$$\widehat{Y} = 42.3 + 3.55X, \qquad \widehat{Y}(7) = 42.3 + 24.85 = 67.15$$

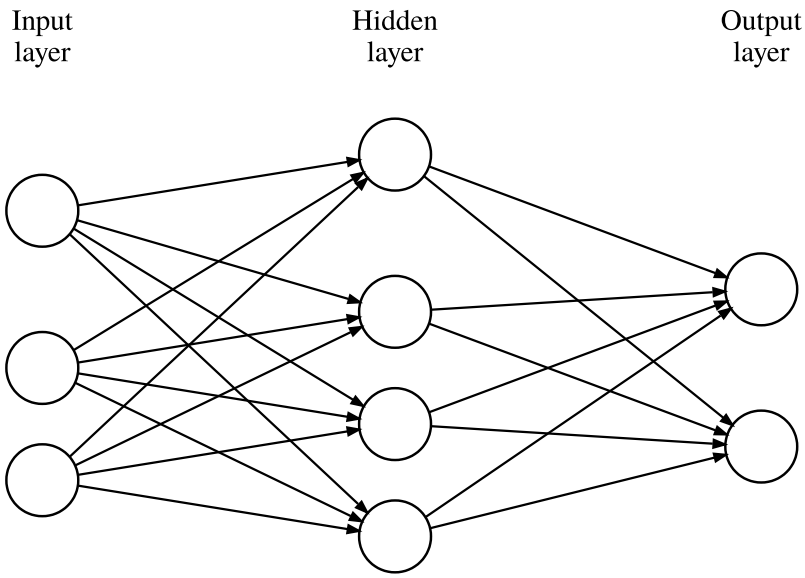
(b) Data {12, 15, 20, 22, 25, 30}:

$$\bar{x} = \frac{124}{6} = 20.67, \quad \sum (x - \bar{x})^2 = 215.33, \quad \sigma = \sqrt{\frac{215.33}{6}} = 5.99 \ (\text{sample } s = 6.56)$$

- **Comment:** CV = 5.99 / 20.67 = **29%**, so the data has a **moderate spread**. It is fairly symmetric (the median 21 is close to the mean) and has no outliers.

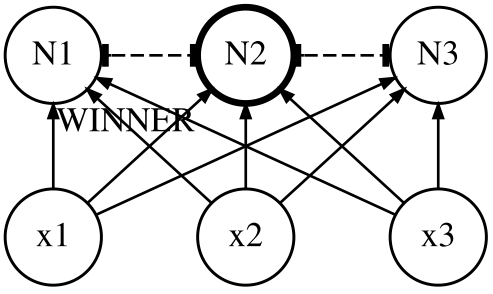
Q5. Neural networks: competitive learning and generalization. *[7 Marks]*

Ans. An **ANN** has **input, hidden and output layers** of neurons, each computing $y = f\left(\sum w_i x_i + b\right)$.



Feed-forward neural network architecture

1. **Architectures:** feed-forward (MLP), recurrent (RNN), competitive (SOM).
2. **Competitive learning (unsupervised):**
   - Output neurons **compete**. The **winner** is the one closest to the input: $j^* = \arg\min_j \| x - w_j \|$.
   - **Only the winner learns:** $w_{j^*} \leftarrow w_{j^*} + \eta \left(x - w_{j^*} \right)$.
   - The result is **cluster prototypes**. In a **Kohonen SOM**, the neighbours learn too.
3. **Generalization:** the network performs well on **unseen data**.
   - **Overfitting** is the main threat.
   - Improve it with **cross-validation, early stopping, regularization, dropout and more data**.

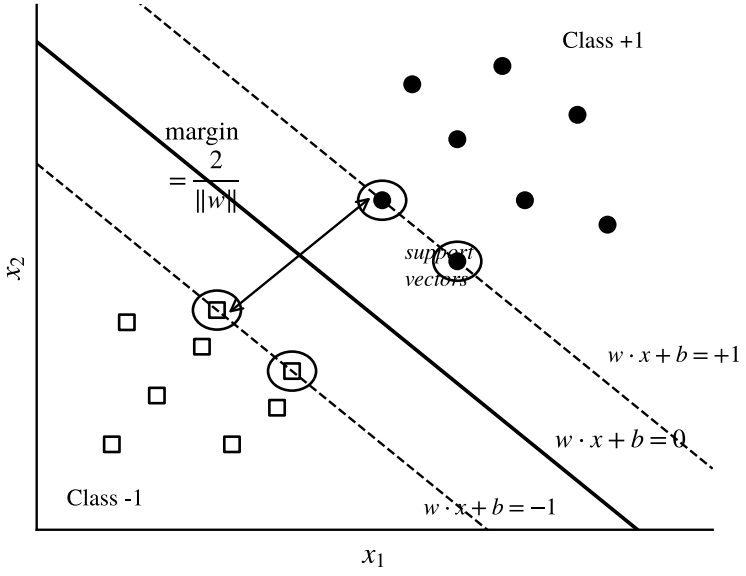


Competitive (winner-take-all) learning layer

Competitive learning	Backpropagation
Unsupervised	Supervised
Only the winner is updated	All weights are updated
Produces clusters	Produces an input-output mapping

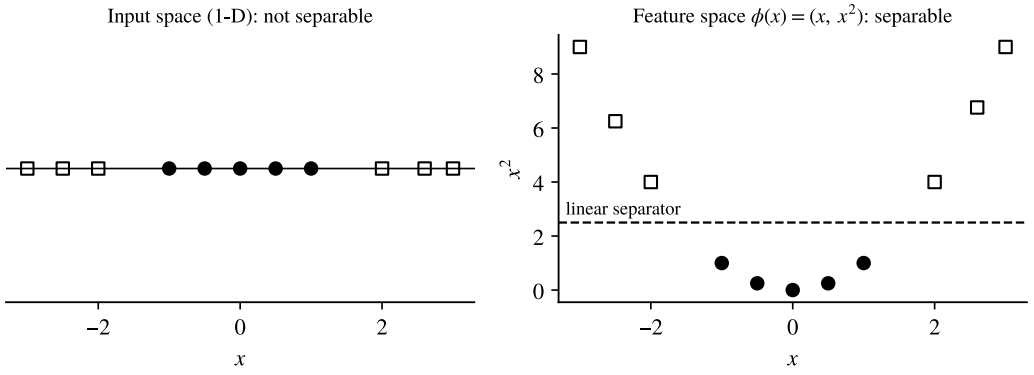
Q6. Explain SVM and kernel methods. *[7 Marks]*

Ans. An **SVM** finds the **maximum-margin hyperplane** $w \cdot x + b = 0$. **Kernels** handle non-linear data by implicitly mapping it to a higher dimension.



Maximum-margin hyperplane with support vectors

1. **Margin** $= 2/\|w\|$. The optimization is $\min \frac{1}{2}\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$.
2. **Support vectors:** the points on the margin. Only they decide the hyperplane.
3. **Soft margin:** slack variables ξ with penalty C.
4. **Kernel trick:** $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$.
    - Kernels: linear, polynomial $(x \cdot y + c)^d$, **RBF** $e^{-\gamma\|x-y\|^2}$.



Kernel trick: data not separable in 1-D becomes separable after mapping x to (x, x²)

Conclusion: used for text, image and gene classification.

UNIT 3: MINING DATA STREAMS

Section A: 2-Mark Questions

Q1. What is a data stream? Decaying vs. sliding window. *[2 Marks]*

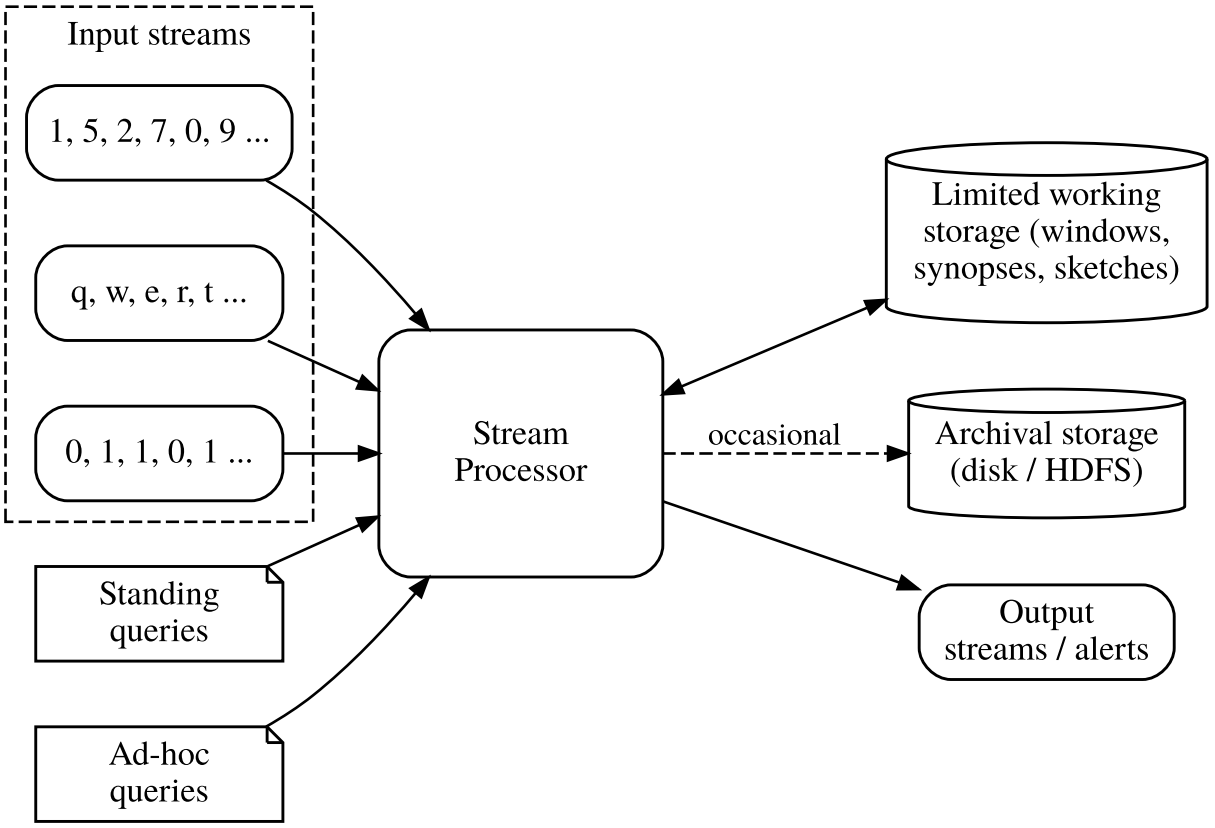
Ans. A **data stream** is a **continuous, unbounded, fast, time-ordered** sequence of data. It is read **once**, with **limited memory**. Examples: sensors, tweets, stock ticks.

Sliding window	Decaying window
Only the **last N** elements count	**All** elements count, with weight $(1-c)^{\text{age}}$
Old data is dropped completely	Old data **fades** gradually
Stores N elements	Stores **one running sum**: $S = S(1-c) + a_t$

Section B: 7-Mark Questions

Q2. Explain the architecture of a stream computing model. *[7 Marks]*

Ans. Stream computing processes **data in motion** as it arrives, using a **DSMS (Data Stream Management System)**.



Architecture of a data stream management system

1. **Input streams:** many sources, each with its own rate.
2. **Stream processor:** reads each element **once** and applies the queries.

3. **Working storage (limited memory):** holds **windows, samples and sketches** (Bloom filter, FM sketch).
4. **Archival storage:** large and slow. It is **not used for real-time answers**.
5. **Standing queries** (continuous, for example "alert if temperature > 50") and **ad-hoc queries** (one-time).
6. **Output streams:** alerts and dashboards.

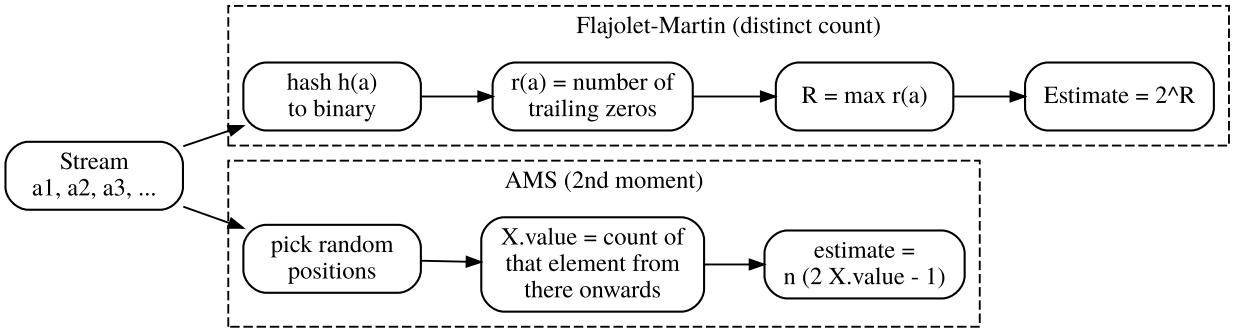
- **Key algorithms:** sampling, Bloom filter, **Flajolet-Martin**, **AMS**, DGIM.
- **Tools:** Kafka, Storm, Flink, Spark Streaming.

DBMS	DSMS
Finite, stored data	**Infinite** stream
Many passes	**One pass**
One-time queries	**Continuous** queries
Exact answers	**Approximate** answers

Q3. Estimating moments and counting distinct elements (Flajolet-Martin and AMS). *[7 Marks]*

Ans. The **k-th moment** of a stream is $F_k = \sum_i m_i^k$:

- F_0 = the number of **distinct** elements.
- F_1 = the stream length.
- F_2 = the "surprise number".



Flajolet-Martin and AMS: one pass, small memory

Flajolet-Martin (F_0):

1. Hash each element to binary.
2. r = the number of **trailing zeros**.
3. R = the maximum r.
4. **Estimate =** 2^R**.**

Why it works: the probability of seeing r trailing zeros is 2^{-r}. Use many hash functions and take the **median of the averages**.

AMS (F_2): pick random positions. X.value = the count of that element from that position onwards. Then:

$$F_2 \approx n\,(2X.\text{value} - 1)$$

- Example: stream a b c b d a c d a b d c a a b (n = 15). The true $F_2 = 25 + 16 + 9 + 9 = 59$.
- X at position 3 (element c) counts 3 occurrences, so the estimate is $15\,(2 \times 3 - 1) = 75$. Averaging many such estimates gets close to 59.

Q4. Numerical: Flajolet-Martin, decaying window, sliding window. *[7 Marks]*

Ans. (a) S = {1, 3, 2, 1, 2, 3, 4, 3, 1, 2, 3, 1}, with $h(x) = (6x + 1) \mod 5$.

x	1	2	3	4
h(x)	7 mod 5 = 2	13 mod 5 = 3	19 mod 5 = 4	25 mod 5 = 0
Binary	010	011	**100**	000
Trailing zeros r	1	0	**2**	0 (taken as 0)

$$R = 2 \Rightarrow \text{distinct} \approx 2^2 = 4 \quad (\text{actual} = \{1, 2, 3, 4\} = 4\ \checkmark)$$

(b) Decaying window:

$$120,000 \times 0.35 = 42,000 \text{ records per minute retained}$$

- **Justification:** the stream is infinite, so not everything can be stored. A decaying window keeps **recent data important**, needs **constant memory** (one running sum), and still tracks trends.

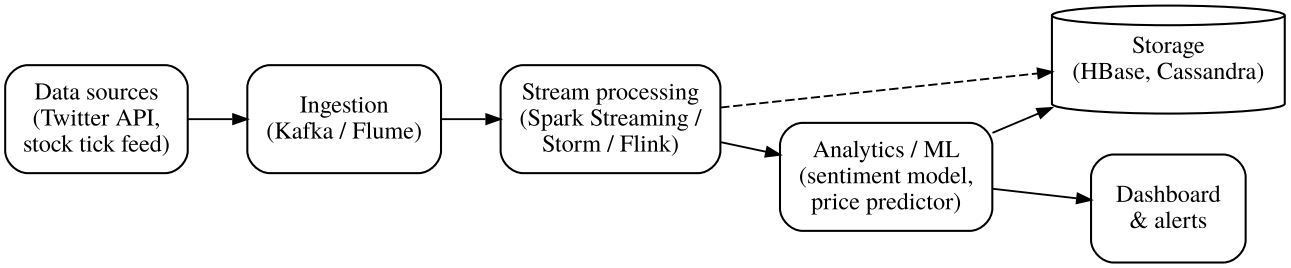
(c) Sliding window (size 200, slide 50, 1000 elements):

$$\frac{N - W}{S} + 1 = \frac{1000 - 200}{50} + 1 = 17 \text{ windows}$$

- **Latency:** a new result comes every 50 elements, so results are **fresher (low latency)**. But the windows overlap by 150 elements, so each element is processed up to 4 times (**more computation**).

Q5. RTAP applications: sentiment analysis and stock prediction case study. *[7 Marks]*

Ans. A **Real-Time Analytics Platform (RTAP) ingests, processes and analyses streams within milliseconds**, so decisions can be made immediately.



Real-time analytics pipeline

Layers:

1. Sources.
2. **Ingestion:** Kafka, Flume.
3. **Processing:** Spark Streaming, Storm.
4. **ML / analytics.**
5. **Storage:** HBase, Cassandra.

6. **Dashboard and alerts.**

Case study 1: Twitter sentiment analysis

1. Stream tweets for a brand into **Kafka**.
2. **Clean** them: remove URLs and stop-words, then tokenize.
3. **Classify** each tweet as positive, negative or neutral (VADER, Naive Bayes, LSTM).
4. Aggregate in a **sliding window** (% positive in the last 5 minutes).
5. **Alert** the PR team if negative sentiment spikes.

Case study 2: Stock prediction

- Ingest price ticks and news.
- Compute moving averages, RSI and sentiment over windows.
- Predict with ARIMA or LSTM.
- Generate **buy/sell signals in milliseconds**.

Other uses: fraud detection, IoT monitoring, Uber surge pricing.

Batch analytics	Real-time analytics
Stored data, hours of latency	Live data, **milliseconds**
Hadoop, Hive	Kafka, Spark Streaming

UNIT 4: FREQUENT ITEMSETS & CLUSTERING

Section A: 2-Mark Questions

Q1. Apriori property; K-Means vs. Hierarchical clustering. *[2 Marks]*

Ans. Apriori property: *every subset of a frequent itemset is frequent*. Equivalently, **if an itemset is infrequent, all its supersets are infrequent**. This lets the algorithm prune candidates.

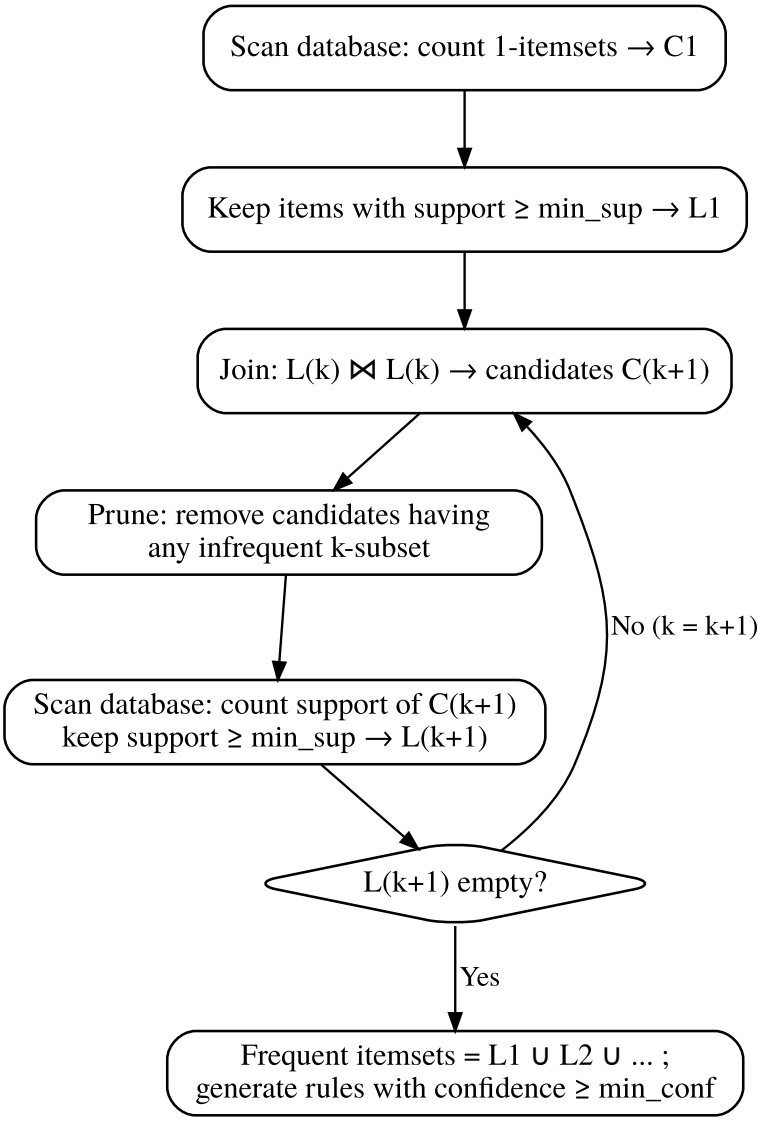
K-Means	Hierarchical
K must be given	No K needed (dendrogram)
Fast, $O(nkt)$	Slow, $O(n^2)$
Depends on the initial centroids	Deterministic

Section B: 7-Mark Questions

Q2. Trace the Apriori algorithm. *[7 Marks]*

Ans. Apriori finds frequent itemsets **level by level**: join L_k to get C_{k+1}, **prune** candidates that have an infrequent subset, count the support, and keep those with support ≥ min_sup.

$$\text{Support}(X) = \frac{\text{count}(X)}{N}, \qquad \text{Confidence}(X \Rightarrow Y) = \frac{\text{count}(X \cup Y)}{\text{count}(X)}$$



Apriori algorithm

(a) Transactions T1 {A,B,C}, T2 {A,B}, T3 {A,C}, T4 {B,C}, T5 {A,B,C}. Minimum support = 60% (count ≥ 3).

Itemset	A	B	C	AB	AC	BC	ABC
Count	4	4	4	3	3	3	**2**
Frequent?	✓	✓	✓	✓	✓	✓	×

- {A,B,C} **passes the subset check** (all its pairs are frequent) but **fails the support check** (2 < 3), so it is pruned.

$$\text{Frequent: } \{A\}, \{B\}, \{C\}, \{A, B\}, \{A, C\}, \{B, C\}$$

(b) Grocery data:

- T10 {Beer, Nuts, Diaper}
- T20 {Beer, Coffee, Diaper}
- T30 {Beer, Diaper, Eggs}
- T40 {Nuts, Eggs, Milk}
- T50 {Nuts, Coffee, Diaper, Eggs, Milk}

Support 50% means count ≥ 3; confidence threshold 60%.

Item	Beer	Nuts	Diaper	Coffee	Eggs	Milk
Count	3	3	4	2 ×	3	2 ×

- Pairs: {Beer, Diaper} = **3** ✓. All the other pairs are ≤ 2.

Rule	Support	Confidence	Strong?
Beer ⇒ Diaper	60%	3/3 = **100%**	✓
Diaper ⇒ Beer	60%	3/4 = **75%**	✓

(The same method applies if your paper has a 6th transaction: just recount.)

Q3. Numerical: K-Means with Manhattan distance (8 points, K = 3). *[7 Marks]*

Ans. Points: A1(2,10), A2(2,5), A3(8,4), A4(5,8), A5(7,5), A6(6,4), A7(1,2), A8(4,9). Initial centres C1 = A1, C2 = A4, C3 = A7. Distance: $d = |x_2 - x_1| + |y_2 - y_1|$.

Iteration 1

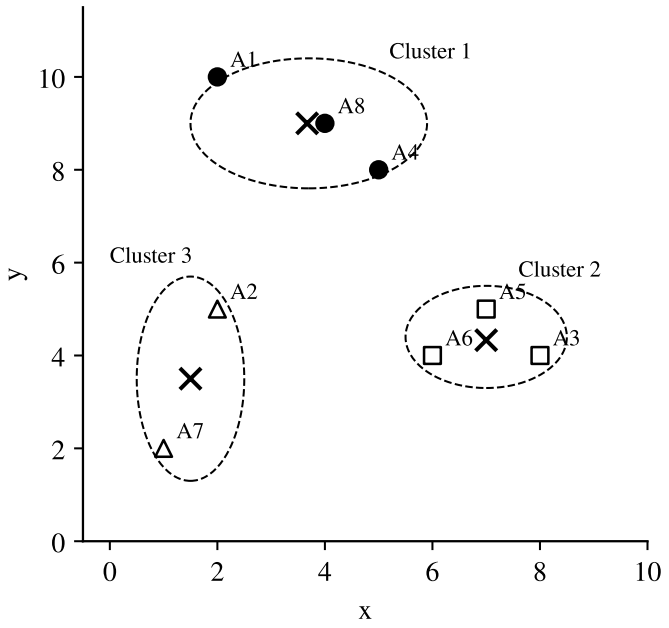
Point	C1 (2,10)	C2 (5,8)	C3 (1,2)	Cluster
A1	0	5	9	C1
A2	5	6	**4**	C3
A3	12	**7**	9	C2
A4	5	**0**	10	C2
A5	10	**5**	9	C2
A6	10	**5**	7	C2
A7	9	10	**0**	C3
A8	3	**2**	10	C2

New centres after iteration 1:

$$C_1 = (2, 10), \quad C_2 = \left(\frac{8+5+7+6+4}{5}, \frac{4+8+5+4+9}{5}\right) = (6, 6), \quad C_3 = (1.5, 3.5)$$

Later iterations:

- Iteration 2: C1 = {A1, A8}, C2 = {A3, A4, A5, A6}, giving centres (3, 9.5) and (6.5, 5.25).
- Iteration 3: C1 = {A1, A4, A8}, C2 = {A3, A5, A6}.
- Iteration 4: **no change**, so the algorithm has converged.

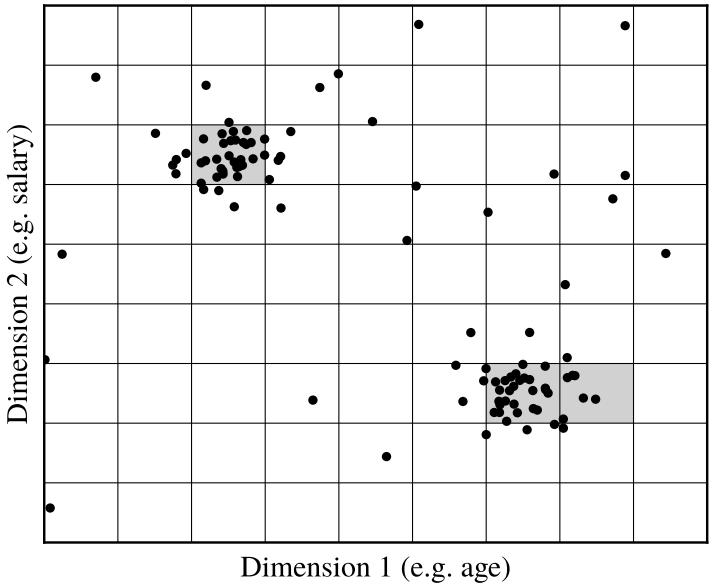


Final three clusters (x = final centre)

$$C_1 = (3.67, 9), \quad C_2 = (7, 4.33), \quad C_3 = (1.5, 3.5)$$

Q4. Explain the CLIQUE and PROCLUS algorithms. *[7 Marks]*

Ans. High-dimensional data suffers from the **curse of dimensionality**: clusters exist only in **subspaces**. **CLIQUE** is a **grid and density, bottom-up** method. **PROCLUS** is a **k-medoid, top-down projected** method.



CLIQUE: dense grid units (shaded) in 2-D; dense units in both 1-D projections suggest candidate dense cells

CLIQUE:

1. Split each dimension into ξ **intervals**, forming a grid.
2. A unit is **dense** if its share of points is greater than τ.
3. Use an **Apriori-like** bottom-up search: if a k-D unit is dense, all its (k−1)-D projections are dense.
4. A cluster = **connected dense units**.

5. Describe each cluster as a **DNF rule**, for example $(30 \le \text{age} < 50) \wedge (4 \le \text{salary} < 8)$.

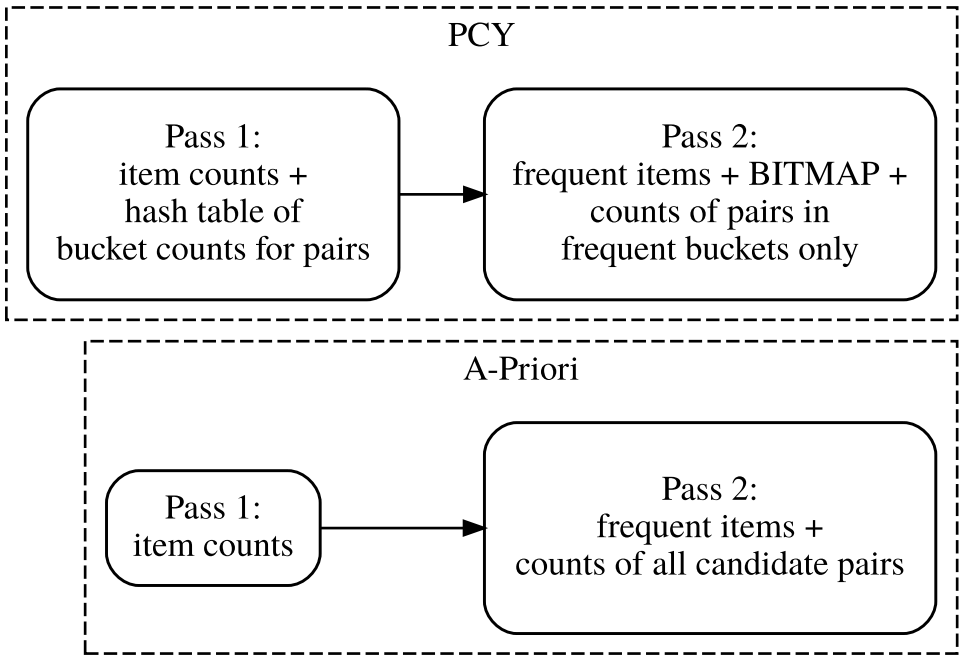
PROCLUS:

1. Choose **k medoids** from a sample, greedily, so that they are well separated.
2. For each medoid, find the **relevant dimensions** (those where its locality is closest).
3. Assign each point using the **Manhattan segmental distance** over those dimensions.
4. Replace bad medoids and repeat.
5. Refine, and mark **outliers**.

CLIQUE	PROCLUS
Bottom-up, grid-based	Top-down, medoid-based
Inputs: ξ, τ	Inputs: k, l
Clusters may overlap	Disjoint clusters
Arbitrary shapes	Spherical in their subspace

Q5. Handling large datasets in main memory: limited-pass algorithms. *[7 Marks]*

Ans. When baskets do not fit in memory, the **second pass (counting pairs)** is the bottleneck. **Limited-pass** algorithms reduce the passes and the memory needed.



Main-memory use in A-Priori vs. PCY

1. **PCY:**
   - In pass 1, also **hash every pair into buckets**. Turn the counts into a **bitmap**.
   - In pass 2, count a pair only if **both items are frequent and its bucket is frequent**.
2. **Multistage:** a second hash table in an extra pass, which gives fewer false positives.
3. **Multihash:** several hash tables in the same pass.

4. **Random sampling:** run Apriori **in memory on a sample** with a lowered threshold. False positives and negatives are possible.
5. **SON (2 passes):** find the frequent itemsets **per chunk**, then count those candidates over the full data. **No false negatives**, and it works with **MapReduce**.
6. **Toivonen:** use a sample plus the **negative border**. If no border itemset is frequent, the answer is exact. Otherwise, repeat with a new sample.

Algorithm	Passes	Exact?
PCY	2	Yes
SON	2	Yes
Sampling	1	No
Toivonen	1 to 2	Yes (if it succeeds)

UNIT 5: FRAMEWORKS, HADOOP & R

Section A: 2-Mark Questions

Q1. Sharding; NoSQL; data types in R. *[2 Marks]*

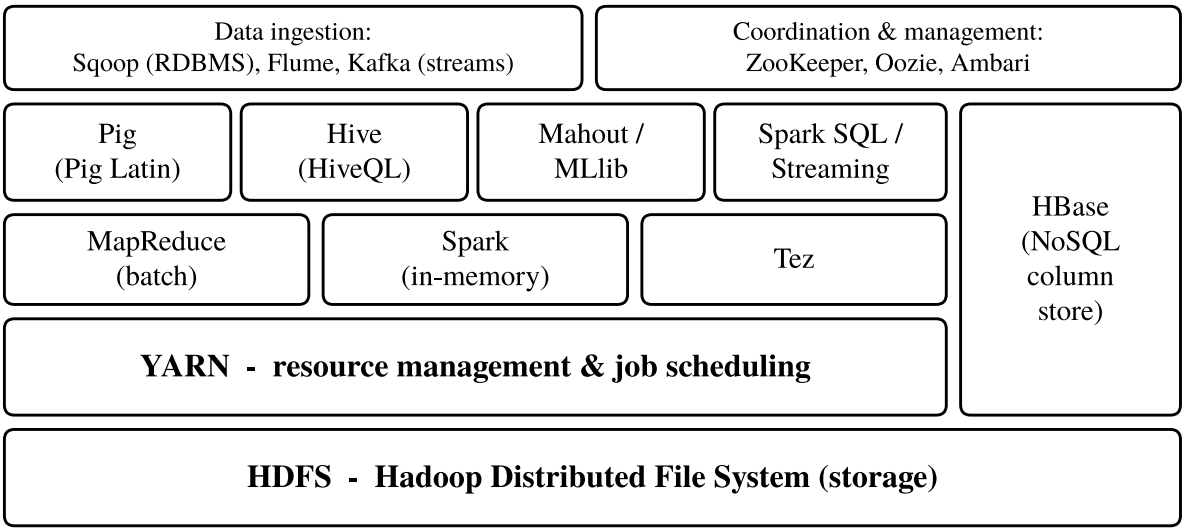
Ans.

- **Sharding: horizontal partitioning** of the rows across servers using a **shard key**. It gives horizontal scaling (MongoDB, Cassandra).
- **NoSQL:** a non-relational, **schema-flexible, horizontally scalable** database that follows **BASE**. Types: key-value (Redis), document (MongoDB), column (HBase), graph (Neo4j).
- **R data types:**
  - numeric `12.5`, integer `12L`, character `"AKTU"`, logical `TRUE`, complex `3+2i`, raw.
  - Structures: vector, matrix, list, factor, **data frame**.

Section B: 7-Mark Questions

Q2. Explain the Hadoop ecosystem and the HDFS architecture. *[7 Marks]*

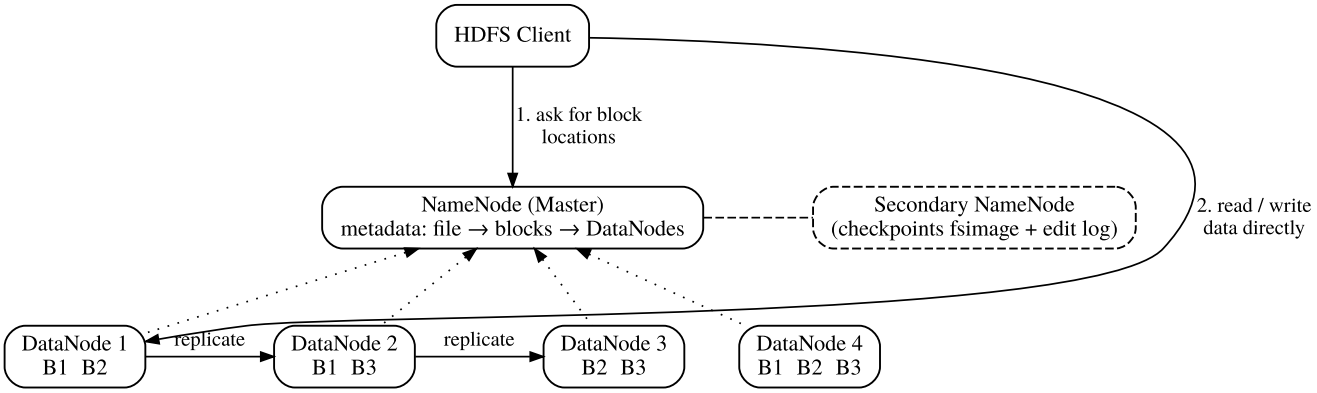
Ans. Hadoop is an open-source framework for the **distributed storage (HDFS) and processing (MapReduce / YARN)** of big data on commodity hardware.



Hadoop ecosystem (layered view)

Ecosystem:

- **HDFS** (storage), **YARN** (resources), **MapReduce / Spark** (processing).
- **Hive** (SQL), **Pig** (scripts), **HBase** (NoSQL).
- **Sqoop / Flume / Kafka** (ingestion), **ZooKeeper / Oozie** (coordination).



HDFS architecture (block size 128 MB, replication factor 3)

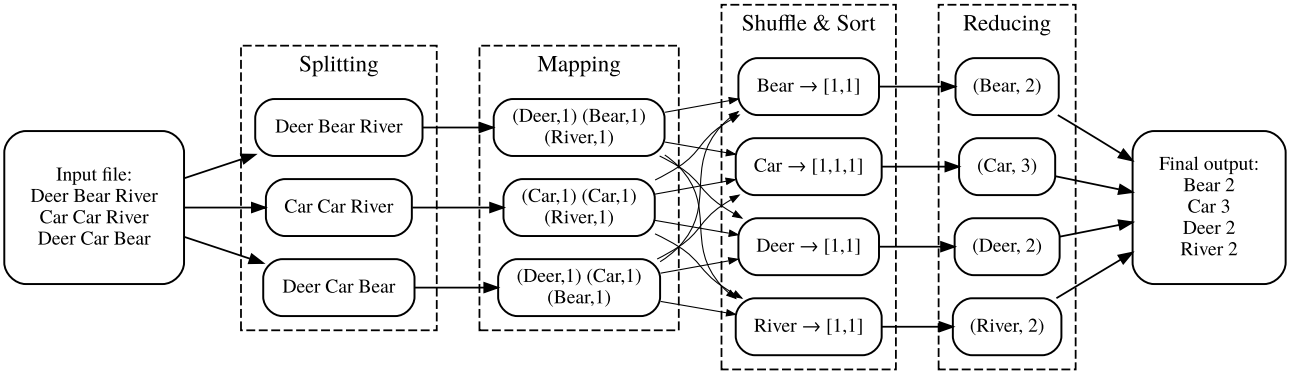
HDFS architecture:

1. **NameNode (master):** holds the **metadata** (file → blocks → DataNodes). It does not store data.
2. **DataNodes (slaves):** store the **blocks** and send **heartbeats** every 3 seconds.
3. **Secondary NameNode: checkpoints** (merges the fsimage and the edit log). It is not a backup.
4. **Blocks: 128 MB** each, with **replication factor 3** and rack awareness.
5. **Write:** the client asks the NameNode, then writes to the DataNodes in a **pipeline**. **Read:** the client gets the locations, then reads from the nearest DataNode.

HDFS	Normal file system
Distributed	Single machine
128 MB blocks	4 KB blocks
Replication for fault tolerance	RAID / backups
Write once, read many	Random read and write

Q3. Explain the MapReduce framework with a diagram. *[7 Marks]*

Ans. MapReduce processes big data **in parallel**. **Map** turns input into **(key, value)** pairs, and **Reduce aggregates the values for each key**.



MapReduce data flow for Word Count

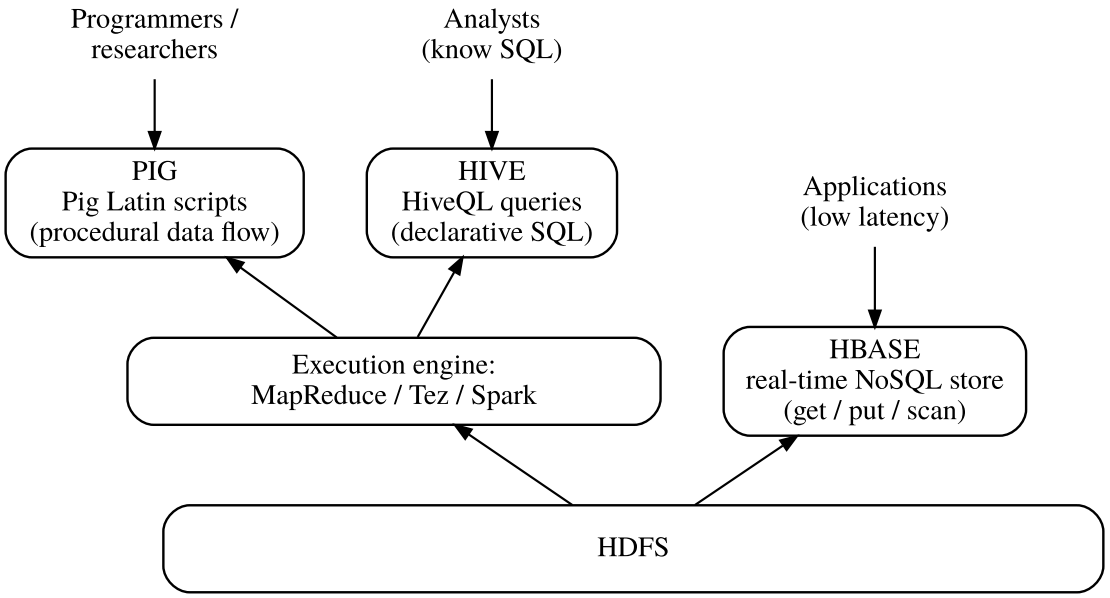
1. **Splitting:** the input is divided into splits (one per 128 MB block).
2. **Map:** `map(k1, v1) → list(k2, v2)`. For word count, emit (word, 1).
3. **Combiner** (optional): local aggregation, which reduces network traffic.
4. **Partitioner:** `hash(key) mod R` chooses the reducer.
5. **Shuffle and sort:** group the values by key, for example (Car, [1, 1, 1]).
6. **Reduce:** `reduce(k2, list(v2))`, giving (Car, 3).
7. **Output** is written to HDFS.

```
map(line):      for each word w: emit(w, 1)
reduce(w, L):   emit(w, sum(L))
```

Map	Reduce
Filter and transform	Aggregate
Many tasks	Fewer tasks
Output on local disk	Output in HDFS

Q4. Differentiate between Pig, Hive and HBase. *[7 Marks]*

Ans.



Where Pig, Hive and HBase sit in Hadoop

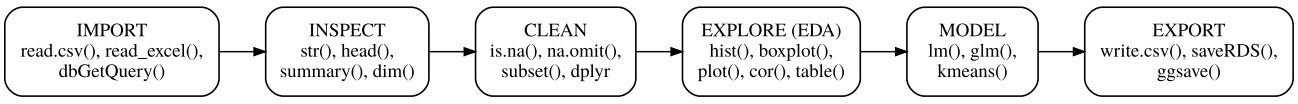
Basis	Pig	Hive	HBase
Type	Data-flow scripting	Data warehouse (SQL)	**NoSQL** column store
Language	**Pig Latin** (procedural)	**HiveQL** (declarative)	get / put / scan API
Processing	Batch	Batch	**Real-time**
Latency	High	High	**Milliseconds**
Schema	Optional	Schema-on-read	Flexible columns
Updates	No	Limited	**Row-level**
Best for	**ETL**	**Reports, BI**	**Online apps**
Developed by	Yahoo	Facebook	Based on Google BigTable

- **Pig example:** `grp = GROUP logs BY user; c = FOREACH grp GENERATE group, COUNT(logs);`
- **Hive example:** `SELECT region, SUM(amount) FROM sales GROUP BY region;`

Conclusion: in a typical pipeline, Pig cleans the data, Hive reports on it, and HBase serves it in real time.

Q5. Data import/export and EDA using R. *[7 Marks]*

Ans. R is a statistical language. **EDA (Exploratory Data Analysis)** summarizes and visualizes data before modelling.



Data analysis workflow in R

```r
df <- read.csv("students.csv")                  # import CSV
library(readxl); x <- read_excel("marks.xlsx")  # import Excel
write.csv(df, "clean.csv", row.names = FALSE)  # export
str(df); head(df); summary(df); dim(df)        # structure and summary
colSums(is.na(df))                             # missing values
mean(df$marks); sd(df$marks); table(df$branch) # statistics
cor(df[, 2:4])                                 # correlation
hist(df$marks); boxplot(marks ~ branch, df)    # plots
plot(df$hours, df$marks)                       # scatter plot
```

EDA steps:

1. Understand the structure.
2. Univariate statistics and plots.
3. Bivariate analysis and correlation.
4. Find missing values and outliers.
5. Form hypotheses.

Conclusion: R is widely used for statistics, research and finance analytics.

Constitution of India, Law & Engineering

AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** in law and constitution answers, **Article numbers, Acts, years and case names earn marks**. Write them in every answer. Draw at least one flow chart or hierarchy diagram per 7-mark answer.

UNIT 1: CONSTITUTION & FUNDAMENTAL RIGHTS

Section A: 2-Mark Questions

Q1. Objectives of the Directive Principles of State Policy. *[2 Marks]*

Ans. The **DPSPs** are in **Part IV (Articles 36 to 51)**, borrowed from the **Irish** Constitution. They are **non-justiciable** (Article 37) but **fundamental in governance**. **Objectives:**

1. A **welfare state** with social and economic democracy (Articles 38, 39).
2. **Equal pay for equal work** (Article 39(d)) and **free legal aid** (Article 39A).
3. **Gandhian ideals:** panchayats (Article 40), prohibition (Article 47).
4. A **Uniform Civil Code** (Article 44), the **environment** (Article 48A), and **separation of the judiciary from the executive** (Article 50).

Q2. Constitutionalism and the unitary bias of the Constitution. *[2 Marks]*

Ans.

- **Constitutionalism** means **government power is limited by the constitution**, through the rule of law, separation of powers, judicial review and Fundamental Rights.
- **Unitary bias:** India is **quasi-federal** (K.C. Wheare), with a **strong Centre**:
  - A "Union of States" (Article 1).
  - Single citizenship.
  - Parliament can change state boundaries (Article 3).
  - Residuary powers lie with the Union (Article 248).
  - Emergency provisions.
  - Governors are appointed by the Centre.
  - All-India Services.

Q3. What are Fundamental Duties? *[2 Marks]*

Ans. Moral duties of citizens in **Part IV-A, Article 51A**.

- Added by the **42nd Amendment (1976)** on the recommendation of the **Swaran Singh Committee**. The idea comes from the **USSR**.

- There were 10; the **11th** (education for children aged 6 to 14) was added by the **86th Amendment (2002)**.
- Examples: respect the Constitution, the Flag and the Anthem; protect the environment; develop a scientific temper.
- They are **non-justiciable**.

Section B: 7-Mark Questions

Q4. Historical background and formation of the Constituent Assembly. *[7 Marks]*

Ans. The **Constituent Assembly** framed the Constitution of India. It was formed under the **Cabinet Mission Plan (1946)**, adopted the Constitution on **26 November 1949**, and the Constitution came into force on **26 January 1950**.



From British rule to the Constitution of India

Historical background:

1. **Regulating Act 1773:** the first control over the East India Company.
2. **GoI Act 1858:** power passed to the **British Crown**.
3. **Morley-Minto Reforms 1909:** separate electorates.
4. **Montagu-Chelmsford 1919: dyarchy** in the provinces.
5. **GoI Act 1935:** federation and provincial autonomy. **Most of our Constitution is based on it.**
6. The demand for a Constituent Assembly was raised by **M.N. Roy (1934)** and by the **INC (1935)**. It was accepted in the **August Offer (1940)**.

Formation:

- **389 members** (296 from British India, 93 from the princely states). They were **indirectly elected** by the provincial assemblies, about 1 seat per 10 lakh people.
- After Partition, the number fell to **299**.

Working:

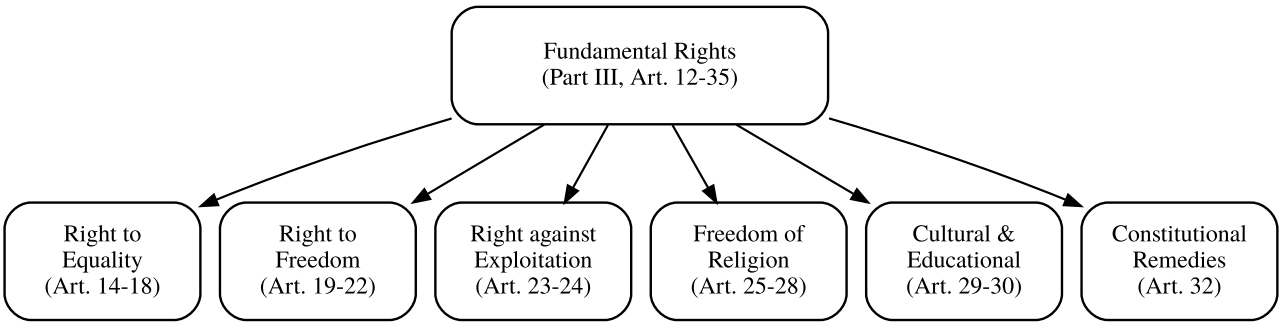
- **First meeting: 9 December 1946.** President: **Dr. Rajendra Prasad**.
- **Objectives Resolution:** moved by **Nehru** (13 December 1946); it became the Preamble.
- **Drafting Committee: Dr. B.R. Ambedkar** (29 August 1947). Adviser: **B.N. Rau**.
- It took **2 years, 11 months and 18 days**.
- The original Constitution had **395 Articles, 22 Parts and 8 Schedules**.

Criticism: not directly elected, dominated by the Congress, too lengthy.

Conclusion: the Assembly gave India the **world's longest written constitution**.

Q5. Explain Fundamental Rights and how they differ from other legal rights. *[7 Marks]*

Ans. Fundamental Rights are the **basic rights guaranteed in Part III (Articles 12 to 35)**, inspired by the **US Bill of Rights**. They are **justiciable**: a person can go **directly to the Supreme Court under Article 32**.



Classification of Fundamental Rights (Part III)

1. **Right to Equality (14 to 18):** equality before law, no discrimination, equal opportunity, **abolition of untouchability (17)**.
2. **Right to Freedom (19 to 22):**
   - **Article 19:** six freedoms.
   - **Article 21:** life and personal liberty.
   - **Article 21A:** education.
3. **Against Exploitation (23 and 24):** no forced labour; no child labour below 14.
4. **Freedom of Religion (25 to 28).**
5. **Cultural and Educational Rights (29 and 30):** for minorities.
6. **Constitutional Remedies (32):** the **five writs**: habeas corpus, mandamus, prohibition, certiorari, quo warranto. Ambedkar called it the "**heart and soul**" of the Constitution.

Key points:

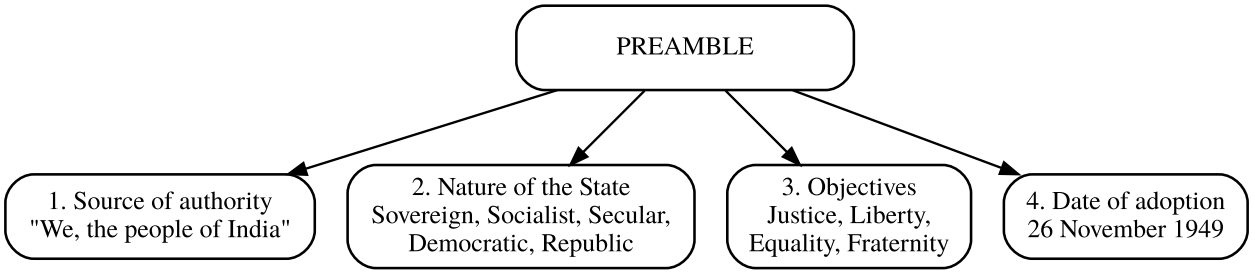
- The Right to Property was removed by the **44th Amendment**. It is now a legal right under **Article 300A**.
- **Articles 20 and 21 cannot be suspended**, even during an emergency.
- FRs can be amended, but **not the basic structure** (Kesavananda Bharati, 1973).

Fundamental Rights	Ordinary legal rights
Come from the **Constitution**	Come from **ordinary laws**
Direct remedy in the SC / HC (Articles 32, 226)	Through ordinary courts
Changed only by a **constitutional amendment**	Changed by an ordinary law
Mainly against the **State**	Mainly against individuals
Example: Article 21	Example: Article 300A (property)

Conclusion: Article 21 now includes the **right to privacy (Puttaswamy, 2017)**, a clean environment and livelihood.

Q6. Discuss the Preamble of the Indian Constitution. *[7 Marks]*

Ans. The **Preamble** is the **introduction** to the Constitution. It is based on Nehru's **Objectives Resolution**. It has been **amended only once**, by the **42nd Amendment (1976)**, which added **Socialist, Secular and Integrity**.



Four components of the Preamble

1. **"We, the people of India":** the **people are the source of authority**.
2. **Sovereign:** independent of any external power.
3. **Socialist:** democratic socialism, ending inequality.
4. **Secular:** no State religion; all religions treated equally.
5. **Democratic:** an elected government with adult franchise.
6. **Republic:** an **elected head of State**.
7. **Objectives:**
    - **Justice:** social, economic, political (from the **Russian Revolution**).
    - **Liberty:** of thought and expression.
    - **Equality:** of status and opportunity.
    - **Fraternity:** dignity and unity (from the **French Revolution**).

Is it a part of the Constitution?

- **Berubari (1960):** No.
- **Kesavananda Bharati (1973): Yes**. It can be amended, but not its basic structure.
- **LIC case (1995):** Yes.

> **Remember:** the Preamble is **non-justiciable**. N.A. Palkhivala called it the "**identity card**" of the Constitution.

Conclusion: the Preamble is the **soul of the Constitution** and guides its interpretation.

UNIT 2: UNION & STATE EXECUTIVE

Section A: 2-Mark Questions

Q1. Define Union Executive and State Executive. *[2 Marks]*

Ans.

- **Union Executive (Articles 52 to 78):** the **President** (nominal head), the Vice-President, the **PM and Council of Ministers** (real executive) and the Attorney General.
- **State Executive (Articles 153 to 167):** the **Governor** (nominal head), the **CM and Council of Ministers** (real executive) and the Advocate General.

Q2. Explain Judicial Review. *[2 Marks]*

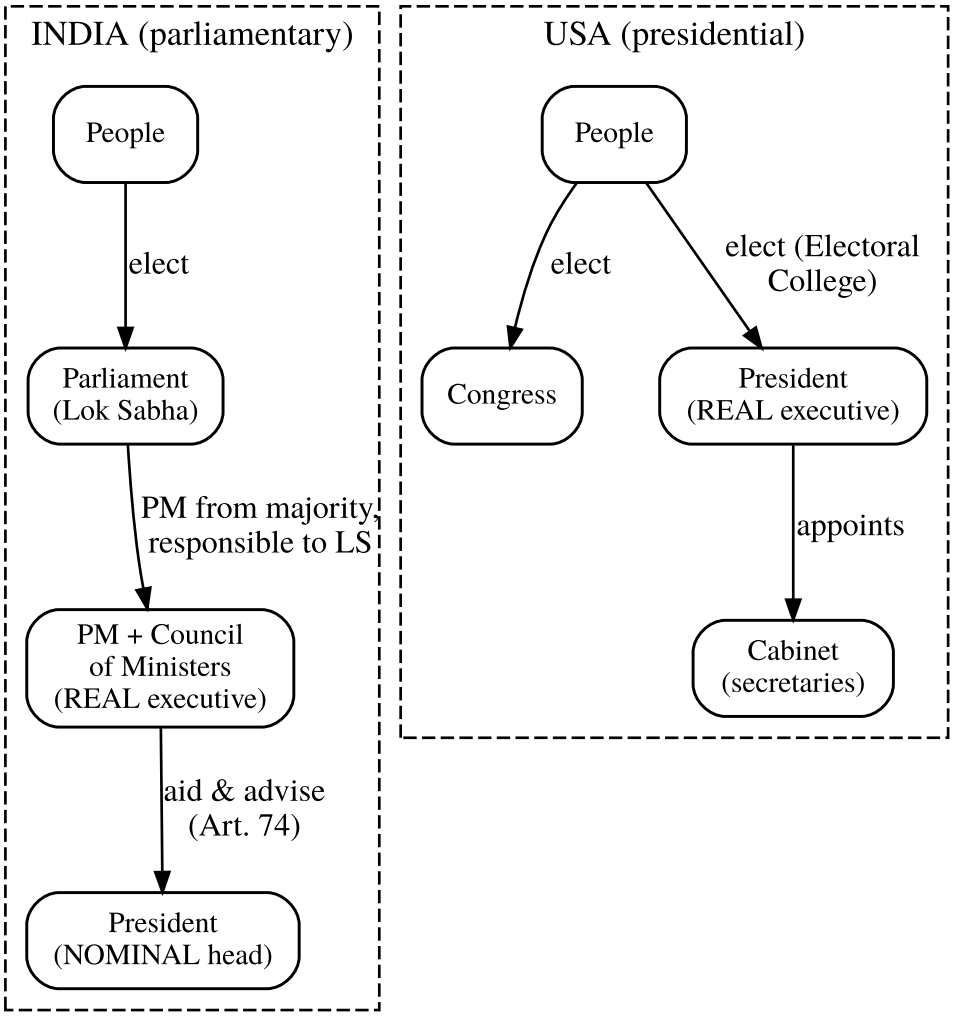
Ans. Judicial review is the **power of the courts to declare laws and executive actions void if they violate the Constitution**.

- **Basis:** Articles 13, 32, 226. **Origin:** Marbury v. Madison (USA, 1803).
- It is part of the **basic structure** (Kesavananda 1973, Minerva Mills 1980), so it cannot be removed.

Section B: 7-Mark Questions

Q3. Compare the powers of the President of India and the President of the USA. *[7 Marks]*

Ans. India has a **parliamentary system** in which the **President is the nominal head**. The USA has a **presidential system** in which the **President is the real executive**.



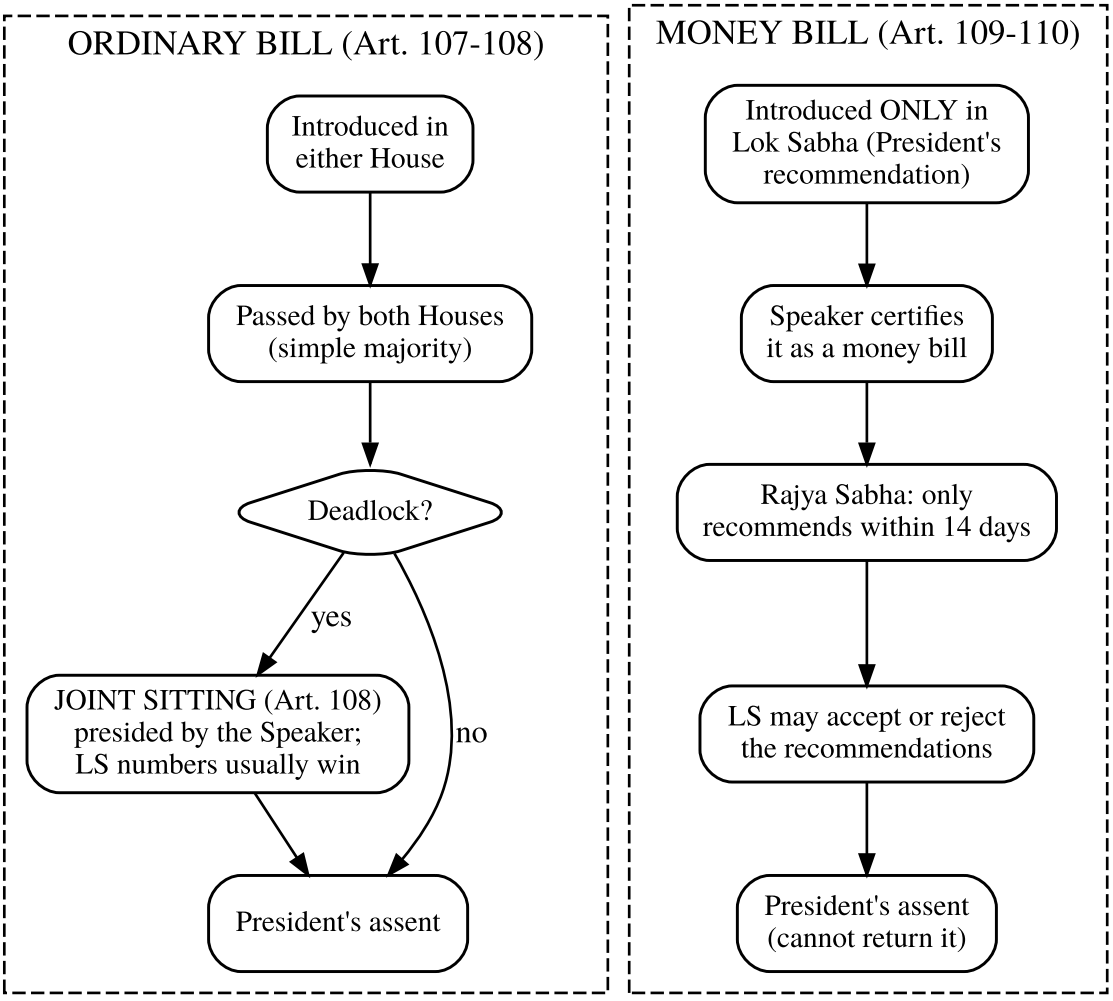
Parliamentary system (India) vs. presidential system (USA)

Basis	President of India	President of USA
Position	**Nominal** head	**Real** head of State and Government
Election	Indirect: elected MPs and MLAs (Articles 54, 55)	Electoral College (near-direct)
Term	5 years, no limit	4 years, **maximum 2 terms**
Advice of ministers	**Binding** (Article 74)	Not binding
Ministers	MPs	Not members of Congress
Dissolve the legislature	Lok Sabha (on the PM's advice)	Cannot dissolve Congress
Veto	Absolute, suspensive, pocket	Qualified (overridden by 2/3), pocket
Ordinance	**Yes** (Article 123)	No
Emergency	**Articles 352, 356, 360**	None
Removal	Impeachment for violation of the Constitution (Article 61), 2/3 of each House	House impeaches, **Senate (2/3)** convicts, for treason, bribery or high crimes

Conclusion: the Indian President "**reigns but does not rule**". The US President both reigns and rules.

Q4. Legislative and financial powers of the Lok Sabha. *[7 Marks]*

Ans. The **Lok Sabha** (Article 81) is the **directly elected** House, with a **maximum of 550 members** (543 elected seats now) and a term of 5 years. It has **more power than the Rajya Sabha**, especially over **money**.



Ordinary bill vs. money bill: role of the Lok Sabha

Legislative powers:

1. **Ordinary bills:** can be introduced in either House. A deadlock goes to a **joint sitting (Article 108)**, presided over by the **Speaker**. The Lok Sabha's numbers usually win.
2. **Amendments (Article 368):** equal power with the Rajya Sabha (no joint sitting).
3. Approves ordinances within **6 weeks**. Elects the Speaker, and takes part in the President's election.

Financial powers:

1. A **money bill (Article 110)** can be introduced **only in the Lok Sabha**.
    - The **Speaker certifies** it.
    - The Rajya Sabha can only **recommend within 14 days**.
2. **Budget (Article 112):** the **demands for grants are voted only by the Lok Sabha** (Article 113).
3. Passes the **Appropriation Bill (Article 114)** and the Finance Bill. Cut motions; vote on account.

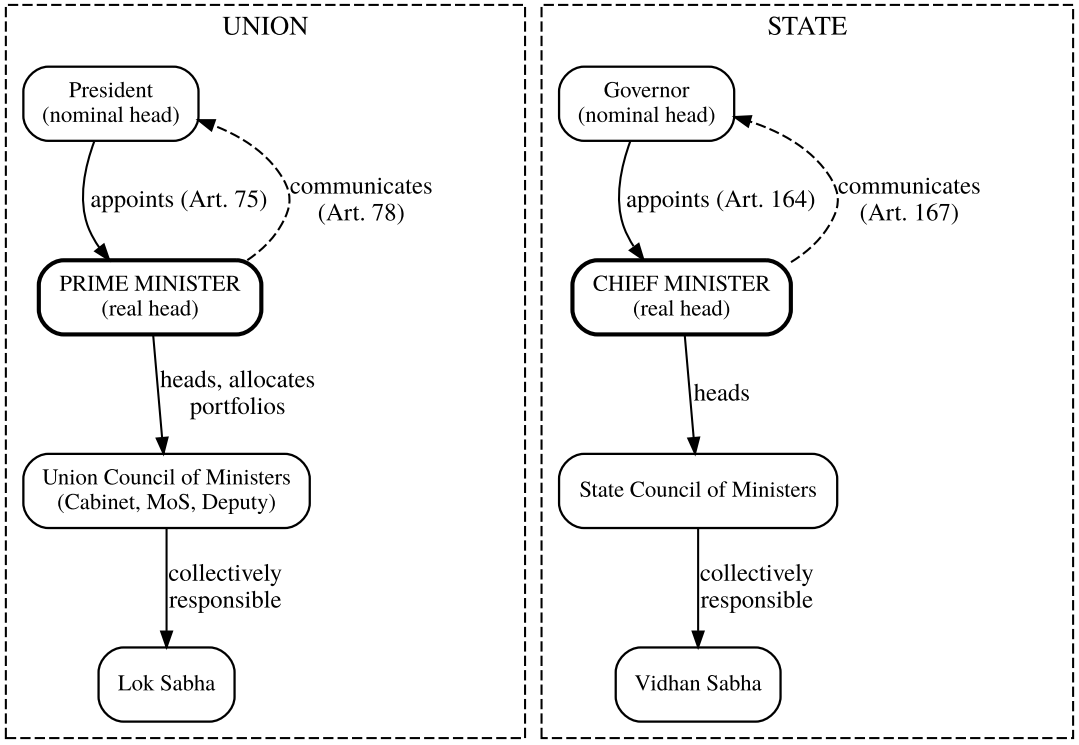
Control over the executive:

- The **Council of Ministers is collectively responsible to the Lok Sabha** (Article 75(3)).
- A **no-confidence motion** can be moved only in the Lok Sabha.

Power	Lok Sabha	Rajya Sabha
Money bill	**Introduces and passes**	Only recommends (14 days)
No-confidence motion	**Yes**	No
Joint sitting presided by	**Speaker**	-

Q5. Powers of the Prime Minister and the Chief Minister. *[7 Marks]*

Ans. The **PM** (Articles 74, 75, 78) and the **CM** (Articles 163, 164, 167) are the **real executive heads** at the Union and State levels.



Position of the PM and the CM in the executive

Powers of the PM:

1. **Council of Ministers:**
   - Recommends ministers and **allocates portfolios**.
   - Chairs the Cabinet.
   - **The PM's resignation dissolves the whole Council.**
2. **President:** is the **channel of communication (Article 78)**. Advises on key appointments.
3. **Parliament:** is the leader of the House. Advises **dissolution of the Lok Sabha**.
4. **Other: chairman of NITI Aayog**; leads foreign policy; crisis manager.

Powers of the CM: the same roles at the State level:

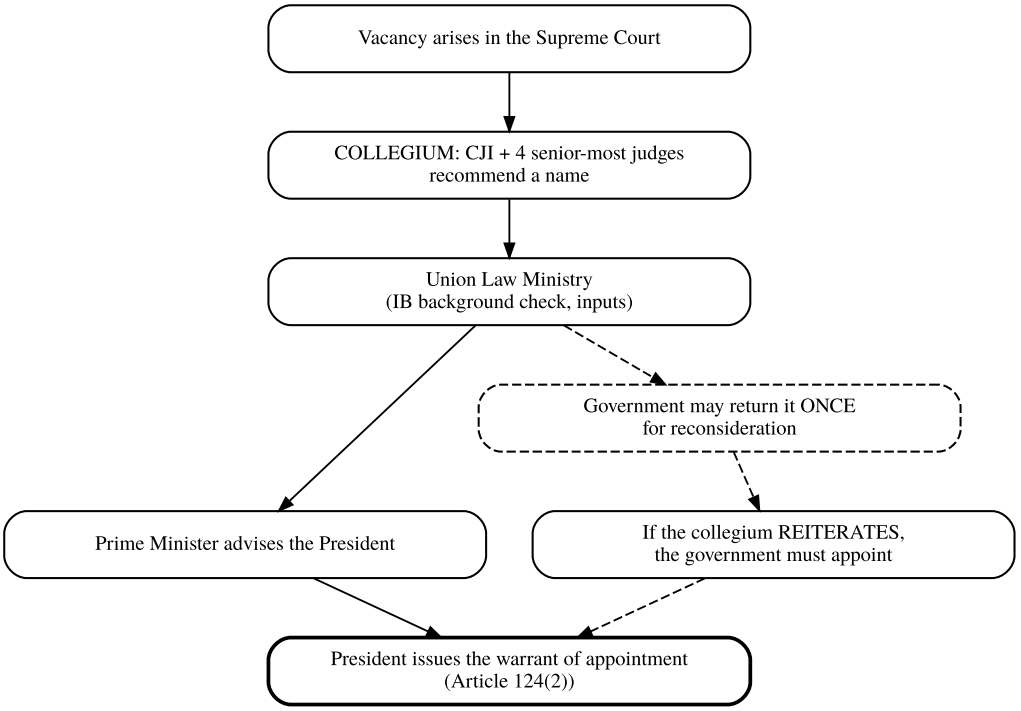
- Ministers and portfolios.
- **Channel to the Governor (Article 167).**
- Advises the dissolution of the Vidhan Sabha.

- Member of the NITI Aayog Governing Council and the Inter-State Council.

PM	CM
Appointed by the President (Article 75)	Appointed by the Governor (Article 164)
Responsible to the Lok Sabha	Responsible to the Vidhan Sabha
Article 78	Article 167

Q6. Independence of the Supreme Court and the appointment of judges. *[7 Marks]*

Ans. The **Supreme Court** (Articles 124 to 147) has **34 judges including the CJI**. It is the **guardian of the Constitution**. **Judicial independence is part of the basic structure.**



Appointment process under the collegium system (Memorandum of Procedure)

Appointment:

- **Article 124(2):** judges are appointed by the President after consultation.
- **Qualifications:** a High Court judge for 5 years, a High Court advocate for 10 years, or a distinguished jurist.
- **Judges cases:**
  - **First (1981):** the executive had primacy.
  - **Second (1993):** the **collegium system was created**.
  - **Third (1998):** the collegium became the **CJI + 4 senior-most judges**.
  - **NJAC (99th Amendment, 2014): struck down in 2015**.

Safeguards for independence:

1. **Tenure until 65.** Removal only by **special majority in both Houses** for proved misbehaviour or incapacity (Article 124(4)).
2. **Salaries charged on the Consolidated Fund**, and **not reducible** (except in a financial emergency).

3. **No practice after retirement** (Article 124(7)).
4. **Conduct not discussed** in Parliament (Article 121).
5. **Contempt power** (Article 129). Own staff (Article 146).
6. **Separation from the executive** (Article 50).

Criticism: the collegium is **opaque**; appointments are **delayed**; post-retirement posts may affect independence.

UNIT 3: LEGAL SYSTEM & WORKPLACE LAW

Section A: 2-Mark Questions

Q1. Key provisions of the RTI Act, 2005. *[2 Marks]*

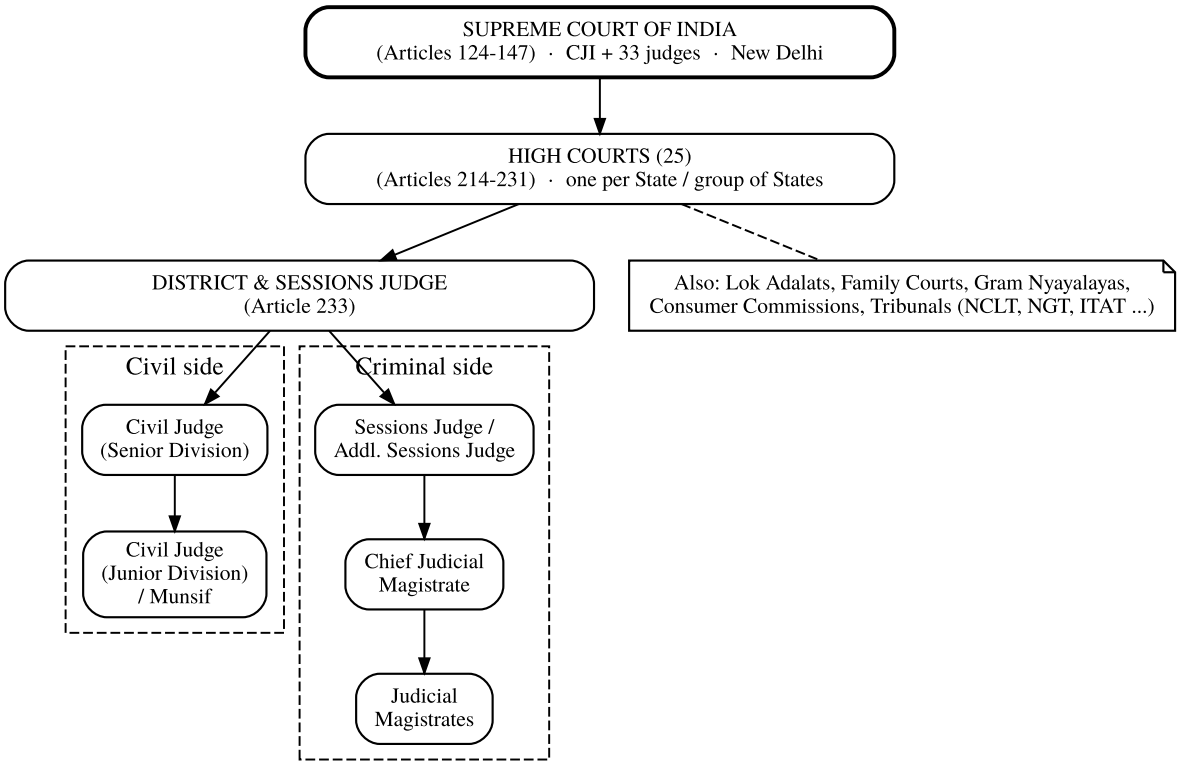
Ans. The RTI Act came into force on **12 October 2005**. Any **citizen** can ask a **public authority** for information.

1. **Section 4:** suo motu disclosure.
2. **Section 6:** apply to the **PIO** with a **Rs 10 fee** (no fee for BPL); **no reason needed**.
3. **Section 7:** reply within **30 days**, or **48 hours** for matters of life or liberty.
4. **Section 8:** exemptions (security, cabinet papers, personal information).
5. **Section 19:** first appeal, then a second appeal to the **Information Commission**.
6. **Section 20:** a **penalty of Rs 250 per day, up to Rs 25,000**.

Section B: 7-Mark Questions

Q2. Structure of the Indian court system: District Courts to Supreme Court. *[7 Marks]*

Ans. India has a **single, integrated, hierarchical judiciary**. Appeals go **upward**, and the **decisions of higher courts bind the lower courts** (Article 141).



Hierarchy of courts in India

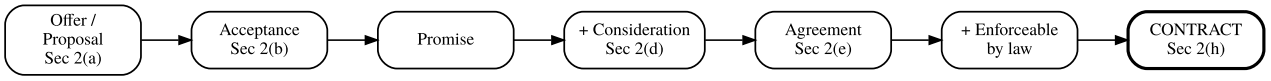
1. **Supreme Court:**
   - **Original jurisdiction (Article 131):** Centre vs. State disputes.
   - **Writs (Article 32).**
   - **Appeals (Articles 132 to 136**, including the SLP).
   - **Advisory (Article 143).**
   - **Court of record (Article 129).**
2. **High Courts (25):**
   - **Writs (Article 226)**, which are **wider** than Article 32 (they also cover legal rights).
   - **Supervision of the lower courts (Article 227).**
   - Judges retire at 62.
3. **District and Sessions Judge (Article 233):** the main court of the district, appointed by the **Governor with the High Court**. A death sentence needs **High Court confirmation**.
4. **Civil side:** Civil Judge (Senior Division), then Civil Judge (Junior Division) / Munsif.
5. **Criminal side:** Chief Judicial Magistrate, then Judicial Magistrates.
6. **Others: Lok Adalats** (no fee; the award is final), Family Courts, Gram Nyayalayas, Tribunals (NCLT, NGT), Consumer Commissions.

Supreme Court	High Court	District Court
Articles 124 to 147	Articles 214 to 231	Articles 233 to 237
Retire at 65	Retire at 62	State rules
Writs under Article 32	Writs under Article 226	No writs

Conclusion: e-Courts, NJDG and Lok Adalats are reducing the pendency of cases.

Q3. Contract law and the essentials of a valid contract. *[7 Marks]*

Ans. Contract law is governed by the **Indian Contract Act, 1872**. Under **Section 2(h)**, a **contract = an agreement enforceable by law**. "All contracts are agreements, but not all agreements are contracts."



Formation of a contract

Essentials (Section 10):

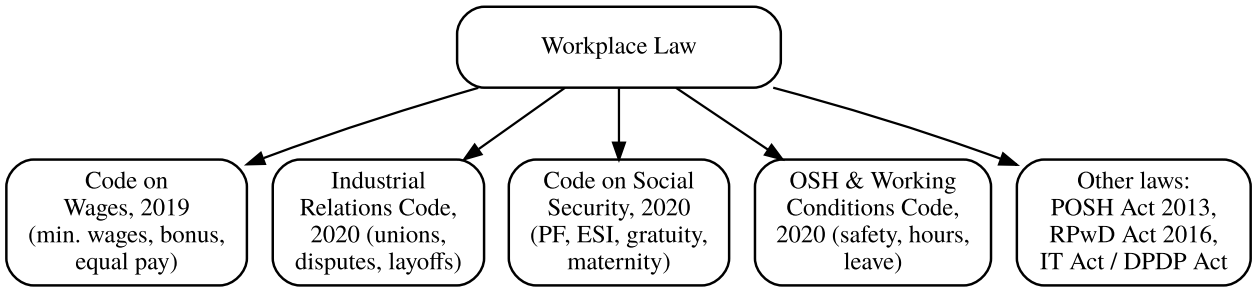
1. **Offer and acceptance:** the acceptance must be **absolute and communicated** (Sections 2(a), 2(b), 7).
2. **Intention to create legal relations:** social agreements do not count (*Balfour v. Balfour*).
3. **Lawful consideration:** "quid pro quo" (Sections 2(d), 25).
4. **Competent parties (Section 11):** a major, of sound mind, not disqualified. An agreement with a **minor is void** (*Mohori Bibee*, 1903).
5. **Free consent (Section 14):** no **coercion (15), undue influence (16), fraud (17), misrepresentation (18)** or mistake. Otherwise the contract is **voidable**.
6. **Lawful object (Section 23).**
7. **Not expressly void (Sections 24 to 30):** for example restraint of trade (27) or wagering (30).
8. **Certainty (Section 29)**, **possibility of performance (Section 56)**, and the **legal formalities** required by law.

Void	Voidable	Illegal
Not enforceable at all	Enforceable at one party's option	Forbidden by law
Example: with a minor	Example: consent by coercion	Example: smuggling

Remedies: damages (Section 73), specific performance, injunction.

Q4. Employee rights, employer responsibilities and workplace law. *[7 Marks]*

Ans. Workplace law governs the employer-employee relationship. India has **consolidated its labour laws into 4 Labour Codes (2019-2020)**, supported by Constitutional Articles 14 to 16, 21, 23, 24 and 39 to 43.



Framework of workplace law in India

Employee rights:

1. **Fair wages:** minimum wages, timely payment, **equal pay** (Code on Wages, 2019).
2. **A safe workplace**, fixed hours, leave (OSH Code, 2020).
3. **Social security:** PF, ESI, **gratuity (after 5 years)**, **maternity leave of 26 weeks**.
4. **No harassment:** under the **POSH Act 2013** (*Vishaka* guidelines), an **Internal Committee** is required if there are 10 or more employees.
5. **No discrimination.** The right to form **unions** (Article 19(1)(c)). Due process before termination. Data privacy (DPDP Act 2023).

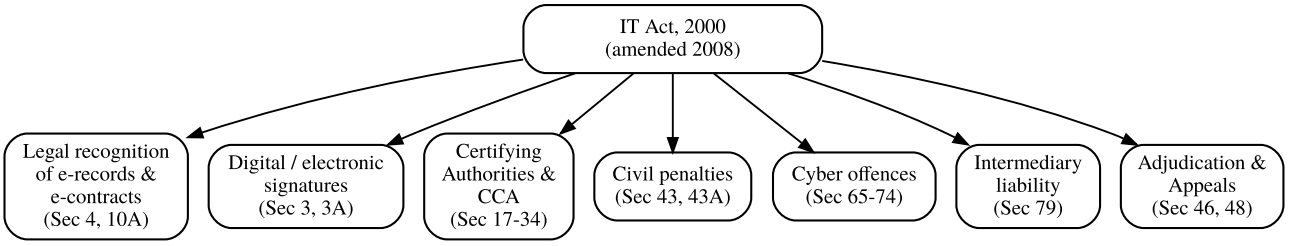
Employer responsibilities:

1. Issue an **appointment letter** with clear terms.
2. **Pay on time**, and deposit **PF, ESI and TDS**.
3. Provide a **safe workplace** and training.
4. Run a **POSH committee** and an equal-opportunity policy.
5. Maintain **records**; follow **due process** for discipline; protect employee data.

Area	Employee right	Employer duty
Wages	Minimum, equal pay	Pay on time
Safety	Safe workplace	Safety measures
Dignity	No harassment	POSH committee
Security	PF, ESI, gratuity	Deposit contributions

Q5. Explain the Information Technology Act, 2000. *[7 Marks]*

Ans. The **IT Act, 2000** is India's law on **e-commerce and cybercrime**. It is based on the **UNCITRAL Model Law (1996)**, came into force on 17 October 2000, and was **amended in 2008**.



Main components of the IT Act, 2000

Objectives: legal recognition of **e-records, e-contracts and digital signatures**; **e-governance**; punishment of **cybercrime**.

Key sections:

- **3:** digital signature. **3A:** electronic signature. **4:** e-records. **10A:** e-contracts.
- **Sections 17 to 34:** the **CCA** licenses the Certifying Authorities.
- **43:** compensation for damage to a computer. **43A:** failure to protect data.

Section	Offence	Punishment
65	Tampering with source code	3 years
66	Hacking (dishonest)	3 years / Rs 5 lakh
66C	**Identity theft**	3 years + Rs 1 lakh
66D	**Phishing / cheating by personation**	3 years + Rs 1 lakh
66E	Privacy violation	3 years
66F	**Cyber terrorism**	**Life imprisonment**
67	Obscene content	3 to 5 years

- **Section 66A** was **struck down** in *Shreya Singhal (2015)*.
- **69 / 69A:** interception and website blocking. **70B: CERT-In. 79:** intermediary safe harbour.

Conclusion: the IT Act supports e-filing, online banking and Aadhaar e-Sign. Data protection is now covered by the **DPDP Act 2023**.

UNIT 4: ELECTIONS, EMERGENCIES & AMENDMENTS

Section A: 2-Mark Questions

Q1. Ordinary Bill vs. Constitutional Amendment Bill. *[2 Marks]*

Ans.

Ordinary Bill	Amendment Bill
Articles 107, 108	**Article 368**
Simple majority	**Special majority** in each House
Joint sitting possible	**No joint sitting**
President may return it	President **must assent** (24th Amendment)
No State ratification	Half the States ratify federal provisions

Q2. When can a National Emergency be declared? Types of amendment under Article 368.

[2 Marks]

Ans.

- **National Emergency (Article 352):** for **war, external aggression or armed rebellion** ("armed rebellion" replaced "internal disturbance" through the 44th Amendment). It needs the Cabinet's

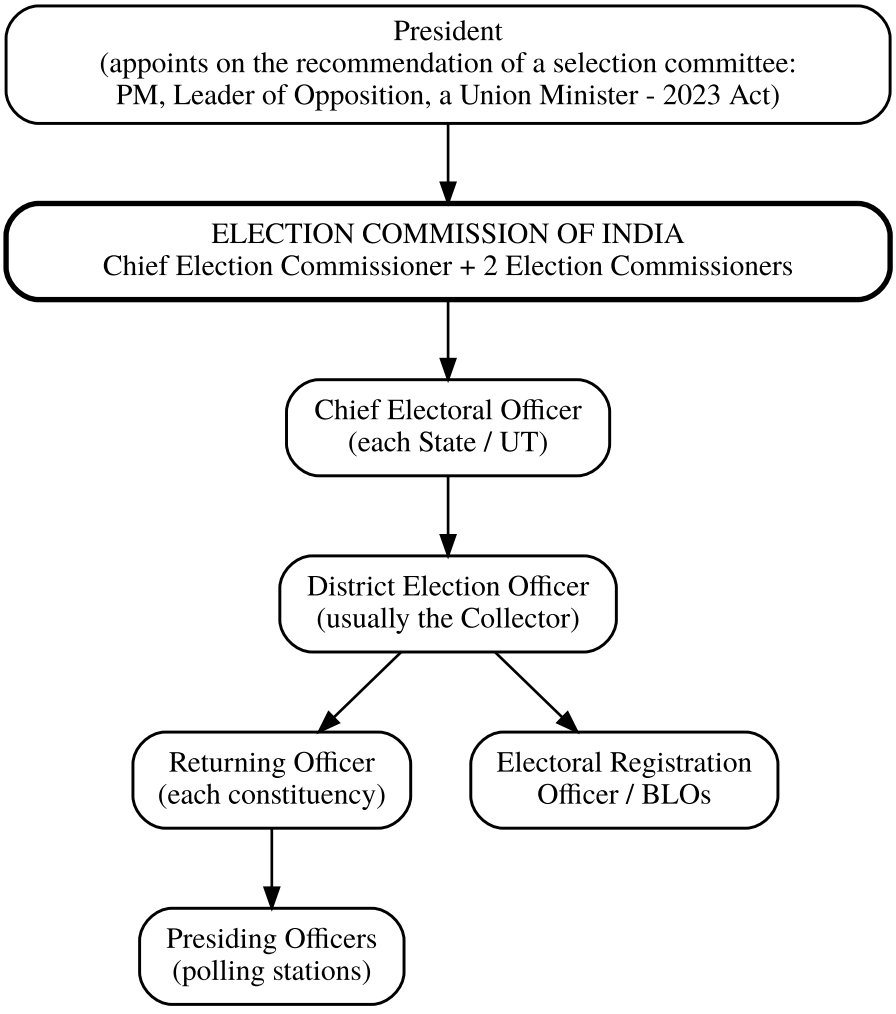
written advice and **approval within 1 month by special majority**. Declared in **1962, 1971 and 1975**.

- **Types of amendment:**
  1. **Simple majority** (Articles 2 to 4, citizenship).
  2. **Special majority** (FRs, DPSPs).
  3. **Special majority + ratification by half the States** (federal provisions, Article 368 itself).

Section B: 7-Mark Questions

Q3. Composition, powers and functions of the Election Commission of India. *[7 Marks]*

Ans. The **ECI** is a **permanent, independent constitutional body under Article 324**, set up on **25 January 1950** (National Voters' Day). It conducts elections to **Parliament, the State Legislatures, and the offices of the President and Vice-President**.



Structure of election machinery in India

Composition:

- **CEC + 2 Election Commissioners** (multi-member since 1993). Decisions by **majority**.
- **Appointment:** by the President, on a **selection committee's** recommendation (the PM, the Leader of the Opposition and a Union Minister), under the **2023 Act**.

- **Term: 6 years or until age 65**.
- **Removal:** the **CEC like a Supreme Court judge**; the ECs only on the CEC's recommendation.

Functions:

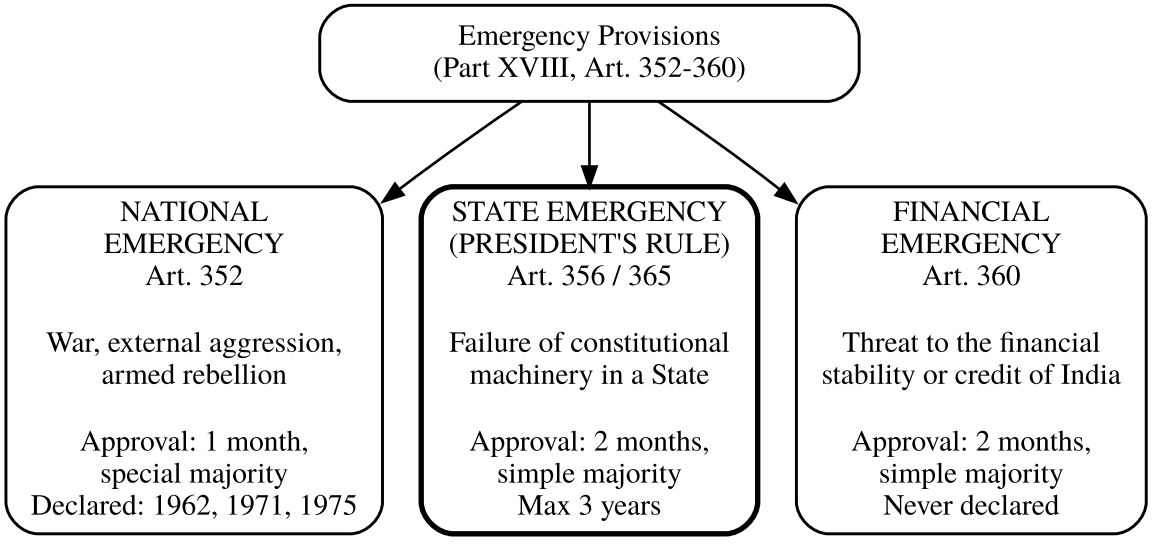
1. **Administrative:**
   - **Electoral rolls**, voter ID (EPIC), the **election schedule**.
   - **Recognising parties and allotting symbols**.
   - EVM / VVPAT.
   - Expenditure limits and observers.
2. **Regulatory:** enforces the **Model Code of Conduct**; rules on exit polls and paid news.
3. **Quasi-judicial:**
   - Settles party and symbol disputes.
   - Advises the **President (Article 103)** and the **Governor (Article 192)** on **disqualification**.
   - Cancels polls and orders re-polls.
4. **Advisory:** recommends electoral reforms.

Limitations: the MCC is not statutory; the ECI cannot deregister parties; the 2023 committee has a government majority.

Conclusion: the ECI ran the 2024 elections with about **97 crore voters**.

Q4. Three types of emergencies, focusing on President's Rule. *[7 Marks]*

Ans. The **emergency provisions** are in **Part XVIII (Articles 352 to 360)**, borrowed from the **Weimar Constitution of Germany**. During an emergency, the federation becomes **almost unitary**.



Three types of emergency

1. **National Emergency (Article 352):** war, external aggression or armed rebellion. Approved within 1 month by **special majority**. FRs can be suspended (**except Articles 20 and 21**).
2. **President's Rule (Articles 356 and 365):**

- **Grounds:** the President, on the **Governor's report or otherwise**, is satisfied that the **State government cannot be carried on according to the Constitution**. Article 365 covers a State that **does not follow Union directions**.
- **Cases:** a hung assembly; loss of majority with no alternative; a breakdown of the constitutional machinery.
- **Not valid grounds:** maladministration; no **floor test** held; political reasons.
- **Procedure:** approval within **2 months by simple majority**. **6 months at a time, maximum 3 years.** Beyond 1 year, a National Emergency and the **EC's certificate** are needed (44th Amendment).
- **Effect:** the President takes over the State executive; **Parliament makes the State's laws**; the High Court is unaffected.
- **S.R. Bommai (1994): judicial review is allowed**; the **floor test is the only test of majority**; secularism is part of the basic structure.
- First used in **Punjab (1951)**; used **more than 100 times** since.

3. **Financial Emergency (Article 360):** a threat to India's financial stability. **Salaries, including judges', can be reduced.** It has **never been declared**.

Basis	Article 352	Article 356	Article 360
Approval	1 month, special majority	2 months, simple majority	2 months, simple majority
Maximum duration	No limit	**3 years**	No limit
Used	3 times	100+ times	Never

Conclusion: the **44th Amendment and the Bommai judgment** now prevent misuse of these powers.

UNIT 5: BUSINESS ORGANIZATIONS & E-GOVERNANCE

Section A: 2-Mark Questions

Q1. What is a sole trader? Types of shares. *[2 Marks]*

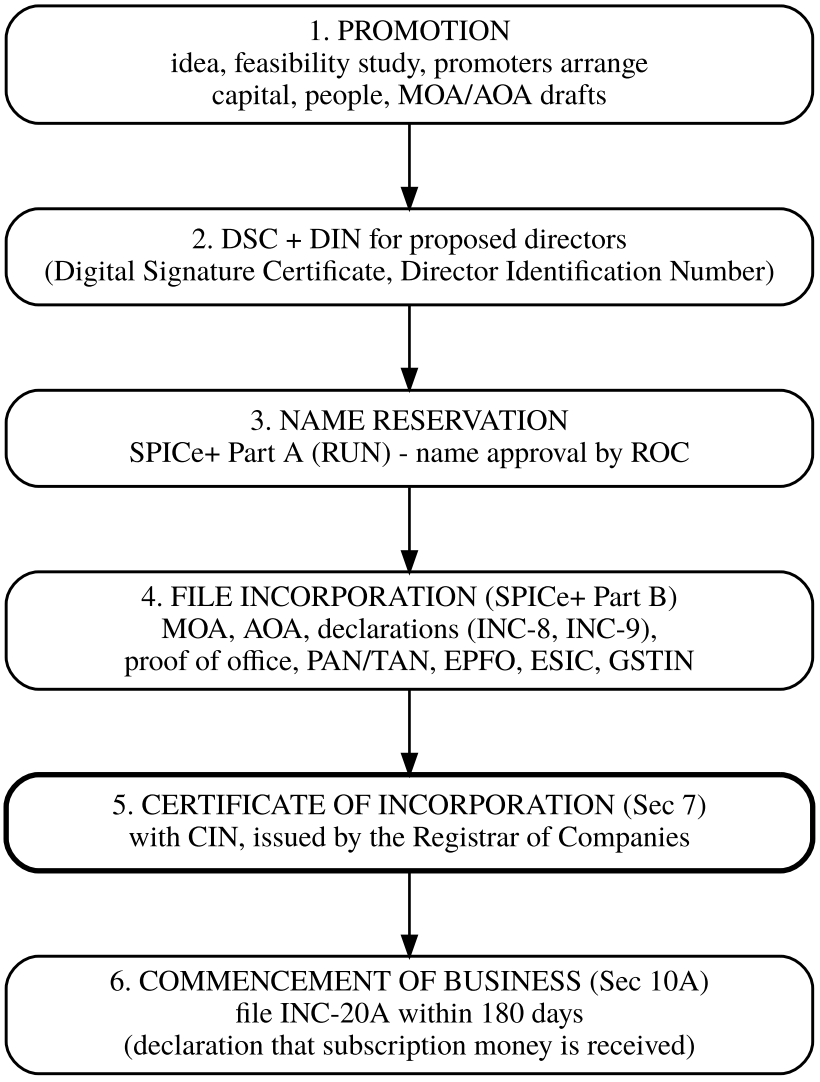
Ans.

- **Sole trader:** a business **owned and controlled by one person**, who takes all the profit and bears **unlimited liability**. It has **no separate legal entity** and is easy to form. Example: a kirana shop.
- **Shares (Section 43, Companies Act 2013):**
  - **Equity shares:** voting rights, dividend paid last, with or without DVR.
  - **Preference shares:** fixed dividend and priority. Types: cumulative, participating, convertible, **redeemable** (Section 55).
  - Also: **bonus (63), rights (62), sweat equity (54)** shares and ESOPs.

Section B: 7-Mark Questions

Q2. Formation of a company under the Companies Act. *[7 Marks]*

Ans. A **company** is an **artificial legal person** (Section 2(20), Companies Act 2013) with a **separate entity, limited liability and perpetual succession**. It is formed **online through SPICe+ on MCA21**.



Stages in the formation of a company

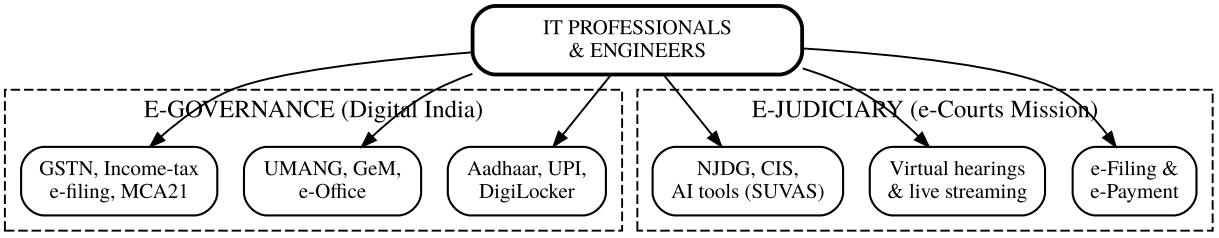
1. **Promotion:**
   - **Promoters** (Section 2(69)) do the feasibility study, arrange capital and choose the type.
   - **Private company:** 2 members, 2 directors. **Public company:** 7 members, 3 directors. **OPC:** 1 member.
2. **DSC and DIN** for the directors.
3. **Name reservation:** SPICe+ Part A (RUN).
4. **Documents:**
   - **MOA (Section 4):** the charter, with the name, registered office, **objects**, liability and capital clauses. The company **cannot act beyond it (ultra vires)**.
   - **AOA (Section 5):** the internal rules.

5. **File SPICe+ Part B** with INC-33 / 34 (e-MOA / e-AOA), INC-8 and INC-9, the office proof and the fees. PAN, TAN, GST, EPFO and ESIC are obtained in the same form.
6. **Certificate of Incorporation (Section 7)** with the **CIN**, issued by the **ROC**. From this date the company is a **legal person**.
7. **Commencement of business (Section 10A):** file **INC-20A within 180 days**.

Private	Public	OPC
2 to 200 members	7 or more members	1 member
2 directors	3 directors	1 director
“Pvt Ltd”	“Ltd”	“(OPC) Pvt Ltd”

Q3. Role of IT professionals and engineers in e-governance and the judiciary. *[7 Marks]*

Ans. E-governance uses ICT to deliver government services (G2C, G2B, G2G). **IT professionals design, build, secure and run** these systems, including the **e-Courts** of the judiciary.



IT professionals at the centre of e-governance and e-judiciary

In e-governance:

1. **Build portals and apps:** UMANG, DigiLocker, **GeM**, CoWIN, Passport Seva.
2. **Digital public infrastructure: Aadhaar, UPI, e-KYC, e-Sign** (India Stack).
3. **Cybersecurity:** PKI, encryption, audits, **CERT-In**, compliance with the **DPDP Act 2023**.
4. **Data analytics and AI** for policy, and fraud checks in DBT.
5. **Networks and cloud:** NIC, SWAN, State Data Centres, MeghRaj, BharatNet.
6. **Accessibility:** multilingual tools (Bhashini) and Common Service Centres.

In the judiciary:

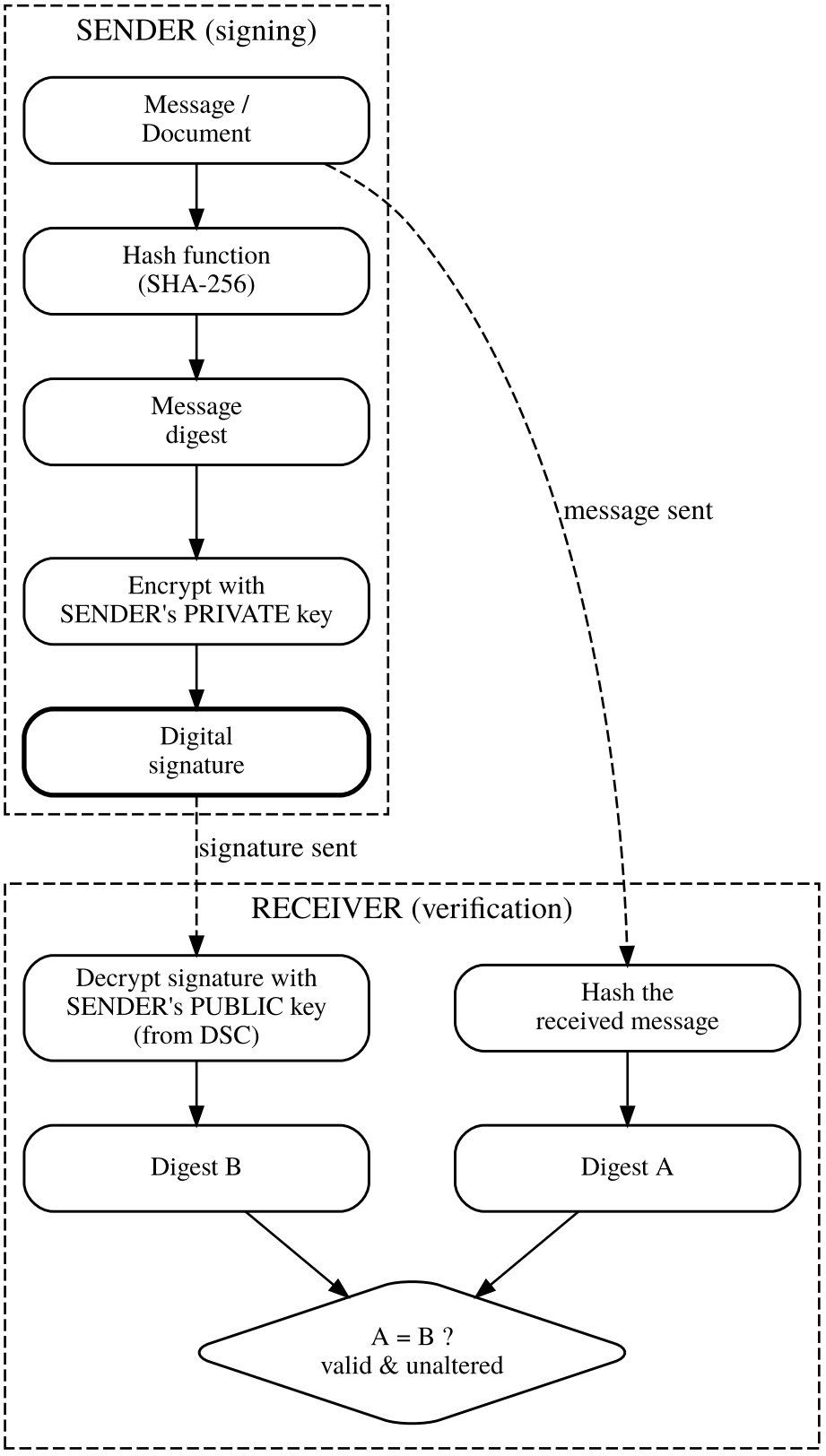
1. **e-Courts project:** CIS and the **NJDG** (case pendency data).
2. **e-Filing and e-payment**, **virtual hearings**, live streaming.
3. **AI: SUVAS** (translation), **SUPACE** (research help).
4. **ICJS** links police, courts and prisons. **Cyber forensics and electronic evidence** (Section 63 of the BSA 2023).

Ethics: privacy by design, secure code, no algorithmic bias.

Traditional	E-governance
Paper, offices	Online, 24 × 7
Slow, less transparent	Fast, audit trails

Q4. Digital Signature Certificates and cyber regulations. *[7 Marks]*

Ans. A **digital signature** authenticates a document using **asymmetric cryptography plus a hash** (Section 3 of the IT Act). A **DSC** binds a person's **identity to their public key**. It is issued by a **Certifying Authority** licensed by the **CCA**.



Digital signature process (sender signs with the private key, receiver verifies with the public key)

Working:

1. Hash the document to get a **digest**.
2. **Encrypt the digest with the private key.** This is the signature.
3. The receiver **decrypts it with the public key** (from the DSC) and hashes the document again.
4. **If the two digests match, the signature is valid.** This proves **authenticity, integrity and non-repudiation**.

DSC:

- **X.509** format: name, public key, validity, the CA's signature.
- **Class 3** (Class 2 was discontinued in 2021). Valid for **1 to 3 years**, on a **USB token**.
- **Uses:** MCA, GST, income tax, e-tender filings.
- **Section 5:** legally equal to a handwritten signature.

Cyber regulations:

1. **IT Act 2000 / 2008.**
2. **SPDI Rules 2011** (Section 43A).
3. **Intermediary Rules 2021.**
4. **CERT-In directions 2022:** report incidents **within 6 hours**.
5. **DPDP Act 2023:** consent, a Data Protection Board, penalties **up to Rs 250 crore**.
6. **BNS 2023 / BSA 2023** (criminal law and evidence).

Digital signature	Electronic signature
PKI with a DSC from a CA	Any reliable method (OTP, Aadhaar e-Sign)
Section 3	Section 3A
Detects any change to the document	Depends on the method

Conclusion: DSCs are **mandatory for MCA and GST filings**. Cyber laws keep India's digital ecosystem safe.